



PLATFORM STRATEGY

INNOVATION THROUGH HARMONIZATION

Gregor Hohpe

An Architect Elevator Guide

With contributions by Michele Danieli
and Jean-Francois Landreau

Platform Strategy

Innovation Through Harmonization

Gregor Hohpe

This book is for sale at <http://leanpub.com/platformstrategy>

This version was published on 2023-12-30



This is a [Leanpub](#) book. Leanpub empowers authors and publishers with the Lean Publishing process. [Lean Publishing](#) is the act of publishing an in-progress ebook using lightweight tools and many iterations to get reader feedback, pivot until you have the right book and build traction once you do.

© 2023 Gregor Hohpe

Contents

About this Book	i
Revision History	viii
Part I: Understanding Platforms	1
1. Standing on the Shoulders of Giants	2
2. The Fab Four of Technology Platforms	14
Part II: A Strategy for Platforms	25
3. Formulating a Strategy	27
4. Becoming a Platform Company	38
5. Mapping Platforms	46
6. The Platform Paradox	56
7. Addendum: “I ACED My Strategy”	66
Part III: In-House Platforms	70
8. In-House IT Platforms	71
9. IT Platform and IT Services Org Are Antonyms	82
10. Mechanisms not Magic	92

CONTENTS

11. Do You Have an Opinion? A mind of your own?	101
12. Making Platform Decisions	109
13. Procuring a Platform	116

Part IV: Designing Platforms 120

14. The 7 “C”s of Platform Quality	121
15. Fruit Salad or Fruit Basket?	125
16. Cantilevered Platforms	131
17. Will Your Platform Float or Sink?	139
18. Beware the Grim Wrapper!	147
19. Build Abstractions Not Illusions	152
20. Failure Doesn’t Respect Abstraction	159

Part V: Implementing Platforms 164

21. Platform Anatomy	165
22. Platform Implementation Choices	170

Part VI: Growing Platforms 174

23. The Cube of Platform Evolution	175
24. The Shape of Platforms	183
25. Charting a Platform Roadmap	192
26. Tiering and Slicing	201

Part VII: Organizing for Platforms	211
27. Platform, Inc.	212
28. The Soft-Side of Platform Teams	221
29. The Customer-Centric Platform Team	230
30. Platform Teams Without Platform	236

About this Book

My prior role as chief architect of a global financial services organization placed me in the middle of several major transformation programs. A major goal of those programs was to speed up software delivery without compromising quality or security, two aspects that were believed to be at odds with each other. Undeterred by existing beliefs, we set out to deploy something called Agile Delivery Platform, which made software delivery “fast and compliant” and placed the broader IT organization on a trajectory toward a modern operational model.

Reflecting back on our work, several important questions remain. Did we truly build a platform as the name suggested, or were we just naming it creatively? Assuming the former, which characteristics or capabilities qualified it as one? Could we have achieved the same results without a platform? Was building on a third-party product the key aspect of providing a platform? Would any “agile delivery mechanism” inherently be a platform? Were related initiatives like the “digital backbone” or the “digital core” also platforms but just forgot to include it in their name?

Deep in our minds, we might have had plausible answers to those questions, but articulating them clearly would have been rather difficult. I’d be the first to admit that we chose the name without deep consideration; we mainly wanted to express both the project’s purpose (to enable agile delivery) and the mechanism (a common platform for all software projects). Both “Agile” and “Platform” also sounded like noble and useful things to us.

A few years later, I benefited from two advancements: Internal Developer Platforms had become a “thing” and I helped build one for the Singapore Government’s GovTech division. Making software delivery more efficient across government agencies, we were convinced that we were actually building a platform, perhaps helped by the inherent fuzziness of the term. However, diving deeper into the design trade-offs of developer platforms also raised a slew of new questions and design decisions to be made. Running our internal platform on top of commercial cloud platforms only added to the list of questions: what aspects should be part of our platform versus the cloud platform? What if the

cloud launches a feature that makes our function obsolete? Who should own the services that our platform provisions? Should our platform handle billing?

Eager to get to the root of it all, I joined a major cloud provider and spent several years working with customers to get the most out of that platform. Although one might expect that powerful cloud services would have made building in-house IT platforms superfluous, I observed exactly the opposite across large customers: most of them were building or deploying some sort of in-house platform, or often multiple: developer platforms, data platforms, business platforms, digital platforms, multi-cloud platforms, and many more. And unsurprisingly, they faced many of the same questions about their platforms.

After assisting many customers on their platform strategy, I took the Architect Elevator further down into the engine room to develop serverless products, which are widely considered the canonical example of a modern compute platform. So, once again I am deep in the mechanics of making platform decisions: what technical details can be safely hidden from the user and which ones should be exposed? How can services be independent while also composing into something that's bigger than just the sum of its parts? Which aspect should be common across services? Is duplicate functionality across services a convenience or a design flaw? It seemed that the deeper I dug, the more questions arose.

Expecting a single book to answer all questions about platforms would be naive. Instead, I set out, aided by formidable contributors, to structure the questions and define decision models that can help readers answer those questions for themselves. After all, they have more information about their undertaking than I would ever have and can therefore make better decisions. That's why you won't find simplistic 1-2-3 recipes in this book. Instead, readers new to the concept will find a smooth on-ramp whereas experienced ones may test and challenge their current thinking with novel viewpoints.

The Lure of Platforms

Over the past decade, platforms have fueled some of the most successful business models, yielding companies with never-before-seen valuations. Although they come in various shapes, all platforms share an “amplifier effect” that makes them enormously successful but deceptively difficult to replicate.

Marketplace platforms get their boost from the “flywheel” effect: more sellers provide a wider selection, thereby attracting more buyers, which in turn attract

more sellers. That's the mechanism that propelled platform powerhouses like eBay, Uber, or Airbnb to success and astronomical valuations. But the magic of the platform business model doesn't end there. None of those companies keep any expensive inventory because the platform's value proposition doesn't depend on directly delivering goods or services but on facilitating an exchange across a broad ecosystem of third parties.

IT also loves a multiplier effect for its investments and therefore values platforms as a shared foundation on top of which diverse solutions can be built. Operating systems and cloud platforms demonstrate that by hiding underlying complexity developers can focus on delivering business value instead of toiling deep down in the engine room. The need for higher levels of developer productivity in fast-moving environments has made in-house platforms a staple of modern IT strategies.

Outside of IT and online marketplaces, platforms already have a fairly long history. The automotive industry, for example, has long employed platforms to better amortize the heavy engineering investments required to build a safe and reliable car. Because few of the engineering elements like engine, transmission, or suspension are visible to buyers, they can be reused across models that differ in styling and interior options to attract different tastes and demographics. Yet, equating platforms with the act of combining all common elements and placing differentiators on top would be far too simplistic.

Building IT Platforms

Lured by the apparent benefits and successes of both platform business models and in-house platforms, many enterprise IT departments find out that building and financing your own platform is far more difficult than it might have initially appeared. Many platform initiatives consume multi-year investments just to be already obsolete or not accepted by users by the time they launched.

Do such misfortunes invalidate the platform concept and value proposition? Surely not. But they highlight that a clear strategy and a deep understanding of the constraints and assumptions that are baked into the platform are paramount. Instead of leaving implementation details for later or, worse yet, with a provider, platforms want to be nurtured and built incrementally with a keen eye on value delivery.

This book lays out the elements of a platform strategy, highlighting critical decisions and trade-offs to help platform teams become major innovation drivers

in enterprises.

What Will I Learn?

This book condenses years of experience building and using platforms into a mental framework for platform development. Covering technical, organizational, and financial aspects, it is structured along the journey that an organization might take toward becoming a platform company.

Part I: Understanding Platforms

When people say “platform”, they likely mean different things. That’s why it’s wise to first look at the history of platforms, catalog different types of platforms, and highlight their benefits.

Part II: A Strategy for Platforms

Building platforms requires significant investment and a clear strategy. That strategy must turn the objectives into an actionable path defined by meaningful decisions.

Part III: In-House Platform

Most IT organizations experience platforms when they set out to build one. This part looks beneath the covers of such platform initiatives to highlight important characteristics.

Part IV: Designing Platforms

Platforms hide complexity but building one isn’t nearly as simple as it looks from the outside. This part illustrates platform design decisions by means of metaphors.

Part V: Implementing Platforms

Diving deeper into the engine room, this part investigates platform anatomies and proposes common platform blueprints.

Part VI: Growing Platforms

Platforms have to be rolled out across the organization. They also require delicate care and feeding over time, so they don't fall victim to excessive entropy or become a bottleneck. This part shows you how.

Part VII: Organizing for Platforms

If you are building platforms, you'll likely need a platform team, which is different from typical application delivery or operation teams. This part describes how to build and run a platform team.

What about the Architect Elevator?

This book is the second title in a growing series of “Architect Elevator Guides”, based on the novel role of architects defined in *The Software Architect Elevator*. Targeting architects and technical decision makers, these guides don't get hung up on buzzwords and don't try to provide simplistic one-size-fits-all answers. Rather, combine decision models that exert architectural rigor with real-life anecdotes from the daily grind of corporate IT.

In line with this approach, the books carry two “warning labels”:



This book doesn't tell you what to do. It teaches you how to come up with the best answer yourself.

You know your situation best. Many things in IT can be copied from Stack Overflow, but strategy isn't one of them. So, instead of giving you a fish, I want to teach you how to fish, i.e. how to analyze your unique situation and selectively compose learnings from other organizations into a suitable strategy.



You are likely to leave with more questions than you came with. But you'll also be better prepared to answer them.

Asking the right questions is a necessary first step for any successful strategy. I hope you enjoy the read and appreciate having more questions!

Do I have to read all chapters in order?

Just like the internet, my books are interlinked with cross-references between chapters and titles. Each chapter stands on its own, allowing you to skip ahead or start with a chapter that's particularly relevant to your situation.

The Architect Elevator guides, despite the title, address a much broader audience than just architects. IT executives and decision makers may skip the technical detail in part IV and V but will hugely benefit from the remaining parts. Folks leading a platform team may focus on parts III, VI, and VII whereas platform developers may skim parts I through III and dive deep into parts IV through VI. Architects will enjoy taking the elevator from the top levels down into the engine room across all parts.

Do I have to read the other books before this one?

The books in the Architect Elevator series are self-contained so that they can be consumed independently. That's why it's totally fine to start with this book. Cloud services are platforms, so you might want to read *Cloud Strategy* also, but you can do so before or after this one. Most books in the series will refer back to the “umbrella” book, *The Software Architect Elevator*, which introduces the way of thinking that we use to dissect complex domains like cloud computing or IT platforms.

There is only minimal duplication across the books, so owning the complete collection is a great asset for any modern IT architect.

What's with the jellyfish?

Each book in the Architect Elevator series for IT leaders and architects features aquatic animals on its cover. Sea and river creatures are incredibly diverse and beautiful, so I expect to run out of content before I run out of images. All photos are my own.

I felt that jellyfish make a suitable metaphor for platforms. Just like IT systems, they carry some complexity but function without a central nervous system (or a very rudimentary one). Unlike some IT landscapes, they are amazingly well-coordinated systems and beautiful to look at.

Getting Involved

My brain doesn't stop generating new ideas just because the book is published, so have a look at my blog to see what's new:

<https://architectelevators.com/blog>

Also, follow me on Twitter or LinkedIn to see what I am up to or to comment on my posts:

<http://twitter.com/ghohpe> <http://www.linkedin.com/in/ghohpe>

To provide feedback and help make this a better book, please join our private discussion group: <https://groups.google.com/d/forum/cloud-strategy-book>

Acknowledgments

Like my prior books, *Platform Strategy* benefited from the involvement of a wide range of people.

Revision History

Thanks to the magic of Leanpub, you're reading a book that's still being written. Buying an unfinished book makes you an awesome person who supports my writing and gives me early feedback. Leanpub is like a mix of Kickstarter and traditional publishing: early supporters enjoy a lower price point and the privilege of seeing material before many others do. They might also find that early releases have a few quirks or unintended features. For electronic books that's not too bad because all updates are always free.

What's There

I wouldn't publish a book that's not worth reading. Many things in this book are ready for your consumption:

- **Content:** 80% of my thoughts are here, even though they could use some refinement. Some chapters might be split or content might shift between them. The final book might have some 250 pages.
- **Structure:** it took me a while to mold a topic as complex and diverse as platforms into a linear storyline. Following the proven format of short chapters structured into well-scoped parts, the main flow and organization of the book should largely remain.

What's Lacking

Here are the things that are missing or incomplete and might make reading a little choppy:

- **Full editing:** while there won't be dragons lurking, you will encounter typos, long-winded sentences, and occasional grammar issues. The book will get a professional copy edit before going to print.

- **Illustrations:** some illustrations are still to come and others are still inconsistent. I am planning to hire a professional illustrator as I did for *Cloud Strategy*.
- **Case studies:** the book will include real-life case studies. However, these take a bit more time because they require PR / legal clearance from the respective organizations.

Your feedback will make this book better. Please join the mailing list referenced in the introduction to comment or provide feedback.

Revisions

December 2023: Major overhaul of Parts VI and VII, adding chapters “Charting a Platform Roadmap” and “Tiering and Slicing”. Added chapter “Cantilevered Platforms” in Part IV.

October 2023: Pre-release with 30 chapters. Parts I-IV are in good shape with the remaining parts still being completed.

Part I: Understanding Platforms

There must be something about platforms. According to a [report compiled in 2020¹](#), 7 of the 10 most valuable companies in 2018 based their success on a platform model. Ten years earlier, in 2008, that number was zero.

This doesn't mean, though, that a company must become a platform to be successful. Many organizations will use existing platforms, such as cloud platforms, managed by a cloud service provider (CSP). Large organizations might also build in-house platforms, which might support a platform business model just as much as a traditional business.

As we quickly discover, the term “platform” is overloaded and can mean different things in different contexts. So, it's beneficial to have a look at what makes platforms unique and when something should actually be called a platform. which flavors of platforms can be found in today's enterprises, and the benefits platforms deliver for your organization.

This part zooms out to consider platforms in the larger context and put more meaning behind the term:

- [Standing on the Shoulders of Giants](#) examines the history and key properties of platforms.
- [The Fab Four of Technology Platforms](#) catalogs different types of platforms that you are likely to encounter.

¹<https://www.thoughtworks.com/perspectives/edition4-platform-strategy-article>

1. Standing on the Shoulders of Giants

Do you really see further or is the air just thinner?

Platforms are a common sight in daily life—they are normally some sort of structure that you stand on, e.g. to catch a train or get a better view. Generally defined as [a raised level surface on which people or things can stand](#)¹, technology platforms also appear to be raising things up a level or two: many successful companies are based on a platform model. Similarly, cloud platforms have democratized access to compute resources and have fundamentally changed how modern IT operates.

Platform effects are not new. Kim et al.² attribute the success of the Roman empire to an open ecosystem strategy that created a platform for the talents of neighboring countries, fueled by the interconnectivity provided by 80,000 kilometers of roads that reduced the friction for interaction between participants.

The most eloquent—and most widely cited—metaphor for using platforms are the words commonly attributed to Sir Issac Newton (although traced back to the 12th century):

If I have seen further, it is by standing on the shoulders of giants.

Many platform businesses have become giants and architects have a keen desire to see further, so let's begin by taking a look at the concept of platforms and their various flavors.

¹<https://www.lexico.com/en/definition/platform>

²Kim, Song, Im, *Platform Strategy: a new paradigm for a changing world*, 2020, World Scientific Publishing

Platforms Elevate

A raised platform elevates you. This applies to train platforms as much as to product or technology platforms: building on top of a technology platform means you don't need to start from scratch but can build on top of what others have created. Using a marketplace platform gives you instant access to a large set of merchants or customers. Building a social media platform can elevate you to the point of having millions of participants follow your content.

As powerful as platforms are, as elusive is their precise definition. In their like-titled book *Platform Strategy*, Kim et al. resort to listing various platform definitions spanning over a decade. More recently, [Zach Church states](#)³ in his course on platform strategy for MIT Sloan's executive education that

A platform strategy is an approach to entering a market which revolves around the task of allowing platform participants to benefit from the presence of others.

Platforms thus act as amplifiers because they allow participants to benefit from the presence of others, creating something that's much bigger than just the sum of the parts.

The inverse is also true:



A platform generates value only through the interaction between its participants.

A platform without participants isn't providing value. It's like a farmers market that's lacking farmers and customers.

To offset your instant enthusiasm for such a powerful model, Church quickly reminds you that building a platform is “really, really hard”. So, before you decide to jump on the platform bandwagon (there's a fundamental flaw in that mix of metaphors that I am intentionally overlooking), you should take a closer look at common types of platforms, their characteristics, and their benefits.

³<https://mitsloan.mit.edu/ideas-made-to-matter/platform-strategy-explained>

Platforms: Faster, Better, Cheaper. Really?

The popularity of platforms in both business and IT domains is easy to grasp once you see the outstanding benefits that platforms deliver. Just like any major buzzword, platforms are also subject to hyperbole and re-labeling, meaning most any supporting service or business may now be considered a platform. My bank is now a financial platform, the local bookstore has become a content platform, and the supermarket has evolved into a marketplace platform. If you take the words from some marketing departments at face value, you might even believe that without a platform your business is utterly doomed.

Let's put our feet firmly on the ground (or the platform, so to speak) and dissect the buzzword to understand what benefits platforms provide in which contexts:

Platforms enable

Platforms generate value by allowing participants to benefit from the presence of others. Those participants can be buyers and sellers in a marketplace who gain access to a wider range of goods and can transact more easily and safely. They can also be users of common technology platforms like cloud computing, which charge for compute services by consumption and therefore encourage experimentation with low or zero fixed costs. They can also be content creators on a social media platform who can cater to and interact with fans globally.

Platforms democratize

Successful platforms make it easy for participants to join thanks to low barriers. For example, modern e-commerce platforms allow independent sellers to join with much less friction than trying to become a supplier to a major supermarket chain. Most influencers and content providers on social media platforms started small and with a very low investment, which would not have been possible in a traditional media model. Likewise, onboarding a new participant carries a near-zero cost for the platform provider: Airbnb doesn't need to build a hotel room to increase its inventory. Such *near-zero marginal cost business models* are a key tenet of fast-scaling digital business model success.

Platforms self-perpetuate

Platforms that enable the exchange of (virtual or physical) goods between sellers and buyers appear to provide some form of *perpetuum mobile*: more buyers make it attractive for sellers to sign up for the platform, which in turn leads to a wider selection, which attracts more buyers.

Walmart has known for a while that having one store carry everything is a promising business model. Modern platforms do so without the endless aisles and long checkout lines.

Platforms accelerate

In an IT context, platforms can take care of common, but laborious tasks (“undifferentiated heavy lifting” in [parlance of a major cloud provider](#)⁴). By making this toil go away, platforms allow its users to focus on innovation and differentiation without being constrained.

Having been sold snake oil for decades, enterprise decision makers might look at these benefits with a healthy dose of skepticism. Should we file platforms along with “no code”, “executable design models”, and “seamless portability” in the large drawer of past technology ideas that not only sounded too good to be true, but also were? Are platforms, in the words of a recent Twitter user, just reinventing the wheel? Not so fast! Platforms are for real and are here to stay.

Established Platform Models

The success of platforms comes at a price: the label is used for numerous, quite different constructs. Although they share some similarities, the mechanisms behind the different types of platforms are quite different. On the upside, platforms aren’t an entirely new concept—it’s just that in IT we always believe we invented everything. Let’s look at a few extraordinarily successful platform examples that have been around for decades, from automotive manufacturing, e-commerce, media, and IT.

Automotive Platforms

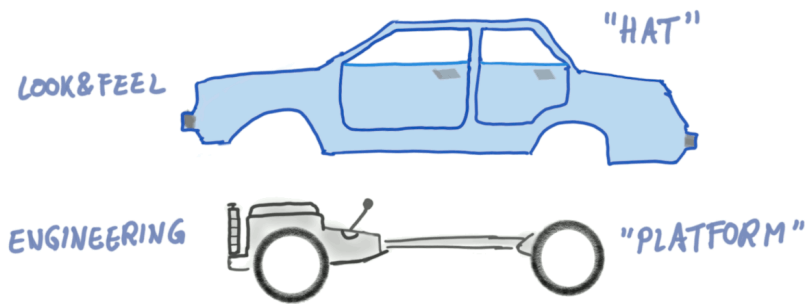
I am somewhat known for car analogies, so I take great pleasure in the fact that automotive manufacturers have been using a platform concept for several decades (see [Wikipedia](#)⁵). Those companies realized that a huge amount of engineering effort went into a vehicle’s technical and safety components, such as engine, transmission, suspension, and anti-lock brakes. However, most of these engineering marvels aren’t really visible to a prospective customer visiting the dealership. While the customer will consider safety ratings, power, and

⁴https://aws.amazon.com/blogs/aws/we_build_muck_s/

⁵https://en.wikipedia.org/wiki/Car_platform

fuel consumption, which do result from the underlying components, purchasing decisions are often made based on the shape of the car, the interior finishing, the suppleness of the seats, or the badge on the trunk (boot).

It was a logical move then, to reduce cost by conducting the base engineering effort once and reusing the chassis and its components across models. Following this strategy, car manufacturers could offer a variety of car models, targeted at different customer segments and often under different brands, without exploding engineering costs. Those cars exhibited a distinct look and feel despite sharing many components “under the hood”.



Automotive platforms separate engineering investments from look and feel

These kinds of platforms are often referred to as “product platforms” or “inner platforms” to distinguish them from external platforms that involve customers or partners. Like most successful platforms, automotive platforms also evolved over time. The initial incarnations reused a complete chassis and many common body parts, leaving relatively little room for customization. Some manufacturers described this approach as placing a “hat” (the car’s body and interior) on top of the shared technical platform.

One of the most successful uses of automotive product platforms can be found at Volkswagen, which started using automotive platforms in the 1970s and evolved the concept into a series of modular platforms, such as the [Volkswagen Group MLB platform](https://en.wikipedia.org/wiki/Volkswagen_Group_MLB_platform)⁶. The name roughly translates into “modular longitudinal toolbox” (longitudinal referring to the engine orientation). This modular toolbox forms the basis for an amazing range of vehicles from the Audi A4 all the way to the Bentley Bentayga SUV (using the so-called MLBevo matrix). Volkswagen’s new MEB platform is looking to achieve the same for electric cars.

Against common belief, standardizing on common platform elements didn’t

⁶https://en.wikipedia.org/wiki/Volkswagen_Group_MLB_platform

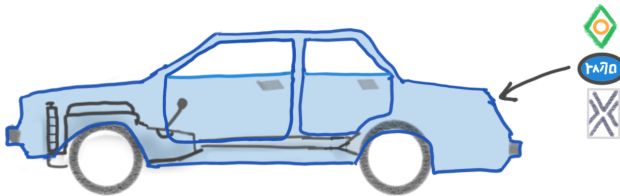
reduce choice or stifle innovation. It actually achieved the exact opposite by increasing product diversity and boosting innovation.



Automotive platforms amortize large engineering investments across a diverse range of models. Harmonizing some elements boosts diversity and innovation in others.

Common platforms also provide better economies of scale for engineering innovations such as anti-lock brakes or autonomous driving. Once again, the standardization boosts innovation. We'll discuss those **magic properties** in detail soon.

Too much of a good thing can become an issue, also with platforms. Automotive platforms teach a lesson about how much can be unified into the base platform and how much differentiation should remain on top of it. Based on the initial success, US car manufacturers took the platform idea to the extreme in the 1980s. They essentially manufactured identical models that differed just in a few options or cosmetic items. At its extreme, only the brand and model badges were different, a technique referred to as “badge engineering”.



Relabeling is not platform engineering

Although tempting, this approach negated a key platform benefit of supporting diverse models and boosting innovation. Unsurprisingly, these attempts largely backfired, immortalized by the Cadillac Cimarron, which was positioned as a luxury car but was virtually indistinguishable from a fully equipped economy-class Chevrolet Cavalier. Its abysmal market response earned it a place on Forbes' list of **Legendary Car Flops**⁷ and its only contribution to the automotive world was lending its name to the “Cadillac Cimarron Effect”.



Automotive history teaches us that both what's in the platforms and what's on top matter. Near-replicas don't work.

⁷<https://carbuzz.com/news/rebadged-disasters-cadillac-cimarron>

A key success factor for platforms is defining those aspects of the final product that can be harmonized versus those that must be kept variable. This thought, and the related challenges, will remain with us for much of the remainder of this book.

Ecommerce Platforms

When speaking about platforms these days, many people refer to digital ecosystems as embodied by companies like Amazon, eBay, Gumroad, Alibaba and the like. These marketplace platforms connect buyers and sellers or people sharing a common interest.

A suitable definition that captures the intention of these business models can be found in Reillier's book, also titled "Platform Strategy"⁸:

A business creating significant value through the acquisition, matching, and connection of two or more customer groups to enable them to transact.

Not all marketplaces are platforms. An analogy from the retail business helps find the delineation. A supermarket connects buyers with goods to be sold while holding its own inventory and being directly involved in the purchasing transaction. In contrast, a farmer's market connects buyers and sellers directly, letting them handle the transaction and maintain inventory. Whereas a farmer's market is a "multi-sided platform"⁹, a supermarket is—strictly speaking—not. Farmer's markets democratize and enable transactions between farmers and consumers (the two sides) whereas supermarkets very actively manage—and control—their supply chain.

E-commerce platforms and marketplaces benefit from significant flexibility in their pricing models:

- They can charge buyers, for example by charging admission to a trade fair or farmer's market. Likewise, online platforms can charge membership fees to buyers, often pegged to premium services like fast or free delivery.

⁸Reillier B. and L., *Platform Strategy: How to Unlock the Power of Communities and Networks to Grow Your Business*, 2017, Routledge

⁹Hagiu, A. (2006). *Multi-sided Platforms: From Microfoundations to Design and Expansion Strategies*, Harvard Business School, working paper 07-094

- They can charge sellers, for example by asking a fee for a farmer to have a stall. In the online world seller fees can be listing fees, transaction fees, membership fees or any combination thereof.
- They can monetize through third parties, for example by inviting sponsors or selling advertising space, both in the physical world or the online counterparts. They can also monetize the data they collect by selling it to third parties.

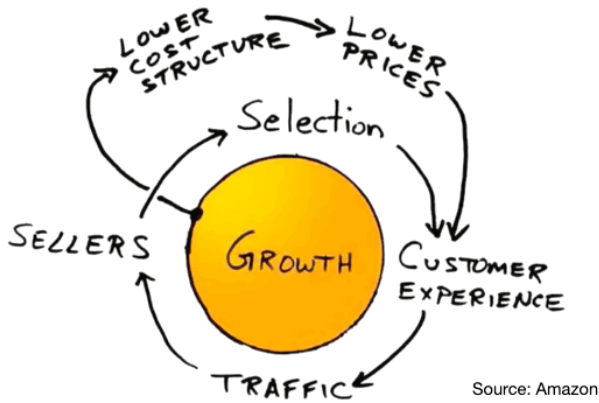
This flexibility allows e-commerce platforms to offer services “for free” to some groups of the multi-sided market. Shifting pricing between participant groups or subsidizing specific types of transactions enables platforms to balance supply and demand to drive growth. Accordingly, changes in pricing models are fairly frequent.

As the platform enables direct interaction between participants, it steps into the background—just like a real-life platform that’s underneath the real action. Some online platforms are barely visible because they allow retailers to operate as white-label shops, e.g. on Shopify. Likewise, when you visit the farmer’s market you might notice the organizer’s presence only by a banner at the entrance and a small organizer’s booth. This is very different from a supermarket where you are keenly aware whose “ecosystem” you are in.

Major e-commerce platforms such as eBay, Amazon (which is careful to label itself as a marketplace), or Airbnb have benefited from the perpetuating scale effects cited at the beginning of the chapter. Amazon refers to this virtuous cycle as [The Flywheel](#)¹⁰, inspired by the model presented by Jim Collins in *Good to Great*¹¹.

¹⁰<https://www.inc.com/jeff-haden/the-1-principle-jeff-bezos-and-amazon-follow-to-fuel-incredible-growth.html>

¹¹Collins, J, *Good to Great: Why Some Companies Make the Leap and Others Don’t*, 2001, HarperBusiness



The Amazon Flywheel

This wheel includes two positive feedback loops: one between the number of buyers and sellers, which fuel each other's participation: more buyers attract more sellers, who carry a wider selection, which attracts more buyers. A secondary loop lowers the cost structure with increasing scale, which in turn allows lower prices, which then further fuel growth. Such feedback loops are the reason why e-commerce platforms often lead to a "winner takes all" scenario where a single platform dominates its respective market segment. That's the power of platforms and positive feedback loops.

Media Platforms

A close relative of eCommerce platforms are social and media / streaming platforms like Facebook, TikTok, Netflix, Twitch, and many others. Like their cousins, they operate a multi-sided market that connects providers and consumers of services, in this case social data (photos, posts, events) or media like audio and video streams. They benefit from a similar flywheel effect that attracts consumers to the platform with the most content, which in turn attracts content providers because they'll get more eyeballs. They may lack the outer loop of lower prices (their cost of goods tends to be near zero) but that hasn't dampened their growth in the least bit.



Many of us may remember the obsession with "eyeballs" back during the internet bubble of 2000, when the number of viewers surpassed all other metrics like revenue or profit. Two decades later, platforms have figured out how to scale and actually monetize those eyeballs.

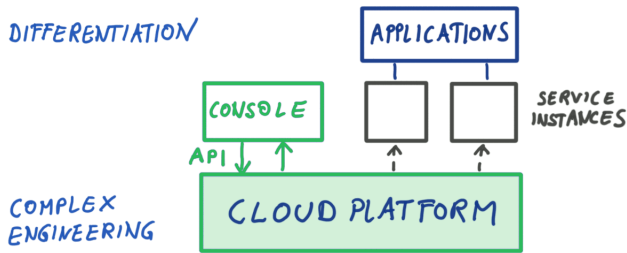
The primary revenue streams for these platforms are advertising or subscription fees. Some media platforms play the role of both the platform operator and a platform participant, for example, producing its own video content (“Netflix originals”) but also distributing third-party content. Platform companies enjoy a large amount of flexibility and tend to experiment all the time.

Cloud Platforms

Cloud computing has perhaps been the most significant IT innovation of the past two decades, being challenged just recently with the rapid rise of AI and large language models (LLMs). The cloud also gave birth to one of the most successful technology business models, with just the top cloud service providers (CSPs) generating some combined \$200Bn in annual revenue. The collection of on-demand IT services provided by these companies are commonly labeled as platforms with Google’s cloud offering proudly carrying the label in its name: GCP = Google Cloud Platform.

The original motivation behind cloud platforms resembles that of automotive platforms. Just like cars, software requires a lot of heavy-duty engineering that isn’t directly visible to the end user. Underneath a successful piece of software lies a vast array of compute infrastructure like data centers, global networks, servers and storage, software delivery pipelines, monitoring and failover mechanisms, synchronizing datastores, backup and disaster recovery, and regulatory compliance reporting. Those aspects can consume a large portion of a software project’s timeline and budget before the first line of application code is ever delivered.

Combining those elements into common components that can be reused across many software projects allows developers to focus on providing customer value and differentiation, for example through rapid feature delivery, unique functionality, or ease of use. They leave the heavy engineering work to the cloud platform providers, who have better economies of scale as they serve many customers. IT departments consume those services in a low-friction way via a so-called cloud console (Web interfaces) or API calls that provision service instances to be used by custom applications.



Cloud Platforms

Cloud platforms also differ from automotive platforms in several ways. First, usage is much broader. Whereas an automotive platform supports a handful of models for a specific automaker, cloud platforms are used for software development across a vast variety of businesses and use cases. It's the equivalent of using Volkswagen's chassis, engine, transmission, and suspension to put your custom interior and bodywork on top. Although this model exists as so-called "kit cars" (which generally start with a finished car to be modified to simplify registration), it's a small fringe business and large-scale automotive platforms lack such flexibility. Bits and bites are more malleable than steel.

Second, cloud platforms allow developers to use the common components easily without any welding or assembling. As described in my book "Cloud Strategy", and elaborated later in this book, a platform's value isn't just defined by what's inside but also by how easily its functions can be accessed. This is where cloud platforms shine. Even though they might look like traditional IT outsourcing from far away, the interaction between platform provider and platform user is very different: month-long contract negotiations are replaced by an API call and near-instant provisioning.

Last, cloud platforms offer more fine-grained components than automotive platforms possibly can. Most cloud platforms offer several hundred individual services to customers, perhaps leading to an abundance of choice that can puzzle customers.

The success of cloud platforms is nothing short of phenomenal, creating a quarter-trillion market (according to [Gartner](https://www.gartner.com/en/newsroom/press-releases/2020-07-23-gartner-forecasts-worldwide-public-cloud-revenue-to-grow-6point3-percent-in-2020)¹²) in just a decade-and-a-half. That's no surprise when you consider that cloud computing has fundamentally transformed IT—from months of infrastructure planning and provisioning to running an automation script and waiting a few minutes.

¹²<https://www.gartner.com/en/newsroom/press-releases/2020-07-23-gartner-forecasts-worldwide-public-cloud-revenue-to-grow-6point3-percent-in-2020>

The Platforms of Giants

Platforms seem to be everywhere these days, and for valid reasons. It also appears that most platform providers are indeed giants, whether it's automotive giants, e-commerce giants, or internet giants. That doesn't mean, though, that there's already a platform for everything or that you have to be a giant to build one. Let's go on to catalog common technology platforms, including those that are often built in-house.

2. The Fab Four of Technology Platforms

Technology drives business models

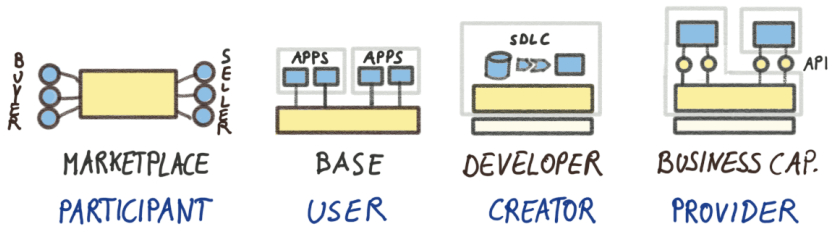
The examples from the automotive, e-commerce, and cloud domains cited in the previous chapter hint that there are different flavors of platforms. Architects don't like fuzzy terms and generally aren't content with enumerating examples as a replacement for a well-structured ontology. So let's add a bit more structure and catalog common types of technology platforms. I can't quite promise that we'll arrive at a universal ontology, but at a minimum, we'll gain vocabulary that we can refer to for the remainder of this book.

Technology Platforms

There are many ways to slice the platform universe and whichever way you do it, you're bound to exclude some or leave gray areas in other places. This book is targeted at IT architects and senior IT decision makers, so it spends less time on platform business models, which are well documented in books like that by Reillier¹. Instead, *Platform Strategy* looks at what it takes to build a technology or business capability platform, the decisions and trade-offs one will face, and how those platforms fit into the overall IT strategy and IT architecture.

To divide and conquer the platform universe, and to sharpen the vocabulary, this book distinguishes four types of platforms that an organization would build or consume:

¹Reillier B. and L., *Platform Strategy: How to Unlock the Power of Communities and Networks to Grow Your Business*, 2017, Routledge



Types of platforms used in the context of IT

Each platform type comes with typical use cases and ways an organization would construct or interact with the platform. Combinations are commonplace, especially for large organizations. For example, a typical organization could be a participant in a 3rd party marketplace, use a cloud base platform to create their in-house developer platform and provide business capabilities to business partners.

Marketplaces

As highlighted in the previous chapter, marketplaces facilitate transactions between customer groups, in most cases buyers and sellers of physical goods or services like media, car sharing, or accommodation.

Common Use Cases

This is the domain of e-commerce platforms like eBay, Amazon, and Airbnb in the US; Tokopedia, Carousell, Mercari, or Taobao in Asia; and mobile.de or Allegro in Europe. Most of them follow the farmers' market model by enabling providers and consumers to transact directly while the platform handles search, advertising, reviews and ranking, fraud detection, and sometimes payment services. Such a setup allows the marketplace to not have to maintain inventory, keeping operating costs down and bringing the marginal cost of onboarding a new seller or buyer to almost zero. The marketplace can generate revenue in a variety of ways, including seller listing fees, placement fees, member fees, transaction fees, or advertising fees.

Interaction

From a technical perspective, participants interact with the platform via a web browser, mobile app, or programming interfaces (APIs). Many marketplaces

provide sellers with functionality beyond listing goods to also define their own storefronts, often without any custom programming effort. Once again this reduces the onboarding friction. Balancing low friction of joining with assuring the quality of services and [minimizing fraud](#)² is a major aspect of many marketplace platforms as it forms the basis for consumer trust in the platform.

Implementation

Most major marketplace platforms are custom-built and thus proprietary. The primary reason lies in the enormous scale and transaction volume that these platforms handle, coupled with rapid growth. These companies' state-of-the-art data analytics and machine learning technologies help them battle fraud, prioritize high-quality listings, and manage ad placements, and are built in-house because they are key assets and often ahead of what's available on the open market.

Luckily, not everything those platforms produce remains proprietary with many marketplace engineering teams publishing open-source projects that are based on their in-house developments: airbnb's [Synapse](#)³, Uber's [Jaeger](#)⁴, Spotify's [Backstage](#)⁵, or Netflix' Chaos Monkey are just a few examples. Often these projects are lower-level engineering tools, but in any case, other organizations can benefit from the giants' engineering efforts.

Becoming a marketplace is less of an IT decision—it's at the heart of your business strategy. Often classified as a B2B2C model, meaning you as a business (visibly) provide a service to other businesses to better serve their customers, marketplaces look easy to build but are extremely difficult to scale. Many successful marketplaces have taken [investments of over US\\$ 1bn](#)⁶, highlighting the massive resources that are needed to grow a platform business.

Considerations

Marketplaces, despite several runaway successes, have well-known challenges. For example, the positive feedback cycle between buyers and sellers can lead to a chicken-or-egg problem at launch. Also, the apparent flexibility in pricing models is strongly subjected to balancing supply and demand. For example, if

²<https://www.vice.com/en/article/43k7z3/nationwide-fake-host-scam-on-airbnb>

³<https://airbnb.io/projects/synapse/>

⁴<https://eng.uber.com/distributed-tracing/>

⁵<https://backstage.io>

⁶https://www.crunchbase.com/organization/tokopedia/company_financials

the platform lacks suppliers (sellers), then imposing seller fees will hinder the platform's growth. Such challenges, including many others, contribute to the "winner takes all" nature of marketplace platforms.

Base Platforms

At the other end of the spectrum, further down the IT engine room, are base platforms. These platforms provide technical products and services to be consumed by developers or IT departments. They are one-sided in the sense that the platform operator creates all services that can be consumed. However, many such base platforms additionally host a software marketplace for third parties, making them a hybrid between a base platform and a marketplace. Things are never quite clear-cut in this space as companies continuously experiment with new products and revenue streams.

Common Use Cases

The most common base platforms in current enterprise IT are cloud platforms, such as AWS, Azure, Google Cloud, Ali Cloud (Aliyun), or Huawei Cloud. Consuming software from third parties is nothing new in IT, but base platforms differ in several aspects. For one, they provide services that traditionally would be done in-house, such as provisioning servers, storage, and network hardware. Second, these platforms aren't a single product like an accounting system or a CRM (Customer Relationship Management) system but a collection of services that customers build on top of. The dividing line once again isn't super crisp and most large software vendors would label themselves as software platforms these days. We'll dive into those nuances later.

Interaction

Users of the base platform users generally interact via several means:

- Online consoles, which actually aren't consoles in the classical sense (see [Wikipedia](https://en.wikipedia.org/wiki/System_console)⁷) but rather web interfaces that allow users to provision and configure resources.
- Command-line interfaces (CLIs), which support direct interaction or scripting for automated execution of platform instructions.

⁷https://en.wikipedia.org/wiki/System_console

- Web APIs that can be called directly from applications.
- Cloud platforms specifically feature additional automation languages in form of Infrastructure-as-Code tools (for example, Terraform, CDK, or Pulumi), which in turn interact with the base platform's APIs.

Most base platforms aim for feature parity across all those channels, meaning the same features are available via any interaction mode. In any case, the platform customers are generally developers or IT professionals, which implies that onboarding effort is higher than with an e-commerce marketplace or social media. Reducing the cognitive load for new platform consumers is a major theme for cloud providers as they are looking to keep scaling up.

Implementation

Just like marketplaces, most base platforms have proprietary implementations due to the enormous scale and complexity of provisioning resources on demand around the globe. The level of customization extends down to the hardware level with cloud providers now providing custom CPUs (e.g. [AWS Graviton](#)⁸), custom hardware architectures (e.g. [AWS Nitro](#)⁹), or security chips (e.g. [GCP Titan](#)¹⁰). Some platform services are based on open-source projects, however most of those will have proprietary run-time extensions to integrate with the platform's "control plane", meaning the common mechanisms for provisioning, billing, monitoring, and so on.

Considerations

For a detailed description of how to be a successful consumer of base platforms, especially cloud platforms, I refer you to my book [Cloud Strategy](#)¹¹ in the same series. Building such broad platforms is a rather involved topic, which we won't be able to address here.

Developer Platforms

In-house platforms are developed by IT departments, generally to provide reuse of common IT services, to boost developer productivity, and to assure

⁸<https://aws.amazon.com/ec2/graviton/>

⁹<https://aws.amazon.com/ec2/nitro/>

¹⁰<https://cloud.google.com/blog/products/identity-security/titan-in-depth-security-in-plaintext>

¹¹[CloudStrategyBook.com](#)

compliance with operational guidelines. These platforms support the in-house Software Development Lifecycle (SDLC) that development teams follow to deliver custom software. [Part IV](#) is dedicated to the detailed design considerations for such platforms.

Common Use Cases

In-house platforms range from software build and deployment platforms that enable autonomous development teams to secure data analytics platforms. They often also include other common software delivery functions like monitoring or visualization. Backstage is a widely used tool to build in-house developer portals and tools. Some organizations embark on large-scale efforts to abstract away cloud platform provider specifics to reduce vendor lock-in, and ostensibly boost productivity. Often, those platforms restrict developer's choices, which actually [hinders productivity](#).

Some companies externalize portions of their internal platform as open-source software. Amazon Web Services (AWS) and Kubernetes are prominent examples. AWS evolved from supporting Amazon's e-commerce marketplace into one of the largest base platforms for IT services. Kubernetes is strictly speaking not an externalized tool because it was specifically built to be shared as open-source software, which incorporates many of the internal features of Google's internal [Borg](#)¹² workload management ecosystem.

Interaction

In the best case, application developers interact with in-house platforms much like they would with base platforms, i.e. through well-designed Web consoles or APIs. However, many in-house platform teams might lack the capacity to provide feature parity across Web consoles, command lines, and APIs. While many in-house platforms co-exist with the underlying base platform, others intentionally restrict access to the base.

Implementation

In-house platforms commonly sit on top of one or more cloud (base) platforms. They might span public cloud and on premises data centers to give enterprises (some semblance of) a uniform development and deployment model across modern cloud providers and existing on-premises environments. [Part IV](#) dives deeper into these aspects.

¹²<https://research.google/pubs/pub43438/>

Considerations

In-house platforms live in an exciting but also hotly contested space between base platforms, developed by large-scale providers with many resources, open-source projects, and development teams. Therefore they have to consider their scope and positioning very carefully and have to live up to the expectations set by external developers. Even though they are not offered on the free market, in-house platform teams have to support a variety of internal customers, so they may find themselves surprised that they need internal marketing, consulting, support, and related functions that makes them resemble a company within a company. A large portion of this book dives into these decisions.

Business Capability Platforms

Whereas developer platforms largely contain technical tools for developers, business capability platforms expose functions that belong to the business domain. The delineation is easy to understand by the fact that a bank, a retailer, and a ride-sharing company could use the same developer productivity platform but have very different business capability platforms that reflect their respective domains. That's also the reason why we see a lot of sharing via open source in the former but not the latter.

Business capability platforms are often built and maintained in-house but are not limited to in-house use. Instead, they are made available to external parties, allowing the provider to create an ecosystem. Examples range from digital identity services in the public sector to payment APIs offered to enable embedded finance in the Fintech industry. These externalized business platforms can in turn serve as base platforms for development inside ecosystem participants, highlighting the layered—or even fractal— nature of the overall platform ecosystem.

Externalized platforms can evolve from well-architected in-house services that originally enabled reuse at the business capability level. Those platforms are then augmented to become accessible from outside the organization.

Common Use Cases

Business capability platforms can be found in most industries. In fashion e-commerce the [About You Commerce Suite](https://commercesuite.aboutyou.com/)¹³ evolved from a [proprietary imple-](#)

¹³<https://commercesuite.aboutyou.com/>

mentation of a fashion ecommerce marketplace¹⁴. In financial services, you'll find platforms like Allianz' now defunct [Syncier](https://www.syncier.com/en)¹⁵ spin-off, which made Allianz' core insurance system available as a software platform for other insurance companies. Its marketplace of third-party solutions, described as Insurance-as-a-Service, was spun off and acquired by Munich Re.

Other enterprises offer collections of lighter-weight APIs to grow external ecosystems. For example, many banks (like [DBS Bank's Developer APIs](https://www.dbs.com/dbsdevelopers/)¹⁶ or [Standard Chartered's Axess](https://axess.sc.com/apis/)¹⁷) have encouraged the development of third-party applications that make use of their payment APIs. For other payment providers like Stripe the business capability platform is the core business.

Most cloud providers also play in this space by offering higher-level services business in addition to the more traditional IT services like compute or storage. For example, AWS provides call center services with [Amazon Connect](https://aws.amazon.com/connect)¹⁸ while Google Cloud includes [medical imaging services](https://cloud.google.com/medical-imaging)¹⁹.

Interaction

APIs are the key enabler for business capability platforms as integration often occurs across organizational boundaries. Some externalized platforms allow higher degrees of integration and customization, usually at the cost of higher friction for adoption. This additional friction of onboarding can counter-act some of the “platform-ness” of such offerings and position them closer to the traditional model of large, customizable software packages like SAP. Just exposing your internal application via an API in a multi-tenant model does not automatically make it a platform.

Implementation

Business capability platforms are developed like regular software products but have additional provisions for external usage, such as public APIs, multi-tenancy, or scalability. Those aspects make them well suited to be built on top of a cloud base platform. Operational aspects like data security or legal considerations tend to play a larger role due to the business-to-business nature of the relationship between platform provider and platform consumer.

¹⁴<https://corporate.aboutyou.de/en/presse/about-you-offers-its-proprietary-e-commerce-infrastructure-as-an-api-solution>

¹⁵<https://www.syncier.com/en>

¹⁶<https://www.dbs.com/dbsdevelopers/>

¹⁷<https://axess.sc.com/apis/>

¹⁸<https://aws.amazon.com/connect>

¹⁹<https://cloud.google.com/medical-imaging>

Considerations

Although offering business capabilities to third parties might benefit direct competitors, it also opens up new revenue streams and market opportunities. For example, About You or Amazon could have considered their internal tooling a competitive advantage that they prefer to keep to themselves. In AWS’ example that would have meant to forgo a \$100B opportunity. The old slogan “When there is a gold rush, sell shovels” appears to apply here: many companies don’t just compete as a business but also benefit from selling tools to others.



The capability platform approach thrives in environments like fashion retail that comprise many niche markets and specializations, because it reduces the risk of a platform provider receiving direct competition from its platform customers.

One of the underappreciated benefits of offering an in-house product as an external platform is the insight one gains into how others use the platform. Such insight can in turn be used to improve in-house products and operations, gaining a critical market advantage. That’s why Simon Wardley calls platforms [future sensing engines](#).

Four in a Row

In summary, we can identify distinct platform models, although in reality companies might engage in more than one option:

Model	Popular Use Cases	Value Proposition	Interaction	Implementation
Marketplace	airbnb, eBay, Amazon, Mercari	Match customer groups, enable transactions	Browser, Mobile App, API	Proprietary
Base	Cloud providers	Instant provisioning of IT resources	Web console, command line, API	Proprietary, can include open source

Model	Popular Use Cases	Value Proposition	Interaction	Implementation
Developer	In-house platforms, cloud “wrappers”	Development speed, reuse, governance	Subset of Web console, command line, API	Proprietary, on top of base platforms
Business Capability	Allianz Syncier, About You	Build an open ecosystem	APIs, custom integration	Proprietary, on top of base platforms

When people use the term “platform”, they often think of marketplaces or multi-sided markets, but there’s a lot more diversity in this space. Most of these platforms are one-sided, meaning one platform provider offers the platform to many platform users, but that doesn’t diminish their power.

This book focuses on the platforms that you are most likely to build yourself in large-scale IT: productivity and business capability platforms.

Combinations Encouraged

Following our ambition to illuminate platform strategies, we might have to heed the advice and demands from strategy books. Those often desire a classification of items to be MECE, **Mutually Exclusive and Collectively Exhaustive**²⁰. Such a classification works well for decision trees to select a course of action: all options are distinct and in summary they cover the available option space.

However, this approach doesn’t work as well in a world where things are routinely layered on top of each other, and one element can play a different role to the next layers below or above. That’s why our list of platform types is decidedly non-MECE. Marketplaces are likely to be built on cloud base platforms and existing in-house platforms might be externalized later, perhaps even turning into base platforms like AWS has done. Likewise, a business capability platform can help its customers build marketplaces as we see in fashion e-commerce.

²⁰https://en.wikipedia.org/wiki/MECE_principle

Combinations Discouraged?

Many providers who pursue multiple models highlight that they are both a platform provider and user to boost their credibility (“We don’t build for the market. We are part of it.”). Those organizations must carefully consider whether to maintain feature parity for their in-house and the externalized platform. The [anti-trust case against Microsoft](#)²¹ was partly based on allegations that their base platform, the Windows operating system, included features and APIs that favor development of Microsoft’s own software products like Internet Explorer versus market competitors. Microsoft subsequently agreed to publish Windows APIs as part of the settlement, which was agreed in 2004. So, while it’s good to be your own customer, it’s even better to play fair.

²¹https://en.wikipedia.org/wiki/United_States_v._Microsoft_Corp.

Part II: A Strategy for Platforms

Platforms can play a pivotal role in IT transformation by reducing complexity, harmonizing software delivery, and stabilizing operations, all the while reducing cost. It's no surprise then, that they have become a staple in today's IT and business strategies. But launching a multi-sided market business model, building an in-house developer platform, or even using a cloud base platform are major strategic decisions, which require significant investments in both technology and organizational change.

That's why success with platforms rarely results from an IT department's effort alone. Bridging both IT and business strategy is a key aspect of a successful platform strategy. Any gaps in this area can become the source of expensive failures.

"Strategy" as a term sadly doesn't stand far behind "platform" when it comes to fuzziness and overuse. So, this chapter starts by recapping what it means to define a strategy, before combining both terms into a meaningful whole. Strategy will never be a "paint by numbers" exercise (nor should it be), but organizations can benefit from common building blocks and recipes harvested from successful platform strategy definitions and executions:

The following key elements play a critical role in defining a platform strategy:

- Before getting into platform strategies, let's first describe [what constitutes a sound business or IT strategy](#) for large organizations.
- Successfully using or running platforms requires organizations to [rethink their existing ways of working](#)
- Models help us clarify decisions. [Wardley Maps](#) make excellent models to understand commoditization effects that drive platform adoption.

- [The Platform Paradox](#) describes how platforms can combine seeming opposites, such as innovation and harmonization.

3. Formulating a Strategy

Strategy is the difference between making a wish and making it come true.

Given the power of platforms, whether internal or external, one-sided or multi-sided, it's natural to want to include them as part of your IT strategy. Unsurprisingly, merely adding a line item to your list of ambitions won't do much to lead your organization to platform enlightenment. So, it behooves us to revisit what makes a meaningful strategy.

Strategy Is Hard

Good strategies look simple in hindsight. However, defining one and documenting it well, is all but simple. The opening statement of A.G. Lafley and Roger L. Martin's book [Playing to Win: How Strategy Really Works](#)¹, captures the challenge well:

Strategy is not complex. But it is hard. It's hard because it forces people and organizations to make specific choices about their future—something that doesn't happen in most companies.

Meaningful strategies must connect the dots between long-term vision and short-term tactics, between business and IT, and between quantifiable success metrics and beliefs. Platform strategies are no exception.

Strategy Is Hard to Argue Against

Strategy as a term operates in similarly fuzzy territory to “architecture” and “platform”. Still, hard data confirms that it's become a rather popular term over the past half-century:

¹<https://hbr.org/chapters/playing-to-win>



Mentions of the word “Strategy”

Strategy’s popularity might be fueled by reams of business books, aided by the need for an anchor point in an ever-changing and increasingly complex world. Apparently I am not above the trend, having chosen to suffix my entire book series with the magic word.

Life writes the best stories and any cynic finds ample fodder in two familiar places: professional services and large enterprises. When working in consulting, I found the term “strategy” most frequently applied to justify actions that eluded any rational argument:



Any potential engagement in an undesirable location that was unprofitable and required scarce skill sets was routinely labeled as “strategic” by the sales team.

Apparently, a strategy is difficult to argue against: it lays out a compelling vision that might materialize sometime in the future and therefore can’t be proven or disproven so easily. Although some amount of speculation can’t be avoided when predicting the future, that shouldn’t be a license to disregard current reality. Good strategies “think big”, but are meaningful only if they can lay out a credible path starting from the here-and-now.

Make Your Platform Wishes Come True

This book is the second title in a series on IT strategy. Volume 1, [Cloud Strategy](#)², describes the power of a strategy rather vividly:

²CloudStrategyBook.com

A strategy is the difference between making a wish and making it come true.

A strategy lays out a path that makes important decisions and trade-offs so that the vision won't remain just a wish but can become reality. This isn't a book about strategy (the virtual shelves are full of those), but it's helpful to highlight characteristics of a good strategy in the context of major IT transformations:

You Can't Copy-Paste Strategy

Many of today's enterprises are transforming their IT to better compete with digital disruptors and remain competitive in a fast-changing world. Naturally, they're reviewing other organizations' experiences within their industries or are looking to learn from those companies that are considered role models for success in the digital world. Although it's good to consider what other organizations have done, a meaningful strategy will be unique to each organization.

A sound strategy depends on your organization's unique assets (such as brand, people, intellectual property, equipment, cash, technology), its constraints (such as resources, labor contracts, regulatory environments), and its environment (such as competitors, market positioning, price pressure). That's why I often remind my customers that:



The goal of your transformation isn't to copy someone else's success. It's to maximize the results you achieve from your own assets.

Organizations are hardly carbon copies of each other, so their strategies also shouldn't be. A strategy should amplify your organization's success as opposed to becoming like XYZ Silicon Valley company. Or, as my colleague (and serious foodie) Mark Schwartz pointedly states, "becoming the Amazon of seared foie gras isn't a meaningful strategy".

IT and Business Strategy Form a Two-Way Street

In traditional organizations, the technology strategy derives from a business strategy, and for good reasons.



A customer once stated that they are planning to define their cloud strategy. However, they are waiting for their business strategy to be in place, which will allow them to define an IT strategy, which will include an architecture strategy, that will be vital input into the cloud strategy.

The role of technology is to support the business, but this no longer implies a one-way dependency. For one, the traditional sequential approach blatantly ignores the cost of delay: the world isn't standing still while you are waiting for these documents to materialize. And in the absence of any cloud strategy, divisions likely forge ahead and trade a well-thought-out strategy for uncoordinated activity. Second, technology also drives business strategy.



A manufacturer of large power generators might transition from selling pieces of equipment to pricing by the MWh produced. Likewise, a train engine manufacturer could sell a transportation service, priced by person-kilometer traveled.

Such a transition from selling assets to providing services closely resembles the move to platforms: cloud platforms sell compute hours as opposed to shipping physical servers. Such business strategies can be extremely successful but make little sense without the advances of modern technology. A broken locomotive will mean that no person-kilometer is realized and hence no revenue is generated—maintenance is now the responsibility of the provider. And the best maintenance is predictive so it can avoid issues before any actual downtime occurs. Sensors, IoT, edge computing, analytics, and machine learning are the technologies that make this business strategy viable. Likewise, selling compute capacity as a service like modern serverless cloud offerings do requires sophisticated technology to manage security, isolation, load optimization, failovers, and much more.

That's why it's apt to apply the following test to new business models:

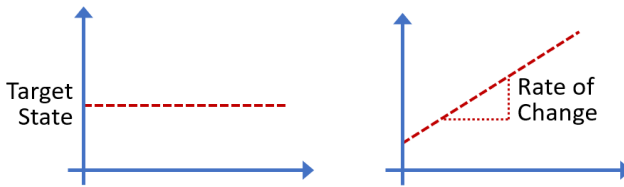


If this is such a great idea, why didn't everyone do it 30 years ago?

In most cases the answer is clear: we didn't have the technical capabilities, or at least not at the needed price point. In some cases, we did do it 30 years ago and just forgot how poorly it went.

Think in the First Derivative

As we saw [in the beginning](#), platforms enable. For example, base platforms and in-house developer platforms enable faster software delivery whereas marketplaces enable the exchange of goods. The goal of rolling out a platform therefore is not to achieve a pre-defined, static target state, but to allow the organization to grow or to change more quickly.



Platforms define the rate of change, not a target state

A platform strategy, therefore, should think in the first derivative, in the rate of change, as opposed to a simple target state. As elaborated in *Cloud Strategy*, a traditional project mindset favors a single target state, which coincides with the end of the project. Platforms, in contrast, keep on evolving. Because they also help platform users evolve faster, we can understand platforms really only [once we take a dynamic view](#).

Documenting a Strategy

A successful strategy consists of two accomplishments: capturing key decisions and documenting them well. Much of this book is about the former, so let's just include a few pieces of advice for the latter:

Aim for Emphasis over Completeness

A like-titled chapter from *The Software Architect Elevator* reminds us that complex topics require abstraction, omitting many details to amplify the important aspects. Naturally, this leaves us with an important skill:



Defining what's most relevant is a critical step towards devising a strategy.

You will see the forest only if you don't concern yourself with every single tree. "What is not included" can be one of the most important chapters near the beginning of a strategy document. Also, remember that you are writing for human readers and those tend not to read dictionaries from cover to cover. So, it's best to not make your strategy document look like one, for example by defining an arbitrarily arranged list of 21 strategic elements and then describing each one in detail.

Use Conceptual Models

The picture that first enters many people's minds when thinking of strategy is a 2x2 matrix (we'll see one in the [next chapter](#)). Those matrices adorn many management consulting slide decks (e.g., as a SWOT matrix) and make a regular appearance in Eben Hewitt's useful book *Technology Strategy Patterns*³. The [central picture from the defining book on enterprise architecture](#)⁴, Enterprise Architecture as Strategy, is a 2x2 matrix that correlates the IT operating model with business process standardization and integration.



The resources that IT vendors dedicate to earning a spot in industry analysts' "magic quadrant" (a 2x2 matrix across completeness of vision and ability to execute) hint that many critical IT decisions are based on 2x2 matrices.

A 2x2 matrix is a very popular and extremely effective example of a conceptual model, which [Wikipedia](#)⁵ defines as follows:

Conceptual Model A representation of a system, made of the composition of concepts which are used to help people know, understand, or simulate a subject the model represents.

Although a mouthful, helping people know and understand is definitely something a strategy should do. Complexity is that objective's biggest enemy.

³Hewitt, *Technology Strategy Patterns: Architecture as Strategy*, 2018, O'Reilly

⁴https://cisr.mit.edu/publication/2005_12_3C_OperatingModels

⁵https://en.wikipedia.org/wiki/Conceptual_model

Models are the single most powerful tool we have to battle complexity and clearly communicate concepts. The simpler a model, the better a job it does at reducing complexity.



The Yardstick At the Singapore government we [developed several simple models](#)⁶ to help us make more informed platform decisions. A particularly effective one was “the yardstick” that visualized which part of a problem has already been solved, what needs to be built as a common platform, and what’s left to individual teams. Despite, or perhaps because of, being about the simplest model you can imagine, it proved immensely useful in our IT decision boards.

A more powerful model that depicts the evolution of commodity elements and bespoke ones is Wardley Maps. Because that distinction is critical to platform design, this book dedicates [an entire chapter to it](#).

The Path and the Terrain

A good strategy covers three dimensions, perhaps in the figurative, not the purely mathematical sense (see [Thinking Strategy? Add Dimensions!](#)⁷):

- 1) A point: Where do you want to go?
- 2) A path: How will you get there?
- 3) The Terrain: What happens when you step off that path (or the ground shifts underneath you)?

Some strategies stop at the first item and aren’t strategies at all—those are the mere wishes cited above. Actual strategies must lay out a path that’ll take the organization from here to there. Two common mistakes plague such strategies. First, the path might not connect to the organization’s status quo. This typically happens when a strategy is adopted from another organization or a success story that worked off different preconditions. Needless to say, a path starting from somewhere else isn’t a very useful option. The second pitfall is a path that only considers the happy-day scenario. Assuming an ideal and predictable world won’t last very long in the face of today’s uncertain reality.

⁶<https://www.linkedin.com/pulse/building-smarter-nation-simple-models-lead-better-decisions-hohpe/>

⁷<https://aws.amazon.com/blogs/enterprise-strategy/thinking-strategy-add-dimensions/>



The shape of the terrain represents your strategy's risk profile, i.e. how resilient it is to unforeseen changes. Are you walking on a narrow ridge or gentle slopes?

A narrow ridge might be the shortest path but it doesn't tolerate any missteps or any shifting terrain. Events of 2020 have made it clear that ground can shift suddenly and more vehemently than often assumed. Ignoring this aspect, the shortest but riskiest path will always seem most attractive—until you actually get there.

Credible Roadmaps

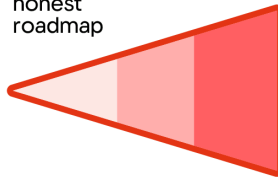
Strategies should be concrete and actionable, but that doesn't mean that they can define a simple linear path—that's simply not what the world looks like. Pretending a linear sequence of predictable steps is the classic fallacy of a misleading roadmap, as nicely illustrated in the following diagram⁸:

misleading
roadmap



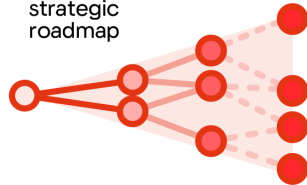
"We will follow this linear path from zero to our inevitable victory, and these are the milestones that will get us there."

honest
roadmap



"We know where we are today, and have a good idea about the near future, but recognize that the future becomes uncertain."

strategic
roadmap



"We know where we are today, and anticipate decision points that will change our path. We are intentionally gathering the data that will help us make those decisions."

Misleading roadmaps vs. strategic roadmaps, © Pavel Samsonov

Misleading roadmaps look convincing on paper but rarely last past the first stages of execution. I have seen enough of these strategies that I tend to describe the inevitable failure as follows:



When reality doesn't match the plan, we blame reality.

⁸Used with permission. Source: <https://twitter.com/PavelASamsonov/status/1296818042928861184>

Seeing the future as uncertain forms an honest roadmap and is a big step ahead. But giving in to uncertainty doesn't make for a very meaningful strategy and runs the risk of "we'll figure it out as we go along". A strategic roadmap therefore anticipates decision points, possible paths to be taken, and the data needed to make those decisions. The original diagram was aimed at product roadmaps, but as we will soon see, platform evolution shares a lot in common with product evolution.

From Strategy to Execution

Now that you are intellectually pumped up about strategy, you need to translate the learnings into an actionable execution plan. Too many strategies jump straight from lofty goals into technology buzzwords—this is where the concept of the Architect Elevator looks to bridge the gap and connect the dots.



When asking technical folks about the benefits of platforms you'll hear many great things, such as empowerment, automation, open APIs, cohesion, undifferentiated heavy lifting, reduced coupling, abstraction, autonomy, knowledge sharing, and innovation. Structuring this laundry list into a logical sequence of goals and mechanisms makes the difference between wishful thinking and a strategy.

A common anti-pattern for strategy documents is the dangerous "IT Hourglass" described in [Cloud Strategy](https://CloudStrategyBook.com)⁹. The hourglass depicts the shape of such presentations: they start wide with exciting claims like "faster innovation at lower cost", then get very thin in the middle, and conclude with wide-ranging funding and headcount requests. Although strategy is never a paint-by-numbers exercise, structuring your strategy document into four distinct but connected layers can help you steer clear of the hourglass:

- **Context** describes why you are following a platform strategy in the first place. This part assures the above-mentioned alignment with the business strategy. If this part isn't clear, executives will stop reading right there.
- **Objectives** (or goals) are business benefits that your strategy is intended to deliver. Reducing costs or speeding up innovation are worthwhile business goals. This section should place a clear emphasis on the top

⁹<https://CloudStrategyBook.com>

objectives instead of listing all possible benefits—too many items will weaken your strategy.

- **Mechanisms** are well-known techniques that help deliver your objectives. Increasing code reuse or enabling team autonomy are suitable mechanisms. The strategy needs to explain how these mechanisms support your objectives. Be careful not to fall for buzzwords like DevOps or microservices at this point.
- **Design decisions** are concrete trade-offs that you need to make during implementation. For example, you might mandate that all developers use the same programming language to make it easier to reuse other teams' code. Alternatively, you might look to achieve the same by wrapping all code assets behind standardized service APIs—more complexity but also more choice. Because your design decisions will be more “technical”, it's important to describe them in a way that keeps them meaningful to all stakeholders.

The following table summarizes these four levels of a strategy with examples from typical platform strategies:

Level	Description	Key Activity	Example
Context	Why you are creating a strategy	Link to business	Increasing competition
Objective	What you want to achieve	Prioritization	Foster innovation
Mechanism	Common ways to get there	Translate objectives	Increase code reuse
Design Decision	Trade-offs you are making	Explain well	Standard APIs

Although this recipe sounds simple enough, many strategies fall into one of several common traps:



Poorly defined strategies tend to go heavy on the objectives (that's how you get funding), trade mechanisms for buzzwords, and propose design decisions without a clear linkage to the other two.

The antidote is to ride the *Architect Elevator*, i.e. to make a solid connection across the multiple layers. As so often, the connection is as (or more) important than the elements in the boxes. The following chapters will elaborate on these layers in individual chapters.

Strategy is a Winding Road

A last piece of advice on defining and executing strategy. A strategy defines an overall direction, not the exact steps that you will be walking. That means that tactically you might be doing something that looks like it doesn't line up with your strategy. As long as it is done deliberately and achieves the overall strategic objective in the long term, that's totally fine"



When you're walking up a hill you likely won't go straight up but instead traverse. A person lacking context may challenge you as to why you're moving sideways instead of toward the peak.

A proper linkage between tactics and strategy reveals that hikers can climb the hill using switchbacks. Even though they traverse the mountain, they understand and support the strategy.

4. Becoming a Platform Company

Transformation Can't Be Understood from the End Product

Platform plays are often considered in the context of an IT transformation, and for good reasons. For one, to fully benefit from modern base platforms, such as cloud platforms, an organization has to fundamentally change—or transform—the way it is working. Otherwise, you get the same old problems just in more modern packaging. Second, platforms help businesses overcome past constraints and effectively compete with digital disruptors, something that they would not be able to achieve otherwise.

More than meets the eye

In a presentation about enterprise transformation, I often cite the example of the disruption of the US automotive industry by Japanese automakers in the 1970s. The “big three” automakers—Ford, Chrysler, GM—enjoyed a fruitful oligopoly until their Asian counterparts introduced cars into the market that were of higher build quality, more reliable, better equipped, less expensive, more fuel efficient, and by many accounts optically more appealing. When someone's product beats you in all categories, it's called a disruption—derived from Latin *disrumpere*, which means as much as breaking apart.

The natural reaction of the incumbent automakers might be to purchase one of the clearly superior cars, learn how it can be so much better, and build similar cars. Sadly, they would not find much. That's because the essence of this transformation took place in the way of working, way before the finished product.



The essence of the transformation cannot be seen from the outside.

US manufacturers employed “quality assurance”, essentially rework, at the end of production. Japanese manufacturers, led by Toyota followed a very different philosophy: they realized that quality must be built-in, not added on. It makes little sense to replicate a systemic defect 50,000 times over on the production line. They therefore did what would have been unimaginable in US plants: when they encountered a problem, they would stop the production line (using the famous “andon cord”) to investigate, so that they could correct it once and for all. None of this fundamental shift can be understood by looking at the finished product.

Don't Burst the Boiler

Companies that try to replicate other companies' success without seeing how those companies work fundamentally differently are subject to the “bursting the boiler” effect. This metaphor borrows from another past disruption brought upon by transformation. It also relates to transportation but took place half a century earlier: the transition from the steam locomotive to the electric train engine:



If you're running a steam engine and an electric train easily passes you, you'll want to speed up to remain competitive. So, you do what has worked before: you put more coals on the fire and increase the boiler pressure. This might initially speed you up, but it won't lead to a transformation—it'll lead to a burst boiler.

Increasing the pressure is an attempt to optimize the way you are currently working. But one cannot optimize their way out of a disruption:



Disruption means that someone has changed the rules of the game. Those who play by the old rules have essentially already lost.

To transform you must think and work differently. Platforms are like electric train engines. They change the rules of the game by removing constraints.

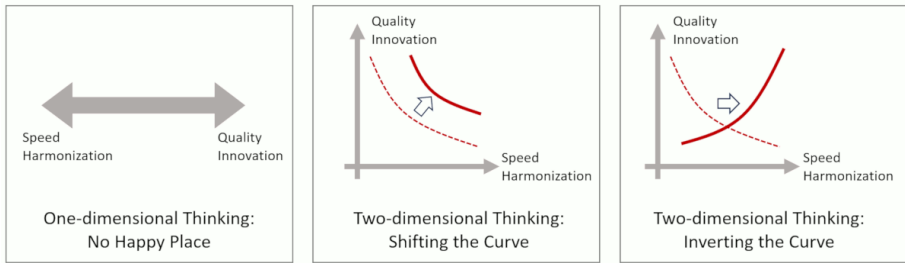
Classic IT Conflicts

Modern CIOs are known to be living between a rock and a hard place. Upper management routinely demands more efficient IT operations based on harmonization and complexity reduction. Meanwhile, uncertain times require the organization to be agile, requiring novel tools that create more diversity and thus work against harmonization. Similarly, the business demands faster feature delivery to better compete in a fast-changing world while regulatory environments prescribe ever-increasing levels of transparency and compliance, which threaten to slow down delivery.

Rarely has this tension been more pronounced than during the pandemic of 2020/21 where many IT budgets were being frozen or actively cut at just the time when organizations were forced to pivot from physical to e-commerce distribution. Naturally, such a shift has a major impact on IT requirements, scale, and operational needs. There's no shortage of vendors or consultants peddling tools or giving advice on how to overcome these IT pain points. A battle-weary CIO can choose from a wide menu of Agile frameworks and certifications, DevOps approaches, Lean methods, low-code or no-code frameworks, VM-to-container conversion tools, and out-of-the-box machine learning frameworks. But lasting change is rarely something that you can buy. Rather, it derives from a new mental model or a different worldview.

Opposites Attract

The example above is typical for a mental model that considers innovation and harmonization to be opposite ends on a one-dimensional spectrum. The sad part is that organizations following this mental model will never be happy: either they reduce diversity and complexity but are too rigid to innovate, or they are innovative but have a giant operational mess on their hands.



Platforms make you think in more dimensions

The disruption (and the subsequent transformation) happens when the world-view changes to a two-dimensional model (yes, almost like the 2x2 matrix!). In such a model, harmonization and innovation aren't diametrically opposed but are two dimensions on a plane. Now you can think about how you can shift the curve so you can have more innovation without giving up harmonization. You could achieve that by automating manual processes or by changing your cost allocation model to better support small experiments. Even better, you can discuss how you can invert the curve so higher degrees of harmonization actually increase innovation. You could achieve that with common APIs, for example, that harmonize communication but boost the reuse of functionality.



The shift from one-dimensional thinking to two-dimensional thinking is critical for any platform transformation.

In my transformation workshops, participants brainstormed properties that used to be opposites but have turned into two independent dimensions that allow both properties to be present. The list of items that are no longer opposites is quite impressive, even though it is far from complete:

Characteristic	Perceived Opposite	Enabling Mechanism
Speed	Quality	Automation
Cost	Agility	Modularity, Iterations
Standards	Innovation	Platforms, Interface standards
Economies of scale	Economies of speed	Cloud platforms
Openness	Profitability	Professional open source
Low Risk	Change	Automated tests, Continuous Delivery
Control	Chaos	Transparency, Automated governance
Short-term gain	Long-term gain	Adaptability, low friction

The list also describes the mechanism that makes these items no longer opposites. For example, automated testing and deployment allow us to deliver at higher speed *and* quality.

Transforming by seeing more dimensions

Some of these old, one-dimensional mental models have been around long enough that they're codified in popular slogans. For example, speed and quality have been seen as at odds with each other for long enough that doing things "quick and dirty" has become a de facto law in IT management. People will routinely debate if they should cut corners (be "dirty") to speed things up. I tend to challenge this assumption as follows:



If I were a subversive developer who'd want to sabotage a software project, injecting some very nasty and difficult-to-find bugs into the system would all but guarantee a major delay.

The idea that compromising quality speeds things should be revisited. Why can't we have things that are quick and clean? High degrees of automation, such as continuous integration and delivery pipelines, reduce friction and eliminate mistakes caused by manual steps. So, it not only speeds things up but also makes the process more repeatable and thus increases quality at the same time.



“Fast and compliant” was the slogan we used to describe the platform we deployed at a major financial services organization to allow rapid software delivery while remaining compliant with internal and external requirements. It was one of the core elements of the organization’s transformation.

Identifying technologies that overcome past constraints and seeing how they can change the way an organization works is essential to any successful transformation, including becoming a platform company. Companies that set out to copy the success of disruptors while being stuck in the old way of working will not only fail to meet their goals, but they likely also incur a lot of pressure and frustration on their own organizations.



Transforming an organization requires realizing that things are no longer either-or.

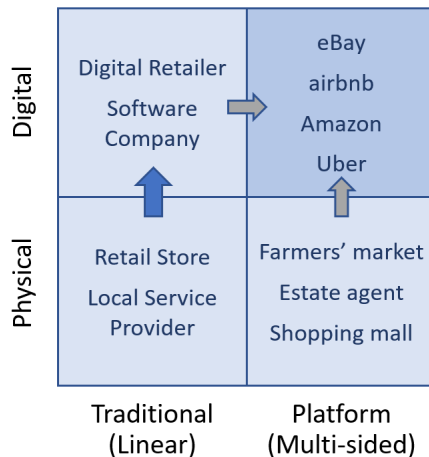
Transforming by rethinking old ways

Many IT organizations are structured around delivering projects with the underlying assumption that once a set point is reached, they can stop and transition to business-as-usual. These organizations **don’t think in the first derivative** and miss out on improvements that allow them to scale, accelerate, or pivot, if needed. Such companies might mistake the journey to using—or becoming—a platform company as a technology retrofit. *Cloud Strategy* begins by reminding readers that cloud isn’t a tech upgrade but a lifestyle change for the entire organization.

A platform company will be keen to translate technical progress to better user experiences or value creation. For example, if such an organization gains the ability to use virtual machines instead of physical servers (cloud platforms anno 2006), they would look to reduce capital expenses, harden server images for increased security, automate provisioning of servers, improve service levels, and optimize resource usage. A traditional company might see a cheaper server and be content with those savings without ever transforming.

The Path to Platforms: On-Off-Line

Thinking in two dimensions also helps organizations chart their course to becoming a platform business model. As the examples at the beginning of the book show, platform models aren't limited to "digital" businesses but also exist in the physical world, such as with farmers' markets or the automotive industry. So, becoming "more digital" and "becoming a platform" are once again two independent dimensions. A 2x2 matrix from Reillier's book illustrates this well:



Physical and Digital Business Models (adapted from Reillier)

Mapping the dimensions onto a matrix allows organizations to plot a path over this terrain. Do you want to up first, then to the right? Or take a shortcut and move diagonally? In the diagram, the arrows indicate which transitions occur and how likely they are:

- Many early adopters of digital technology moved from the lower-left quadrant to the upper-left: hotels and airlines added online booking sites; retailers enabled online orders and perhaps delivery options. While this might look like progress, they created what I refer to as "digital copies" of their existing business, staying in the traditional model, but with modern, digital technology. Such a move can boost business, but it doesn't entail a transformation to a platform company.
- Only a few organizations managed the transition to a digital platform business (top right), a sector primarily occupied by so-called "disruptors".

Those new entrants challenged traditional business models without being hampered by legacy. One notable example is Amazon, which started as a digital retailer (top left) but very successfully shifted to the digital platform model (top right) with the Amazon Marketplace in 2000 that admitted third-party sellers.

- One might think that the transition from a traditional platform model (lower right) into a digital platform (upper right) is straightforward. However, they are quite rare - only very few traditional real estate agents moved into the upper quadrant—it's generally occupied by new entrants like Zillow.

Digital platform models are thus an apt example of the great power that lies in aligning an organization's business and technology models. Neither can substitute the other, but when properly aligned, they can become a force multiplier.



An organization's business and technology models, although unable to substitute one another, can become a force multiplier when properly aligned.

We also observe that a business model transformation is more difficult than technology adoption because it questions or even cannibalizes the existing business model, an effect that was labeled by Clayton Christensen as The Innovator's Dilemma¹. The resulting complacency or outright resistance of traditional businesses to transform opens the doors to disruptors entering the market, often to the apparent surprise of the incumbent businesses.

¹Christensen, C., *The Innovator's Dilemma: The Revolutionary Book That Will Change the Way You Do Business*, 2011, HarperBusiness

5. Mapping Platforms

If you don't know where you are, a map won't help.

As described in the previous chapters, a meaningful strategy links the business environment with technology evolution. It lays out a path to succeed in the former by utilizing (dare we say, “leveraging”?) and sometimes anticipating the latter.

Although you'd think that most established business work off a well laid-out strategy, the reality looks very different. Simon Wardley is one person who's not shy to point to the giant gaps in many large organizations' strategies and is aiming to shrink it with his modeling technique called Wardley Maps. Maps are a great tool to depict strategies (or reveal their shortcomings), so let's look at some maps.

Maps as Visual Models

Maps are models—they are an abstraction that depicts the relevant features of a physical environment. “Relevant” are those things that help you answer questions or make decisions, like which turn to take.

Maps are some of the oldest models in use, so it's no surprise that numerous quotes guide us in their application:



“The map is not the territory.”

Coined by Alfred Korzybski in 1933¹ and expanded and popularized by Hayakawa², this popular saying reminds us that the representation of a thing

¹Korzybski, *Science and sanity; an introduction to Non-Aristotelian systems and general semantics*, 1933, International Non-Aristotelian Library Publishing Company

²Hayakawa, *Language in Thought and Action*, 1949, Harcourt

(the model) isn't the thing itself. Hayakawa outlines the difference by describing two worlds: the extensional world that we experienced ourselves (authentic but limited) and the intensional world that we know from descriptions like words or maps. The latter have a much wider reach, but they are subject to emphasis and abstraction, and can occasionally misrepresent reality, as warned by Hayakawa:



"No matter how beautiful a map may be, it is useless to a traveler unless it accurately shows the relationship of places to each other, the structure of the territory."

Maps aren't artwork but serve a purpose in the real-life territory, for example to find the shortest path or win a battle. Ironically, too many IT maps fall into this trap: they are either beautiful but far detached from reality or aim to be authentic and end up a mess. It might make us wonder about the terrain we are working with in corporate IT.

Kent Beck, author of *Data and Reality*³ from 1978, makes this point very clear in his "message to mapmakers":



"Highways are not painted red, rivers don't have county lines running down the middle, and you can't see the contour lines on a mountain."

Bill's book provides an introduction into the representation of information in computers, highlighting that this task is a lot more involved than trying to represent reality.

No chapter on models would be complete without a quote by statistician George Box:



"All models are wrong; some are useful."

Box shaped this famous aphorism also in 1978 to remind us that accurately representing reality isn't a model's (or map's) goal. In many cases, the more

³Kent, *Data and Reality*, Technics Publications, 3rd edition, 2012, Technics Publications LLC

“wrong”, i.e. the more abstracted, a model is, the more useful it becomes. As Bill Kent stated, highway maps are far detached from reality, but are extremely useful.

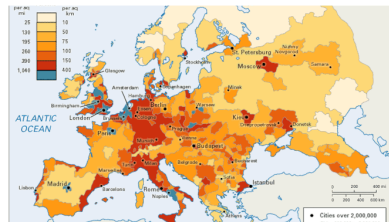
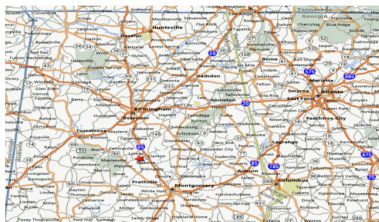
Which Map is Best?

Cheekily placing myself alongside this great lineage of model thinkers, my sequel to George’s insight highlights the context in which models are used:



“To know which models are useful, you must first know which question you’re trying to answer.”

To illustrate this point in my architect workshops, I share the following graphic of four two-dimensional models of a well-known system called planet earth. Then I ask: “Which map is best?”



The best map depends on the question you’re trying to answer.

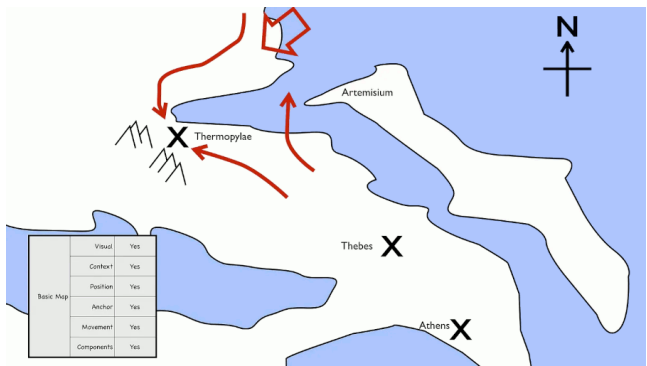
It’s a trick question because there is no single best map. The most useful map is the one that can answer your question at hand. Interestingly, the list of questions can be more varied than originally thought. For example, a topographical map can tell you the easiest hiking path, the best place to construct a ski resort, where to construct a dam for a hydropower plant, or where to build your house outside the river’s expected flood zone (or the lake

created by the dam). Political maps are great to explain election results, road maps help you choose the fastest driving routes, and population density maps (often combined with transport network maps) let you find a suitable location for a distribution center.

IT strategies tend to raise a lot of questions, and IT reality is complex and messy, so maps are some of the best tools to help us answer those questions.

Maps and Movement

Maps have been used for a long time. [Simon Wardley](#)⁴ likes to start his description of maps and mapping with the battle of Thermopylae that took place in 480 BC when the Spartans fended off Xerxes I and his invading army⁵:



The strategy for the Battle of Thermopylae, © Simon Wardley, CC-SA

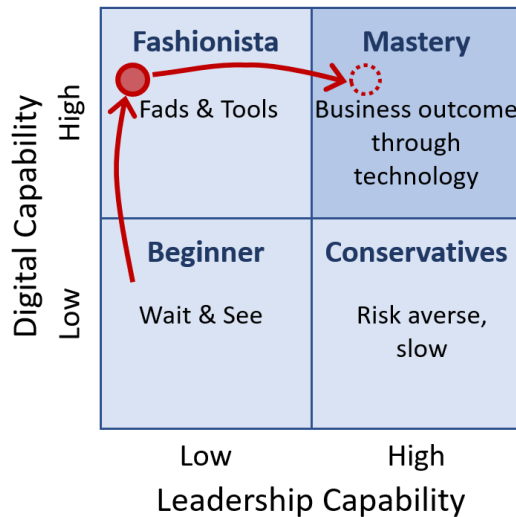
This map could be a rather uninteresting outline of some Greek coastline, but plotting the movements of the opposing armies makes it interesting even some 2500 years later. That’s why Simon considers movement to be one of the six basic elements of maps: visual representation, context (i.e., the question you are trying to answer), components (in this case military contingents) and their position, an orientation anchor (a chess board grid or a north compass), and movement of the components (in this case the armies and navies).

⁴<https://medium.com/wardleymaps/on-being-lost-2ef5f05eb1ec>

⁵Source: <https://medium.com/wardleymaps/on-being-lost-2ef5f05eb1ec>, Licensed under CC-SA 4.0: <https://creativecommons.org/licenses/by-sa/4.0/>

2x2 Maps

Simple models are often most useful. The much derided 2x2 matrix is one of the simplest models, which still passes Simon’s map test as long as it contains movement. In the [previous chapter](#) a 2x2 matrix also helped describe the path from a traditional to a platform model along a physical or digital format. Similarly, a 2x2 matrix presented by George Westerman and his coauthors⁶ plots an organization’s technical capabilities against its leadership capabilities for a digital transformation journey. This map also depicts a grueling battle, that between change agents and conservative leadership:



Movement across digital capability with leadership capability

This matrix’ visual representation lays out the territory as four distinct zones as orientation anchor. The typical symptoms of organizations in each zone may help you determine if you are in one of those quadrants, perhaps without realizing it. If it does, the map has already been helpful.

This territory map also allows you to add components (your organization) and its movement. For example, you might have built up digital capabilities through so-called lighthouse or innovation projects, moving upwards on the map. Now

⁶Westerman, Bonnet, McAfee, *Leading Digital: Turning Technology into Business Transformation*, 2014, Harvard Business Review Press

you are in need of increasing leadership capabilities to avoid fragmentation and digital silos.

A simple map like this one makes this strategy utterly clear. Adding your current position and your path turned the 2x2 matrix into a useful map for your strategy.

Wardley Mapping

Wardley Maps are also two-dimensional grids, but significantly more expressive than simple 2x2 matrices. Living at the intersection of technical capability (the tools you have or are going to have) and business value (what your customers will get by you using these tools) makes them great strategy tools. Wardley Maps plot technologies (the components) along two axes:

- The vertical axis represents the value chain, indicating how visible a component is to the user. Dependencies between components flow from top (visible) to bottom (less visible). A strict labeling of this axis isn't necessary as it is mainly about the relative positioning of the components.
- The horizontal axis represents the stages of evolution of the components: genesis (newly discovered things), custom built (uncommon things we still learn about), product (increasingly common, repeatable), and commodity (highly standardized, utility).

Wardley maps include movement, so components move about the map based on two mechanisms:

- Commoditization describes a component's evolution towards the right (along the X-axis), from genesis towards commodity. This movement is driven by competition and can be hindered by inertia, such as patent protection or an organization's inability to change.
- Componentization allows a system to be broken down into identifiable and reusable pieces, which can then move independently towards commoditization.

The combination is a powerful potion as [stated by Simon](https://blog.gardeviance.org/2011/03/componentisation.html)⁷:

⁷<https://blog.gardeviance.org/2011/03/componentisation.html>



Commoditization to standard components leads to an explosion of innovation for higher order systems.

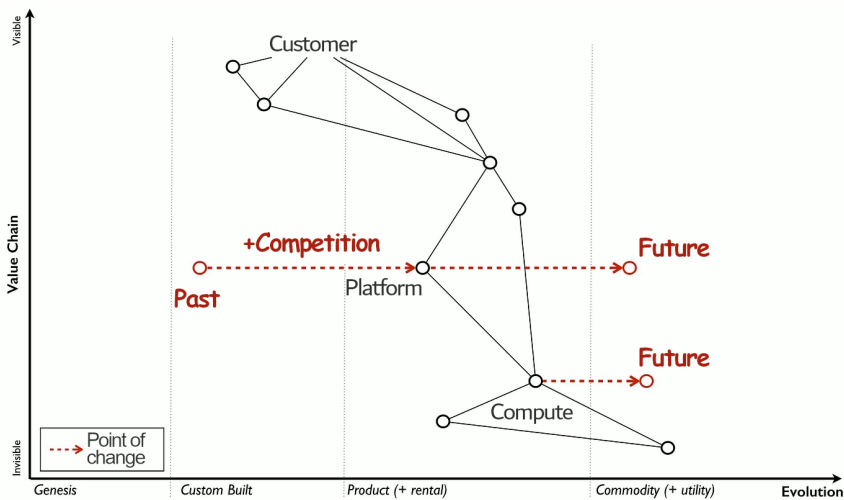
Using one of our many car analogies, cars have become heavily componentized, assembled from standardized, often interchangeable parts, that range from light bulbs to brake pads and entire transmissions. The componentization has pushed commoditization of these parts. Whereas in the early days of the automobile the brake assembly might have been a custom-developed component (on the left of the Wardley map), you can now choose brake pads and calipers from a dozen or so manufacturers like Brembo, Bosch, or ACDelco. These parts have become a commodity, widely available at competitive prices.

The wide availability of standard components and an overarching component model (the parts of a car and their interactions are well understood) has not caused all cars to look the same. Quite the opposite: standard components boost innovation and differentiation of higher-order systems like cars. Without standards brakes, lights, and other components, it would be time consuming and uneconomical for manufacturers to offer sedans, wagons (estates), two-seater coupés, convertibles, pick-up trucks, SUVs, crossovers, and minivans. Once again magic of platforms comes into play, boosting innovation through harmonization.

Platform Evolution

Compute resources follow a similar path towards the right, from custom built to commodity. The following map plots the evolution of platforms on a Wardley Map⁸:

⁸Source: <https://medium.com/wardleymaps/exploring-the-map-ad0266fad59b>, text enlarged for readability

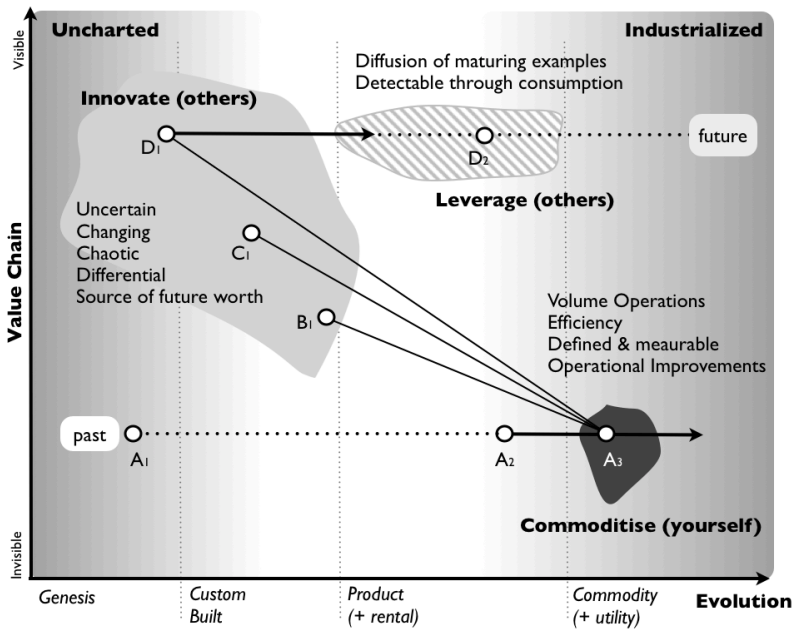


A Wardley Map showing platforms and commoditization, © Simon Wardley, CC-SA

Building a run-time environment used to involve installing and connecting multiple components, for example a so-called LAMP stack (Linux, Apache, MySQL, PHP). Demand competition pushed for easier and faster ways to get such stacks up-and-running, ranging from automation scripts to pre-built machine models and container images. Modern cloud run-time platforms, especially serverless platforms, largely free developers from such toil. In the Wardley Map above, the platform (for example a PaaS product) sits above the pure compute, indicating that it is higher up in the value chain, i.e. more visible to the customer.

However, the observation of platforms moving to the right barely scratches the surface. The secondary effect of commoditizing a platform is that it **enables innovation in form of additional components**⁹, indicated by B, C, and D in the following map:

⁹<https://blog.gardeviance.org/2014/03/understanding-ecosystems-part-i-of-ii.html>



Treating Components, © Simon Wardley, CC-SA

Some companies may resist commoditization of their component (shown as moving from A_1 to A_3) as it subjects it to replaceability and pricing pressure. However, having others innovate on top of your now commoditized component also presents a unique opportunity. Your platform can sense changes in the upper components, for example D_1 advancing to D_2 , giving you a plausible target to further industrialize (and commoditize) that component, allowing you to repeat the process.



Platforms can be future sensing engines by detecting trends of innovations built on top.

This cycle of repeated commoditization is referred to as ILC, Innovate - Leverage - Commoditize: while other companies innovate, you can leverage your platform to identify patterns and trends, and then commoditize. In a simple example, a marketplace like Amazon can monitor sales of consumer goods like batteries and offer a private label version (Amazon Basics) once sales reach a certain volume or a trend towards standardizing on AA and AAA cells can be

detected (C and D cells have become increasingly rare due to the prevalence of rechargeable batteries).

Software platform teams can use the same approach to sense trends by monitoring the usage of their platform and utilizing this data as an input into their [roadmap](#).

6. The Platform Paradox

It's a kind of magic.

Now that our view of platforms has become less fuzzy, we are ready to tackle one major aspect of IT platforms that sets them apart from many other IT initiatives (and that made it into the subtitle of this book): platforms can [break through age-old IT conflicts](#), specifically those between achieving standardization and boosting innovation. With platforms, you don't need to choose between one and the other; you can have your IT cake and eat it, too!

The Magic of Platforms

When it comes to overcoming perceived opposites, platforms hold a special place. Platforms are per definition harmonized—a platform's strength lies in its broad usage. A platform can't be customized for each user because the high onboarding friction would break one of the key enabling factors for platforms (many [in-house platforms](#) fall victim to this trap). At the same time, platforms clearly boost innovation: they reduce friction and give their users many degrees of freedom.

To highlight this counterintuitive quality of platforms I occasionally remind cloud customers in somewhat blunt terms:



The cloud you are getting is the same one your competitors are getting.

Cloud platforms are also the biggest innovation drivers that IT has seen in the last decade-and-a-half. That's because they harmonize, i.e. place constraints, to remove other constraints: by running your applications on a single platform, you can get compute capacity on demand without paying for spare hardware.

That's the paradox of platforms—they unify to boost innovation and diversity:



Platforms break the perceived dichotomy between achieving harmonization and allowing independent innovation.

Removing Constraints by Constraining

Although this combination might appear like magic, especially to IT leaders who collected many battle scars while trying to find a balance between the two opposing objectives, similar dynamics date over a hundred years back. A vivid example, detailed in *The Software Architect Elevator*, is the Baltimore Standard for fire hydrants:



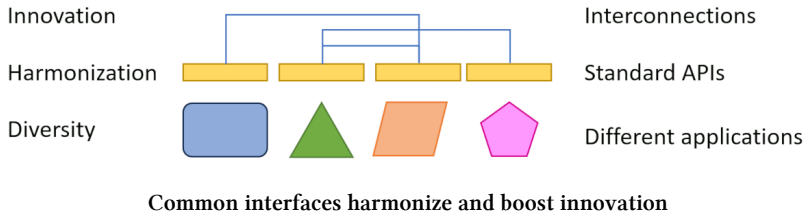
In 1904, when the city of Baltimore burned to the ground, firemen from surrounding towns came to assist but weren't able to do much: their hoses wouldn't fit on Baltimore's fire hydrants. Harmonizing those connections, in the Baltimore Standard of 1905, enabled new use cases such as fire crews assisting another town.

Information technology has seen similar effects—without burning anything down. Agreeing on *interface standards*, which is a clear act of harmonization, is a huge innovation booster. HTTP is a standard, it is quite precise and universally adopted. By being a standardized protocol, it allowed diversity between web browsers and web servers and thus gave rise to the perhaps most innovative construct we have seen in the past decades, the internet. Imagine a lack of harmonization, where one browser could only work with websites running on a server procured from the same vendor. The internet would have surely never happened in such an environment.



HTTP is a standard that dramatically boosted innovation through harmonization.

HTTP is just one example of the power of interface standards. Common APIs achieve the same. By constraining, e.g. by demanding common data formats and authentication mechanisms, diverse components can now interact and are no longer constrained in the choice of programming language or underlying runtime.



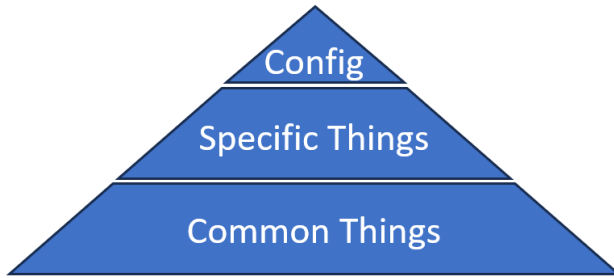
Common interfaces foster reuse, accelerate the creation of new solutions, and boost innovation by making more functionality instantly available.

Platforms aren't Pyramids

From far away, in-house platforms can resemble classic IT frameworks: all common elements that apply across business units / geographies / product lines are implemented in a base layer, on top of which resides a collection of more specific, but still reusable elements. The cherry on top is that each business unit or geography merely has to configure some settings, and voilà, your business application is ready to serve customers without an additional line of code written. Although large software applications like CRM or ERP systems do unify many common processes, this approach looks much better on PowerPoint than in reality. As described in the chapter “They don’t build’em like that anymore” in *The Software Architect Elevator*, the issue with this approach is twofold:

- 1) Anticipating all users’ needs to the point where they just have to flip a flag is very, very hard or, more likely, impossible.
- 2) Even if you do guess correctly, building the base layer that accommodates all users’ needs takes a massive effort.

Interestingly, these visions are often depicted as pyramids in IT strategy documents, notwithstanding the fact that people stopped building pyramids some 5000 years ago, partly due to impracticality and partly due to horrible economics:



We stopped building these things 5000 years ago—except in IT

So, how are platforms different? Platforms enable users to construct the needed functionality without having anticipated each and every possibility:



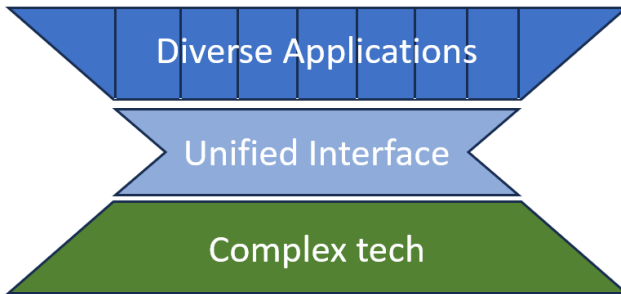
Platforms don't try to anticipate every use case.

This key aspect makes platforms leaner and more flexible at the same time. Users build on top of a platform; they don't just set some configuration bits. Anticipating users' needs seems such a noble endeavor that it's easy for platform builders to fall into this trap. That's why I recommend this simple but very useful test to all platform builders:



If your users haven't built something that surprised you, you probably didn't build a platform

Technology platforms like cloud platforms do have large base layers because they need to manage networking, security, data centers, communications, provisioning, workload placement, operations, and much more. The key aspect of a good platform is to hide that complexity behind a simpler interface, which in turn enables a broad set of use cases, some of which could not even have been anticipated by the time the platform was built:



Platforms try to hide the complexity underneath and enable diversity above

How Platforms Break Barriers

What's the magic sauce here? There isn't a single aspect that gives platforms this amazing property, but rather it's the interplay of multiple factors:

Componentization

Breaking something complex into standardized and recombining components speeds up innovation through re-composition. To choose a simple example, standard-sized bricks have significantly sped up construction without reducing possibilities. Combining technical components isn't as easy as stacking bricks, though, as we will soon see.

Separating commodity from differentiators

Platforms bake widely used functions into a common layer and make them easily consumable. Although the idea of reusable components isn't new, finding the dividing line, especially as it keeps shifting, is no easy task. Successful platforms get this right.

Building Economies of Speed on Economies of Scale

Building platforms is a scale business - they thrive on growth and often require massive investments. However, platforms hide these scale effects from their customers by allowing incremental usage, thus democratizing access to resources.

Centralizing decentralization

Platforms are central elements that foster autonomy and independent decision-making, essential elements of decentralized organizations. At the same time, platforms provide a safety net and necessary guardrails.

Let's look at how each of these effects contributes to platform magic:

Componentization

As elaborated in the previous chapter, Simon Wardley highlights the linkage between commoditization (something becoming commonly available and undifferentiated) and componentization (the ability to assemble something new from pre-made parts). Platforms achieve their potential by not being monoliths, instead exposing many re-combinable elements.



Cloud platforms routinely feature hundreds of individual services that provide an on-ramp for new users and support a broad variety of use cases.

Componentization is more than splitting something into parts. Chopping wood isn't componentization. Instead, the pieces need to have a clear relationship to each other, an aspect that we'll [dive into a bit later](#).

Componentization generally requires an overarching architecture that defines boundaries and connecting elements. The [automotive industry](#) was successful with its platform strategy because cars have a well-understood architecture of individual components. Increasing componentization was the key mechanism that allowed companies like Volkswagen to progress from the basic hat-platform model to the modular “toolbox” approach.

Separating Commodity from Differentiators

Product platforms, much like frameworks, look to draw the line between commonly needed components and those that differentiate: whatever can be widely used goes inside the base layer and the rest is built bespoke on top of it. While that sounds intuitive enough in principle, in reality, it's a delicate balancing act. Several factors make drawing that line difficult:

- **Needs vary by user group:** Some people might consider something being included in the platform a convenience whereas others might consider it a restriction. Platforms can have a “soft edge” by allowing user contributions, but that doesn't make the problem go away entirely.
- **The boundary shifts:** IT evolves and so do the needs of platform users. AWS, the largest cloud provider, started out provisioning just virtual machines (and message queues) but today provides over 200 services,

ranging from serverless compute to data analytics and machine learning. Platform users also need time to adapt to new services, so finding the right pace of innovation is important.

- **Cohesion over precision:** Finding the exact dividing line between commodity and differentiator for each platform element doesn't necessarily yield a good result as it may lack a uniform level of abstraction across services.
- **Interaction matters:** It's not just about what's in the platform but as much about how one accesses it. Valuable functionality that's difficult to access won't make a great platform.

Figuring out where to draw the line between what goes inside the platform and what is left to the platform requires a clear strategy but also tight feedback cycles. Perhaps that's one reason that the most successful cloud platform providers are the ones that are relentlessly customer centric.

A great tool to visualize products that are differentiators vs. commodities is [Wardley Mapping](#).

Economies of Speed Built on Economies of Scale

Platforms thrive on scale. Some of the most successful technology platforms, cloud services platforms, require enormous upfront investments, such as massive data centers and global fiber networks. The size of these investments limits the market to just a handful of so-called hyperscalers.

At the same time, cloud platforms have enabled users to operate in *economies of speed*, where an organization's pace of learning and innovation outweighs its size. Cloud has democratized IT: whereas only large enterprises had access to fancy data centers and mainframes in the past, nowadays any start-up business, no matter how small, has access to the same powerful services. This achievement made me conclude in *Cloud Strategy* that the real magic trick of cloud is as follows:



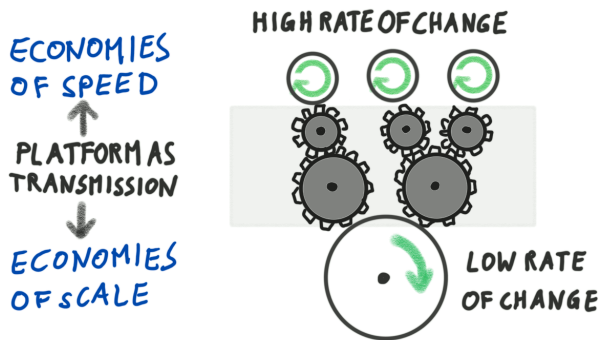
The cloud managed to draw a very precise line between a scale-optimized implementation and a speed-based business model.

If we zoom out from cloud platforms to platforms in general, one of my favorite architect moves can help us put our finger on this aspect of platform magic: thinking in the first derivative.

IT is often depicted as a “stack” of dependencies, i.e. the layers from an application program or service to the middleware (like application servers or databases), operating systems, processor architectures, networks, and ultimately down to power supplies and physical server rack sizes. When looking at such a stack, we notice that lower layers generally have slower rates of change than higher layers. That’s a good thing because items on top depend on the items lower down.

The most prevalent processor architectures in use today (Intel x86 and ARM) date from 1978 and 1985, respectively, making it all but certain that they’ll reach a popular lifetime of over half a century. The Windows operating system is leading a happy adult life at 35 years and Linux is turning 30 this year. And it turns out the 19“ rack, the standard size for mounting computer hardware in data centers dates back to 1922 (look it up on [Wikipedia](https://en.wikipedia.org/wiki/19-inch_rack)¹—I did as well). Higher up, things move faster: developers now code mobile apps in Kotlin, whose 1.0 release dates 5 years back, and deploy on the latest Kubernetes release from a few months ago.

The critical items in such a stack are the ones that evolve comparatively slowly themselves but enable high rates of change “above”, i.e., items that resemble a transmission:



Enabling higher rates of change

Platforms are such elements: they can support a high rate of change above while undergoing a much lower (albeit not zero!) rate of change. Operating systems match this criterion and are hence often described as platforms.

¹https://en.wikipedia.org/wiki/19-inch_rack

Centralizing Decentralization

The last aspect of the power of platforms relates to organizational forces, following the definition by [ThoughtWorks’ Peter Gillard-Moss](#)²:

Platforms are a means of centralizing expertise while decentralizing innovation to the customer or user. This essentially happens through commoditization of services, but in a way which relinquishes an amount of control and is open to extension. This is a significant change to many organizations’ core business models, which tend to favor standards unification and work centralization.

Peter’s wording aptly describes the platform paradox: we are relinquishing some control but we do so in a centralized manner. This important nuance also explains why many so-called platform initiatives started by traditional infrastructure teams fail: they aren’t looking to relinquish control but are aiming to strengthen it:



The most difficult step for organizations embarking on a platform journey is to relinquish control.

As illustrated in *The Software Architect Elevator* chapter “Control is an Illusion”, platforms can enable autonomous teams without descending into chaos:



Cloud platforms can enable teams and at the same time assure consistency in “tactics” through a common set of tools.

Although it’s a bigger topic, relinquishing some control often increases compliance. If IT infrastructure is rigid and cumbersome, teams are likely to just go rogue and do their own thing. An open platform allows users to use at least part of the common layer and perhaps even contribute to it.

²<https://www.thoughtworks.com/insights/blog/platform-tech-strategy-three-layers>

A4 Paper Doesn't Stifle Creativity

I am never short of a good metaphor, so allow me to conclude with one from *The Software Architect Elevator*. A4 paper is one of the most widely used standards in the world (outside the US at least, where most of the world is located anyhow). However, anyone would be hard-pressed to claim that they aren't able to exercise their creativity due to restricted paper size. That's what a good platform should aim for.

7. Addendum: “I ACED My Strategy”

A Simple Framework for Writing IT Strategy Documents

IT strategies that I have seen work well had several characteristics in common:

- **Alignment:** an IT strategy must line up with a business strategy (not my favorite IT word, but appropriate in this context) .
- **Clarity:** a strategy is only meaningful if it’s easily understood by a broad audience.
- **Evolution:** although strategies are meant to last, they still need to evolve as opportunities come along, priorities shift, or existing constraints are removed.
- **Decisions:** a strategy that doesn’t make decisions will sound like a list of wishes. And those rarely come true.

While this list is by no means based on scientific evidence, it’s a useful checklist to apply to your document or slide deck that has the word “strategy” on the cover. It’s also easy to remember as you’re going to want to claim that you “ACED” your strategy.

Let’s look at each aspect in a more detail:

Alignment

A platform strategy isn’t self-purpose very much like an IT strategy isn’t. Both must support the business today and in the future. A platform strategy will invariably be part of an overall IT strategy, for example, a digital transformation, which in turn (hopefully) supports a business strategy, perhaps a pivot into a marketplace platform model or an entry into new markets. A key job for architects is to make sure that these strategies support each other:

Business Strategy $\Rightarrow$ IT Strategy $\Rightarrow$ Platform Strategy

Too many IT strategies fail not because they are poorly executed (that also happens) but because they aren’t aligned with a business strategy and therefore can’t bring the anticipated benefits. Predictably, the business side will be rather unhappy despite, or perhaps because of, much effort having been made. In a defensive move, IT will be tempted to portray some form of success by resorting to vanity metrics—IT-internal metrics that are under IT’s control. It might proudly report that the platform was in fact built or all servers have been migrated to the cloud in the hope that no one will notice that these achievements didn’t actually improve any business-observable metrics, such as cost of operations or speed of innovation.

A better way to tie an IT (and platform) strategy to a business strategy, is to define success metrics that are meaningful to the business. *Cloud Strategy* described what this can look like in the chapter “Value is the Only Real Progress”.

Clarity

Different parts of the organization heading in different directions is unlikely to make for a great strategy. Alignment is therefore not just needed between the different layers of abstraction (business - IT - platform - implementation) but also across organizational units. That’s why a successful strategy needs to address a broad set of audiences. Too often do we like to make things more complicated—after all we want everyone to appreciate how hard we worked.



A strategy should be well-thought-through but easy to understand.

Conceptual models, as described earlier, can go a long way to adding clarity. Another common technique is to describe the strategy in horizontal layers of detail: a high-level slice across all topics, akin to an extended executive summary, followed by more detail. Structuring your document that way allows the audience to read up to the point that is relevant to them.

Evolution

This section is still work-in-progress

A strategy cannot predict the future—after all, its main purpose is to help you cope with uncertainty. It therefore does need to absorb changes and evolve over time. This evolution incurs two levels:

- 1) The evolution of the elements described in the strategy
- 2) The evolution of the strategy itself

For example, if your in-house platform augments an existing cloud platform with capabilities that aren’t yet included in the [base platforms](#), your platform can evolve in two distinct forms. You could expect your platform to shrink over time as its features become displaced with the base platform’s. Or, you’d want your platform to stay the same size (expressed in scope or value delivered), but “float” on top of the base platform, e.g. by adding new features as you shed obsolete ones.

Decisions

Last, as already highlighted in my previous book *Cloud Strategy*, decisions are a critical as part of any sound strategy:

Strategy is defined by a series of meaningful decisions, those that require conscious trade-offs.

Through these decisions, a strategy narrows the playing field. We choose to forgo some options, perhaps in favor of gaining others. Decisions are critical to reduce complexity, an observation encoded in “Gregor’s Law”:



Gregor’s Law: Excessive complexity is nature’s punishment for organizations who can’t make decisions.

The need for decisions applies to platform strategies just as well. Such decisions can relate to many aspects, for example, the platform’s scope (what’s in your platform and what isn’t?), its level of openness (can everyone contribute?), or its expected lifespan (will you be happy to toss your platform in a few years when the same functionality becomes available from a third party platform?). Due to their importance, this book [dedicates a whole chapter](#) to platform decisions.

The following table summarizes these four aspects of a technology strategy and how we can express them:

Property	Rationale	Mechanisms
Alignment	Assure that outcomes support business strategy	Business-relevant success metrics
Clarity	Avoid misunderstandings or mismatched expectations	Evocative conceptual models
Evolution	Deal with changes	Scenarios, paths
Decisions	Narrow the playing field, reduce complexity	Meaningful options and choices

Part III: In-House Platforms

Many IT organizations experience the nuances of platforms when they set out to build an internal IT platform for developers. Although such platforms are generally built on top of commercial platforms or assembled from open-source components, their design and roll-out differs substantially from typical IT projects. Therefore, rolling out an internal platform isn't as simple as it may appear and can present numerous challenges for traditional IT departments.

This part provides IT organizations with guidance on how to build and deploy an internal developer platform:

- [In-House IT Platforms](#) come in several shapes and flavors.
- Be aware that platforms teams are different animals and that [Platform and IT Services Org are Antonyms](#).
- Don't rely on magic, but understand the [Technical Mechanisms for an IT Platform](#) to achieve anticipated platform benefits.
- Isn't it odd that developers [love opinionated platforms](#) but despise restrictive ones?
- As with any architecture, you need to [Make Conscious Platform Decisions](#).
- [Using Platforms](#) is easier than building them yourself, but there are still many things to watch out for.

8. In-House IT Platforms

My way is the highway

Platforms have become extremely popular inside large organizations thanks to the promise of harmonizing software delivery while also removing friction and thus speeding things up. Alas, like many successful concepts, the term platform has suffered from *semantic diffusion*¹, meaning that not everything carrying the “platform” label would actually qualify as one. Let’s take a closer look at the common types of platforms eagerly being built inside large-scale IT departments.

IT: Steering from the Top

Any useful element of an IT strategy must consider the context in which it will be implemented. Although IT’s main goals [vary with the CIO reporting line](#)², several common elements drive large-scale IT: increased business efficiency (deliver value for the business), compliance (no one wants to go to jail), and reliability (no one likes to pay for IT that isn’t running). All along, IT should minimize the money spent achieving those goals, i.e. reduce the overall cost. These high-level goals typically match the main metrics that the business, who ultimately funds the IT, uses to evaluate the health of its IT.

Whereas the traditional metrics optimize for a known steady-state, modern organizations—those who aspire to become more “digital”—have learned that velocity (pace of delivery) and agility (rate of change) are equally critical to the success of their business. When organizations find that some of these worthwhile pursuits are at odds with one another (for example, building new solutions quickly can increase complexity and operational costs), they often look to in-house platforms to achieve both. As we have seen in [The Platform Paradox](#), the magic of platforms can indeed break through perceived opposites, but achieving those benefits requires teams to dive deeper into the IT engine room.

¹When a word is coined with a good definition but gets spread through the community in a way that weakens that definition, ultimately losing the definition entirely—and with it any usefulness to the term. (Martin Fowler)

²<https://www.linkedin.com/pulse/reverse-engineering-organization-gregor-hohpe/>

IT Platform Benefits

In-house IT platforms, when scoped and implemented well, can provide both development and operations teams with a handsome list of benefits:

- **Reduce cost:** platforms allow organizations to reuse IT assets, whether provided by the platform or contributed by users, across a wide range of use cases, reducing the cost for new development.
- **Increase velocity:** reuse not only reduces cost, it also speeds up new development and experimentation because new projects don't need to start from scratch—you're standing on top of a platform. Automated processes and build pipelines can further reduce friction and speed up delivery.
- **Assure compliance:** building solutions on top of shared platforms can help assure compliance with internal processes and external requirements, such as regulatory oversight. This can be achieved, for example, by curating specific versions of third-party products, configuring underlying base platforms, or providing pre-built software delivery pipelines that include automated quality and compliance checks.
- **Improve transparency:** a frequently underappreciated benefit is the improved transparency into applications running on top of an in-house platform. Traditional IT suffers from a general intransparency, exacerbated by numerous, often proprietary systems. Common platforms significantly improve transparency, for example, by easily showing the number of applications, the resources they consume, their uptime, patch levels, and the last time they were updated.
- **Reduce lock-in:** in-house platforms can also serve as abstraction layers over external tools or platforms to reduce direct dependencies on those products. We discuss the trade-offs and dangers of this approach in a [later chapter](#).

Thanks to these benefits, in-house platforms are important elements of many IT strategies and are often viewed as the element that [finally allows developers and operations to work together instead of against one another](#).

IT Platform Varieties

Building in-house IT platforms has become so commonplace in large-scale IT organizations that it's helpful to categorize them first. [McKinsey Digital's article on platform play](#)³ distinguishes three broad areas of in-house platforms:

- **Customer journey platforms** contain reusable elements that define the customer proposition and experience. Such platforms contain coarse-grained functional components assembled from commercial components. For an e-commerce business, such a platform may define click-and-collect experiences or same-day home delivery.
- **Business capability platforms** consist of business solutions, for example, payment services or inventory management services for retail domains. Such business capabilities are intended to be modular and wired together to support the customer journey platforms.
- **Core IT platforms** provide the shared technology on which the journeys and business capabilities run, such as a cloud platform, build and deployment pipelines, or data analytics environments. They also include higher-level technical services such as image analysis or in-store face recognition that directly support customer journeys.

Referring back to the semantic diffusion of the term “platform”, it's useful to include McKinsey's definition:

Services or products that operate as independent entities and bring together business, technology, governance, processes, and people management.

The platforms matching this classification are likely smaller than those following the definition in this book, which considers a platform as a comprehensive collection of services. For example, the article mentions a typical business having 20-40 platforms. In contrast, the IT platforms described in this part group many IT capabilities and may be a single IT platform across a large organization.

³<https://www.mckinsey.com/business-functions/mckinsey-digital/our-insights/the-platform-play-how-to-operate-like-a-tech-company>

Nevertheless, the layers defined in the article help us think about the positioning of in-house platforms and their benefits.

ThoughtWorks' Gillard-Moss⁴ describes a similar three-layered view of what types of platforms are commonly used inside businesses:

- **Service Platforms** are the main offering to be used by customers, for example, an [e-commerce platform](#). They are often the most visible part of successful platform companies.
- **Digital Business Platforms** extend the service platform to be integrated with partners, usually via APIs, into a platform ecosystem.
- **Foundational Technology Platforms** underly other products and services, similar to the Core IT Platforms described in the McKinsey model.

Both models follow a “stack” metaphor with specialized items closer to the customer positioned on top and more generic technical items at the bottom.

Typical IT Platforms

Let's look at in-house platforms that can be commonly found in organizations ranging from large-scale IT to so-called digital-native companies:

Digital Platforms

When enterprises embark on something modern in a departure from their traditional way of working, the word “digital” often becomes a stand-in for whatever the new model might entail. Platform initiatives fare no better, with “digital platform” serving as an organization's stepping stone into the modern, aka “digital” world. Naming challenges aside, looking towards platforms as an enabler to compete in a fast-changing environment is indeed a sound idea.

Most “digital platforms” are actually a combination of the multiple kinds of platforms per McKinsey's three-layered model and map roughly to Gillard-Moss' Digital Business Platforms. Didier Bonnet and George Westerman's [article in MIT Sloan Review](#)⁵ summarizes the value of a digital platform as supporting the core elements of a digital transformation:

⁴<https://www.thoughtworks.com/en-th/insights/blog/platform-tech-strategy-three-layers>

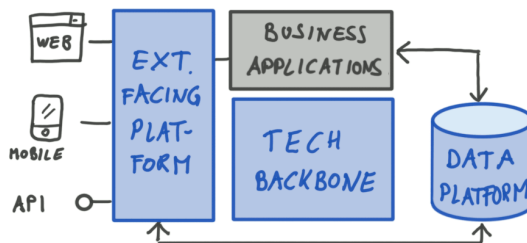
⁵<https://sloanreview.mit.edu/article/the-new-elements-of-digital-transformation>

- Enable new business models
- Provide better customer experience and employee experience
- Assure efficient, reliable, and data-driven operations

These core elements hint at the building blocks that are needed to make such a platform work:

- The *externally facing platform* powers websites, mobile apps, and other processes that connect to customers and ecosystem partners via APIs. This platform shares similarities with ThoughtWorks' notion of Service Platforms and McKinsey's notion of Customer Journey Platforms.
- The *platform core*, or "technology backbone", includes all elements needed for operational and transactional systems. In most organizations that would include a cloud or modern on-premises runtime platform. This layer is analogous to McKinsey's Core IT Platform or ThoughtWorks' Foundational Technology Platform.
- The *data platform* performs analytics or trains machine learning models to provide better customer experiences and operational insights without disrupting operational systems.

In a simplified view, these platforms work together as shown in the following, variations of which can be found in many IT strategy documents):



A common blueprint for digital platforms

Updating an existing IT environment to reuse common platforms across front-end, core systems, and data is a major step forward for many traditional enterprises. However, the broad scope of such digital platforms is also their Achilles heel. Too often they're positioned as a cure-all for an organization's IT ailments, whether it's resistance to change, excessive operating cost, slow rate of innovation, or all of the above.



So-called digital platforms run the risk of being seen as a cure for any and all IT ailments.

That's a promise that hardly any product can fulfill, platform or not, particularly if lacking the requisite organizational changes. Many digital platform initiatives become proxies for actual organizational transformation, which is more difficult to put into place, as elaborated in "Transformation doesn't have a SKU" in *Cloud Strategy*. Such platforms end up as heavily funded multi-year initiatives that deliver more slideware than lines of code. The bigger these initiatives are, the longer they seem to be able to defer any measurable value delivery.

To embark on a digital platform initiative therefore must go along a clear roadmap with intermediate deliverables and the value achieved. [Plotting a platform's value curve](#) over time or adoption can be a useful tool to manage incremental value delivery.

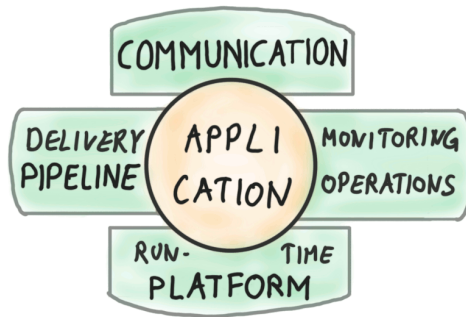
Engineering Productivity Platforms

Modern architects live in the first derivative⁶ because the rate of change can determine a company's future success. The critical element that determines software's rate of change is software delivery. Boosting software's first derivative, so to speak, is the declared goal of engineering productivity platforms, also referred to as internal developer platforms.

Such platforms provide the necessary components for building, deploying, and operating software solutions, thereby harmonizing in-house software delivery toolchains. Done well, they can assure consistent code quality or detect security vulnerabilities early, an approach referred to as "DevSecOps". The chapter "Application-centric Cloud in *Cloud Strategy*"⁷ illustrates the high-level architecture of such platforms in the form of "four-leafed clover", consisting of a toolchain, runtime, monitoring and communications:

⁶Hohpe, *The Software Architect Elevator*, 2020, O'Reilly

⁷<https://cloudstrategybook.com>



Four elements of an engineering productivity platform

Many elements of productivity platforms can be found in cloud platforms or in open source projects, attributing the core value of such platforms to the tighter integration and simplification of the individual tools. Composing in-house platforms from existing elements speeds up their development, but also increases the risk of settling for a [fruit basket](#).

Cloud platforms and code repository sites such as GitLab or GitHub have recognized the huge potential of productivity platforms and have started to include elements of the software delivery toolchain into their offerings. GitLab added [GitLab CI/CD](#)⁸, GitHub added [GitHub Actions](#)⁹, while AWS provides a set of tools around AWS CodeCommit, [AWS CodeBuild](#)¹⁰, AWS CodeDeploy, Amazon CodeGuru, and several others. Google Cloud, in turn, features [Google CloudBuild](#)¹¹ while Azure features [Azure Pipelines](#)¹² and also pitches GitHub Actions since GitHub is part of the Microsoft family. Needless to point out that most cloud providers offer comparable services.

With so much capability being added to the base platforms, in-house teams need to carefully consider their investments into productivity platforms to avoid [going underwater](#).

Data Platforms

Data platforms are another common sight in large IT organizations. Such platforms often follow an organization's strategy to become data-driven, looking to

⁸<https://docs.gitlab.com/ee/ci/>

⁹<https://github.com/features/actions>

¹⁰<https://aws.amazon.com/codebuild/>

¹¹<https://cloud.google.com/cloud-build/>

¹²<https://azure.microsoft.com/en-us/services/devops/pipelines/>

derive more value from data that currently leads a shadow life in isolated data silos.

Early attempts at making data more accessible relied on massive data warehouses operated by specialists. Realizing the slow rate of value creation from that data, those warehouses were later broken down into smaller data lakes to make producing and consuming data easier. Somehow the producing part worked out better, making many data lakes resemble the [Pyramid Lake in the Nevada Desert](#)¹³, which has inflow but no outflow.

Such setups were often hailed as “data platforms” (I warned about the dilution of the term) but were lacking some of the critical qualities of a platform:

- Adding a new data source remained labor-intensive, necessitating for example new ETL (Extract-Transform-Load) pipelines to be built and deployed. Due to high onboarding friction, they failed to [democratize access to the platform](#).
- Platform users were not able to extend the platform, often rendering the supporting platform team a bottleneck for required changes. Thus, the platform was not able to evolve.
- Platform evolution generally lagged behind vendors’ version upgrades, depriving users of features that are generally available outside of the platform. Subsequently, the platform did not enable but rather disable teams.

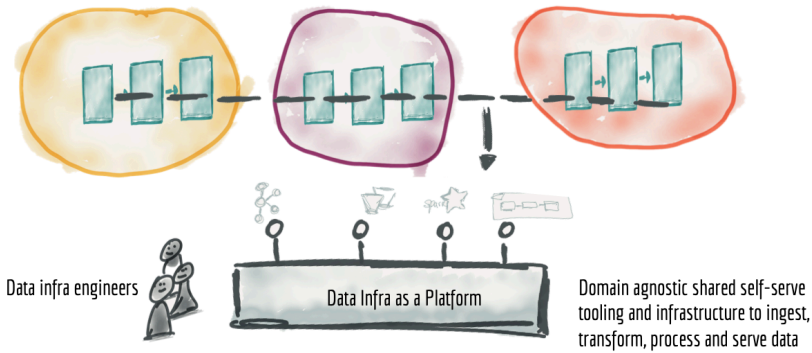
The teams did not actually build a platform but rather deployed a central system to be accessed by different users. While that sounds appealing in its own right, it’s not sufficient for a system to be classified as a platform. The [next chapter](#) dives deeper into the important differences between centrally operated services and platforms.

Data Meshes

Data meshes are looking to overcome the main limitation of those centralized systems. Data meshes separate the “innovation layer” from the platform layer (see [How to Move Beyond a Monolithic Data Lake to a Distributed Data Mesh](#)¹⁴):

¹³<https://www.linkedin.com/pulse/your-data-lake-like-pyramid-gregor-hohpe/>

¹⁴<https://martinfowler.com/articles/data-monolith-to-mesh.html>



Data Mesh uses Data Infra as a Platform - NEED PERMISSION

The shift to a data mesh rests on four pillars:

- Decentralizing ownership, decomposing data around domains
- Seeing data as a product, not just content. This includes aspects of ease of use, secure access, and trust
- Abstracting the infrastructure complexity into a common platform (removing the friction and duplication by raising the abstraction level)
- Providing central governance despite distributed ownership

All items steer data meshes to be much closer to our definition of a platform: reducing duplication and retaining governance while enabling independent usage and innovation.

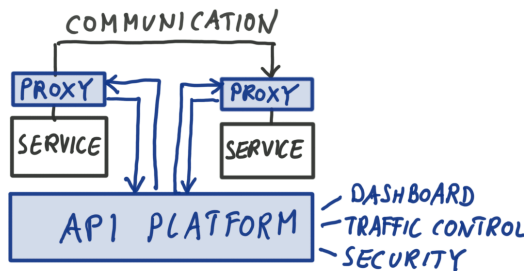
API Platforms

Communication is an integral part of most applications, especially in the era of microservices architecture and serverless run-times. It's also one of the leaves on the four-leaf clover of an application-centric platform.

Machine-to-machine communication between modern services and applications usually takes place via APIs (Application Programming Interfaces). APIs play an important role in platforms at multiple levels:

- Platform users typically interact with IT platforms through APIs, e.g. to request service instances.

- APIs allow enterprises to make their legacy assets available, for example, to be called from modern web or mobile front-ends (providing commensurate scalability, latency, or robustness can be a challenge, though).
- Internal APIs can foster reuse and create an innovation platform. Splitting a software monolith into smaller services accessible via APIs was one of the critical steps Amazon took toward becoming a platform.
- Externally-facing APIs can enable a digital ecosystem because they allow easy onboarding of partners and integration of services into third-party products.



A very simple example of an API platform

There isn't a razor-sharp definition of what's in (and outside of) an API platform. Many external-facing API platforms contain commercial or open-source API management products such as Kong, Apigee, Axway, or 3Scale. Many products operate in this space, ranging from little more than an API proxy to full-fledged developer platforms. Internal API platforms more likely operate on open-source service meshes like Istio or Kong, which form a control plane across internal API calls. API platforms consist not only of communication run-time but also of test suites like Postman. Most API platforms provide one or multiple of the following capabilities:

- Proxies to monitor and route service calls, such as an [Envoy proxy](https://www.envoyproxy.io/)¹⁵
- API gateways to manage operational concerns like authentication, quota enforcement, and throttling (often referred to as traffic shaping).
- Self-service portals for developers to sign up to use external APIs
- Application connectors to API-enable legacy applications
- Certificate management for secure service-to-service communications

¹⁵<https://www.envoyproxy.io/>

- A catalog or registry for API discovery
- Stream or event handling to augment traditional RPC-style APIs
- Monitoring and dashboards to provide observability and detect anomalies

The topic of APIs deserves its own book (with some luck, it'll be the next title in this series), so for the remainder of this book we consider APIs an enabling technology for platforms rather than a platform into themselves.

Abstraction / Cross-Platform Platforms

Another species of in-house platforms focuses on reducing the dependency on base platforms as described at the beginning of this chapter. My book “Cloud Strategy” contains a whole chapter on lock-in and the trade-offs involved in trying to eliminate it, highlighting the fact that a platform's benefit should be productivity first and portability second:



Focus on productivity first and portability second. If you're not productive (and your competitors are), you'll have nothing to port.

Nevertheless, abstraction platforms have gained in popularity but run the risk of becoming hindrances more than enablers. When such platforms can't keep up with the evolution of the underlying platform or require users to learn a whole new proprietary interface, they start to look very much like the [Grim Wrapper](#),

The push towards cloud abstraction platforms typically originates from infrastructure teams who in the past were tasked with managing vendors and lock-in. Intent on not completely losing control or, more positively, looking to apply existing compliance mechanisms to the new technology, such teams find an abstraction layer between developers and the underlying cloud platforms to be an attractive option. However, many of these layers don't fulfill the promises of a platform that enables developers and has low onboarding friction.

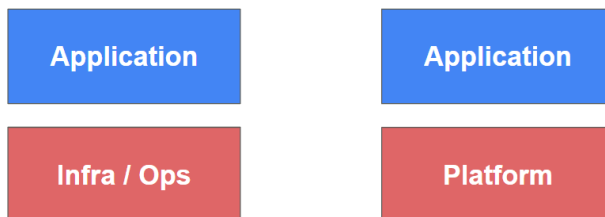
9. IT Platform and IT Services Org Are Antonyms

After renaming all teams still nothing improved...

The examples of common IT platforms in the previous chapter make the concept of in-house platforms more tangible. However, many enterprises might also consider their existing data center or IT services a platform, or at least face the temptation to apply that label. After all, they provision VMs on demand, which sounds a bit like a cloud, which in turn is a platform. As so often, things that might appear similar from far away reveal important differences upon closer inspection. Let's take a closer look at what makes a platform deserve the name and why traditional IT services are unlikely to pass that test.

Isn't it Just a Box on top of Another?

When we draw high-level diagrams of platforms (including most of the diagrams in the [Technology Platform Overview](#)), they tend to look like a big box of “common things” with another box (or multiple boxes) of diverse things on top of it. Such a picture looks oddly familiar to anyone who has spent time in large-scale IT:



A structural model cannot show the differences

Much of IT is structured into a common infrastructure and operations layer, on top of which diverse applications are deployed. So, where's the difference between that model and all the excitement about platforms?

A Static Model Can't Show Dynamic Differences

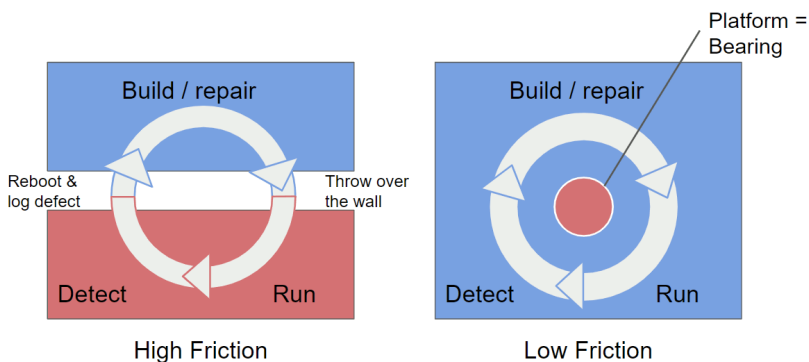
There are huge differences, but this structural model cannot show them. That's because this model is static—it only shows the pieces but not the interaction between those pieces. Platforms are everything but static, so we need a new mental model to explain the big difference.

The issues with the traditional “Dev and Ops” model are well-known. Application developers (“dev”) push for higher levels of flexibility and independence to release new functionality and meet customer expectations. They are also keen to try out new technologies to make sure that they use the best tooling possible and that their resume remains up-to-date. Infrastructure and Operations, in contrast, are more conservative, and perhaps resisting change, as they are tasked with maintaining stable and secure operations.

Putting the two together, you get a classic conflict of interest: developers throw half-tested software over the wall because the operations team has to wake up when the pagers go off. In return, operations can't do much to address outages besides rebooting a server and logging a defect for the development team to fix. This negative cycle is decorated with ample finger-pointing in both directions.

A Dynamic Model for a Dynamic World

A dynamic model makes the issue clear:



A dynamic model makes the differences clear

As the left part of the diagram shows, the loop across deploying software, detecting its operational characteristics (or usage metrics), and correcting them,

spans two organizational units whose opposing incentives pit them against each other. Often referred to as the “outer loop” of software delivery (the “inner” comprising the code/test/debug), modern software approaches like continuous integration place higher demands on making the outer loop spin faster and with less friction. In a traditional organizational model, the outer loop crosses organizational boundaries and inhibits these new ways of working.

Placing operational responsibilities in the development team, thus avoiding the organizational boundary, is one way to reduce this friction. This “you build it, you run it” model does indeed align the incentives into a single team, but it also burdens that team with a broad set of tasks and responsibilities. This means that cognitive load for teams increases (every developer now must also be a cloud and ops specialist) or team members will specialize (some folks are ops-heavy and others dev-heavy), which essentially reverts the state of affairs to the starting point, just under a common team label.

Placing operational responsibilities within the development team is the right setup, but those teams must have the right tools to perform these tasks as efficiently as possible. That’s where platforms come in. Developer/engineering productivity platforms can be depicted as the axle that helps the outer loop spin faster.



A platform team is tasked with building the axle that makes the outer loop spin faster. But it’s not part of that loop as it’d be bound to become a bottleneck.

The platform team is part of another loop, though, the loop that [takes input from development teams to evolve the platform](#). This loop spins much more slowly and does not directly impact the delivery teams’ velocity.

Platform Characteristics

Understanding that developer platforms are a stark departure from the traditional operational model, it’s easy to see how existing teams might believe they’re building a platform when in reality they’re not.



Platform teams telling me that they provision or operate resources for the development teams is a warning sign that might be stuck in the old model, just with a modern label.

That's why it's helpful to have a list of key characteristics that qualify an in-house project as a genuine platform. Consider these like a checklist that helps you determine whether something you're building is a platform or not:

Provides Value Indirectly

Platforms deliver value via other projects. Such value can be realized immediately by the platform users, for example by reducing projects' development effort. Platforms can also deliver value indirectly, for example by providing better transparency across an IT organization's project portfolio, which in turn allows better decisions or more effective resource allocation. This broad set of benefits reflects the nature of multi-sided markets for IT platforms. For example, an in-house software delivery platform might serve the following user groups:

- Project developers who can speed up software delivery
- Component developers who can find more users for their functional blocks
- Management who gains more transparency into workloads and resource utilization
- HR who can attract talent who are interested in working in a modern technology environment

Delivering value through platform users implies that a platform can only deliver value in combination with other projects or products. Running the risk of stating the obvious, this means that platforms without users generate no value. Platforms are indirect value enablers, not direct value creators.

Thrives on Scale

A project that is a one-off creation for another project isn't a platform. Platforms are built to host a wide variety of other projects, reducing duplication of common components while enabling diversity in project implementation. Platforms thrive on scale—the more users are on the platform, the more attractive it becomes to be on the platform. In comparison, popular IT systems often become victims of their own success, becoming bottlenecks and slowing the organization down.

Minimizes Marginal Cost

Successful platforms grow because new customers can sign up easily and at low cost for both the user and the platform. The latter implies that the platform's marginal cost for an additional customer is low.

Implemented well, IT platforms inherit this property from e-commerce platforms like Airbnb. Airbnb doesn't have to build any new rooms to sign up a new host, allowing it to scale almost infinitely. For in-house IT platforms, automation, self-service APIs, and building on an elastic infrastructure are common mechanisms to achieve this.

Reduces Friction

The mirror image of low marginal cost for the platform is low friction for users to sign up. Especially in traditional IT departments, this characteristic sets true platforms aside from more traditional IT processes that are locally optimized and require would-be users to submit complex ticket requests that result in a time-consuming (semi-)manual provisioning process.

It's worth pointing out that "low friction" doesn't always translate into "low barrier". In-house platforms and base (cloud) platforms often require a different mental model than traditional services. Users who adopted that way of working, for example by using declarative scripts for infrastructure provisioning, will find the friction to be low. Others will encounter a steep learning curve at first.

Supports Self-Service

Self-service is the way most IT platforms assure low friction. Instead of filing a ticket, teams directly provision a virtual machine. Transparency forms the other half of making a team self-sufficient: they need to have a good read on what's going on, so they can pick the right course of action through self-service. This way of working leads to a shared responsibility model (see below).

Has an independent lifecycle

As illustrated in the beginning, a platform isn't directly integrated into the application lifecycle. It's available as an always-on production service that often pools resources for standardized and automated management. Platforms evolve (see below) but at a different cadence of applications built on top of it.

Puts Customers ahead of Processes

Many attempts at in-house platforms are intended to restrict users to a given set of software libraries, third-party products, or specific processes. While the motivation is easy to understand—organizations are looking to harmonize, reduce complexity, and boost compliance, these goals can't stand in the way of platforms enabling their users. Recall that “platforms enable” was the very first benefit described in [Chapter 1](#).

Platforms therefore have to find a way to serve its customers ahead of the platform stakeholders. After all, without customers the platform will provide no value whatsoever. Many companies that are extremely successful with the platform model are notoriously customer-centric. Amazon even adopted “customer obsession” as one of its leadership principles and has managed to build not one but two successful platform businesses.

Is Centrally Built and Operated

Platform users don't have to concern themselves with the operations of the platform because it is operated by a dedicated team and is available as an “always on” service. This property is key to reducing the friction of teams operating on top of the platform.

Shares Responsibility

Most platforms have a shared responsibility agreement with their client projects. While the platform takes care of certain operational qualities like security or compliance, the client projects carry other aspects of the same qualities, such as availability, security, and compliance. You can't just deploy any old crappy application to a platform and expect miracles.

Self-service can be an important part of the shared responsibility, allowing client teams to perform operational tasks like monitoring resource usage or performing restarts. An apt analogy is cars: a manufacturer can equip the car with a seatbelt and an airbag, but you still need to be a responsible driver. AWS articulates their expectations clearly in their [Shared Responsibility Model](#)¹ for security and compliance.

¹<https://aws.amazon.com/compliance/shared-responsibility-model/>

Evolves Continuously

Most IT platforms evolve constantly, both in the versions of the services they offer (depth) and in the scope of services they provide (breadth). Platform users see the evolution as an advantage rather than a hindrance because the evolution provides them with a richer platform over time.



One of the critical decisions when building the *Agile Delivery Platform* inside a large enterprise was to allow the platform to be continuously upgraded to the latest version of the underlying software product (an on-premises PaaS platform). This decision ran against the recommendation of the traditional IT operations team, which was used to postponing updates until after all application owners agreed. The understanding of a shared responsibility model played a key role in allowing us to work this way.

Users Extend

Most IT platforms are [extensible by platform users](#), e.g. to share functionality they have developed. This approach stands in contrast to many traditional IT services organizations that are looking to maintain central control and give users very limited influence over the services that the platform provides. Platforms are often whereas IT services are generally closed.

The Platform Gestalt

A collection of characteristics is a great start but ignores that many of these characteristics support each other. As architects, we don't see characteristics as a simple checklist but also want to understand how they are connected - once again the lines are as interesting as the boxes (see "Drawing the Line" in *The Software Architect Elevator*²).

- A shared responsibility model enables continuous evolution, as described in the anecdote above.

²Hohpe, *The Software Architect Elevator*, 2020, O'Reilly

- Self-service is a key contributor to low marginal cost, the ability to thrive with scale, and low friction.
- Making a platform user extensible is the ultimate form of being customer centric.

As a side note, a similar effect applies to good pattern languages: individual patterns are useful, but a cohesive language that shows how the patterns relate is much stronger. The best description of designing pattern languages I have seen is in the fifth volume of the Pattern-Oriented Software Architecture (POSA) series³.

Honorable Mentions

My books don't aim to give you a single right answer because I believe we operate in a space where that's not possible and trying to do so could be a dangerous proposition. Rather, I try to put structured thoughts out for intelligent discourse. So, there are a few additional characteristics that are considered essential by some but that I didn't feel 100% comfortable putting in the above list. They therefore get an "honorable mention" here.

Voluntary Adoption

Many people prefer platform usage to be voluntary instead of mandated, and I tend to agree. Mandated usage tends to fly in the face of being customer centric (a tax authority might claim to be both, but I'll defer judgment on that one). At the same time I routinely remind folks that a large organization isn't a democracy—whether you like it or not, there are hierarchies and decision powers for a reason.

As a pragmatist, I caution teams wanting to make their platform mandatory that it's nearly impossible to enforce things in large, federated organizations. There are one-thousand-and-one ways to work around "required" tools and even if things come to a head, you won't have sufficient political capital to fight and win every battle. This isn't a Hollywood movie where the heroes prevail. It's enterprise IT and the hallways are littered with skeletons from those who fought the noble battle.

³Buschmann, Henney, Schmidt, *Pattern-Oriented Software Architecture Volume 5: On Patterns and Pattern Languages*, 2007, Wiley

What you can do, though, is make people’s lives significantly easier if they use the platform and harder if they don’t. If that sounds ever so slightly mischievous, just think of it as a core property of multi-sided markets. Most successful platform businesses subsidize early users or charge a different party to make a valuable product available at near-zero cost until they reach scale. If works for the “giants”, why shouldn’t you get a slice of it?

Managed by a Dedicated Team

Most platforms are managed by a dedicated team that acts like a small business of its own (see [Platform Inc](#)). This makes good sense, but I am reluctant to call it a defining characteristic. If your platform exhibits all the above characteristics but is managed in a community model, more power to you! You may also find the opposite, a [platform team that doesn’t actually build a platform](#).

Platform ≠ IT Service

As elaborated at the beginning, IT platforms often evolve (or are occasionally re-labeled) from more traditional IT Services, but have entirely different characteristics. A side-by-side comparison can highlight the contrast between developer platforms and traditional ITSM (IT Service Management):

Characteristic	Platform	IT Service
Scale Effect	Thrives	Bottleneck
Marginal cost	Low	Medium to High
Friction	Low	High
Interaction	Self-Service	Ticket-based
Orientation	Customer-Centric	Process-Centric
Responsibility	Shared	Separated
Evolution	Continuous	Sporadic
Extensibility	Open or semi-open	Closed
Adoption	Voluntary	Mandated

This list can help debunk “lipstick on a pig maneuvers” maneuvers that re-label existing processes and operational systems as platforms. I did include the voluntary adoption characteristic here for the sake of completeness.

Non-Technical Aspects

The platform model extends far beyond the technical aspects to also include organizational and operational aspects. That’s why this book contains a whole part on [Organizing for Platforms](#). Several definitions of IT platforms include the elements of services, knowledge and support (See [What I Talk About When I Talk About Platforms](#)⁴).

⁴<https://martinfowler.com/articles/talk-about-platforms.html>

10. Mechanisms not Magic

Making things work is not an implementation detail

Business and organizational goals for [in-house platform development](#) are plentiful: platforms help enterprises speed up innovation, reduce cost, increase compliance, foster code reuse, enable autonomous teams, boost morale, and make the organization more attractive to prospective employees. However, these benefits don't magically materialize just because a major piece of the IT estate is (re-)labeled as "platform". Like many other IT projects, many platform initiatives suffer from a weak linkage between well-stated ambitions and technical implementation proposals—the [dangerous hourglass](#) mentioned in Part II.

Platform proposals also routinely pitch different benefits to different stakeholders: developers are promised more productivity and autonomy whereas operations teams are assured better transparency and stricter governance. As highlighted in [Part II](#), platforms can overcome past constraints such as harmonization inhibiting innovation. However, that doesn't happen automatically but relies on a carefully crafted combination and balance of platform characteristics.

A successful platform strategy must clearly connect expectations and implementation details. Detailing how the technical aspects realize the anticipated benefits also enables a more balanced consideration of alternatives. Let's see the *Architect Elevator* in action, connecting the levels of the organization. Interestingly, the solution lies in yet another buzzword.

Making Things Simpler isn't Simple

Base platforms, such as cloud platforms or comprehensive enterprise products, are powerful, but can be complex to set up and overwhelm users with choices. Shielding users from this complexity is a legitimate goal. Reduced complexity brings many benefits: it speeds up developments, reduces the chance of errors, and makes it easier to onboard new staff.

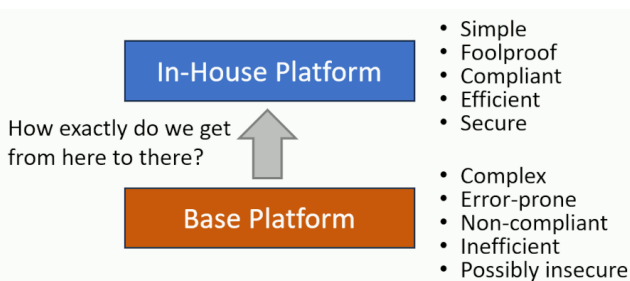
Many organizations lack skilled developers who can build sophisticated applications on top of commercial cloud platforms due to their large and ever-increasing surface area consisting of hundreds of services, distributed architectures, access control, network partitioning and much more. They therefore look towards in-house platforms to soften the learning curve for developers and also to keep them from making avoidable beginner mistakes.

However, building a platform often turns out to be an even more challenging tasks that requires developers not just skilled in cloud development but also in platform development. As a result, such organizations find themselves between a rock and a hard place.

Platform Marchitecture

A disconnect between a platform's stated objectives and the implementation is the surest way to sabotage a platform strategy. Such a platform sounds good on paper, but "leaving the details to the technical implementation partner" all but guarantees that reality won't fulfill those expectations. And that means that the [strategy remains just wishful thinking](#).

The symptoms of such a disconnect often appear when in-house platforms layered on top of commercial cloud platforms are pitched as a cure-all:



Platform Marchitecture based on wishful thinking

Such glorified [marchitecture](#)¹ diagrams (usually they look more snazzy than this one) emphasize the great benefits of a potential in-house platform. Alas, they fail to elaborate how exactly one should expect to get from "there" to "here". Considering that the underlying cloud platforms have been built and refined

¹<https://en.wikipedia.org/wiki/Marchitecture>

over decades with billions of Dollars of investment, one should describe clearly how they can do so much better.

When asked to elaborate on the platform implementation, presenters routinely resort to technical details or broadly used slogans like “it’s all open source, loosely coupled, and componentized”. However, this obfuscation maneuver may silence the challengers, but it surely doesn’t increase the odds of a successful implementation. Architects can do better than this.

Mechanisms provide linkage

As [previously discussed](#), spelling out the core mechanisms that are expected to achieve the advertised benefits brings much-needed rigor into such discussions. I would go as far as stating:



Never believe a technical proposal that is soft on how and why the technical implementation achieve the advertised benefits.

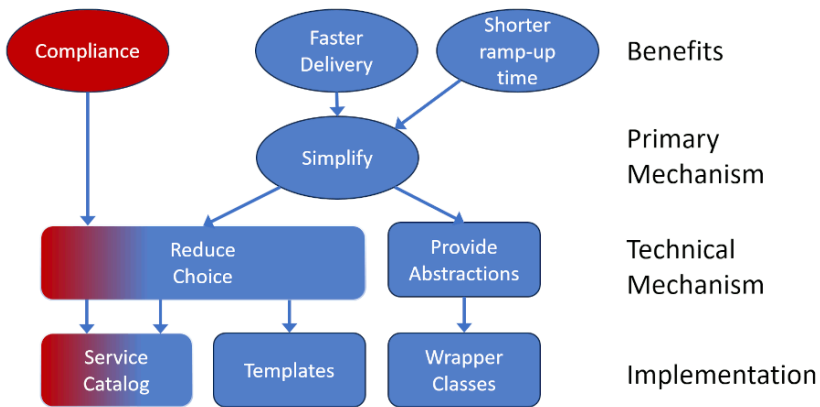
Separating the strategy into three distinct layers can help achieve a better outcome. Or at least, it’ll show the dire state of the middle layer, which shows the mechanisms:

- *Benefits* are desirable properties. They describe the value to be achieved. Beware of proxy or vanity goals like “simplification” or “harmonization”, though: they are merely proxies for something else that you are looking to achieve, for example better economics or faster delivery.
- *Mechanisms*, yet another fuzzy term in the league of “architecture” and “platforms”, explain how the goals can be achieved by specific technical implementations.
- *Implementation* details describe the other end of the mechanism, i.e. what needs to be built or used.

Wikipedia defines a mechanism as a device that transforms input forces and movement into a desired set of output forces and movement. Never short of a car analogy, a crank (as in crankshaft) combined with a connecting rod is a device that converts linear back-and-forth motion of pistons into a rotating motion of

the crankshaft and ultimately the wheels of a car. This mechanism has been in use for almost two millennia, so it serves us well to respect its significance.

Platform mechanisms might not last that long, but they nevertheless provide the necessary linkage that allows us to understand how the platform will function. There isn't a 1:1 mapping between technical implementation, mechanism, and benefits, as the following diagram illustrates.



Mechanisms as logical connecting element

Simplifying, or reducing cognitive load as we will soon discuss, is a mechanism that can fulfill multiple goals, such as speeding up development or reducing the time it takes a new team member to onboard. Simplification is itself still a fuzzy concept that can be achieved in many ways, for example by reducing the number of choices a developer has to make. Useful abstractions like APIs or network sockets have reduced cognitive load for developers for decades and can be valuable for in-house platforms as well.

One implementation to restrict choice are templates that fix certain parameters while leaving others to be set by the developers. Other implementations rely on cloud services like service catalogs that define which services or geographic regions a developer can use. Both service catalogs and templates can also increase compliance, a worthwhile but entirely different benefit.

Such many-to-many relationships, when not properly understood or spelled out, can lead to miscommunication or missed expectations. For example, a platform team might implement a templating system to achieve compliance because the template won't allow developers to set sensitive or undesired settings. This implementation is then pitched to developers as reducing cognitive load, which

may not actually come to fruition.



Settings that are fixed or defaulted often determine relevant aspects of the system behavior and can't just be hidden from developers. In the most simplistic example, fixing the geographic region of a cloud platform just to one setting doesn't alleviate developers from knowing regions. It can also have implications on latency and resilience.

Instead, cognitive load could have been reduced more by introducing meaningful abstractions like a “Single Region Deployment”, which makes the concept explicit but easy to implement.

Vice versa, too often specific implementations are conflated with mechanisms or goals. For example, governance can be achieved through the mechanism of restricting choice up-front (by not allowing certain settings or services) or by validating choices that were made (e.g. through automated checks). Teams that skip mechanisms might fixate on one implementation without considering alternatives.

Welcome to the Twilight Zone of Architecture

Technical mechanisms aren't a paint-by-numbers exercise for better architecture. They are a great vehicle to create a better strategy. Sadly, the value of clearly spelling out mechanisms is often overlooked because they tend to live in the twilight zone of high-level architecture:



Technical mechanisms can be deemed too conceptual / fluffy by technical teams but too technical for decision makers and high-level architect.

Some simple mechanisms, like default settings, are widely used and well known. That's why folks might not even consider them worth including, but that'd be a mistake because it'd ignore the trade-offs involved even in such simple mechanisms. As we learned above, default settings can reduce or increase cognitive load, so they deserve a nuanced treatment.

Cognitive Load

Cognitive load has almost become synonymous with in-house platforms. Cognitive load was characterized in 1988 by psychologist John Sweller as “the total amount of mental effort being used in the working memory” and has been the center point for the impactful book *Team Topologies*². The core hypothesis of the book is that excessive mental load slows teams down and makes them more prone to mistake. Therefore, platforms that look to enable or speed up development teams should reduce the cognitive load.

Reducing cognitive load is a worthwhile pursuit, but it might be easier said than done. Abstractions can be a useful mechanism to reduce mental load, but many teams have learned that [finding good abstractions is difficult](#)³ and that building [dangerous Illusions](#) instead of useful abstractions is a real risk. Illusions initially appear to reduce the cognitive load but lead to bad surprises because they fail to provide the developers with necessary details to implement acceptable solutions.



Removing choices or elements does not automatically reduce cognitive load—it can have the opposite result.

It’s too easy to believe that taking choices or elements away would automatically reduce cognitive load. However, that’s oversimplified and often not true. An apt metaphor is trying to make a 1000-piece jigsaw puzzle easier to solve by taking 500 pieces away. Actually, you made solving the puzzle a lot harder! Instead, you would need to create a new puzzle of 500 (larger) pieces, which is much harder.

Platform Mechanisms

Although mechanisms don’t provide a 1:1 linkage between implementation and benefit, cataloging a range of mechanisms is helpful in shaping a platform strategy:

²Skelton, Pais, *Team Topologies: Organizing Business and Technology Teams for Fast Flow*, 2019, IT Revolution Press

³<https://architectlevator.com/cloud/abstractions-difficult/>

Restricted choice

Picking a “golden path” out of an abundance of choices seems like a great way to reduce complexity and also provide governance. Governance places guard rails around the use of a base platform to meet regulatory demands or a (valid) desire to reduce complexity. But it might also eliminate useful options and slow down development teams’ progress. For example, if a competitor allows development teams access to a valuable cloud service type, restricting such access can turn out to be a competitive disadvantage. A later chapter on [wrapping](#) dives deeper into techniques to restrict choice.

Platforms can do good or harm when restricting choice. Let’s not blame the tool but remember that [platforms should be an enabler](#). Restricting choice therefore should not slow down progress or stifle innovation. Instead, it can help reduce complexity. We’ll dive into how to achieve this in the following part.

Meaningful Defaults

A softer variant of restricted choice are meaningful defaults. Defaults don’t try to hide the existence of certain settings but avoid mistakes by providing useful values if not set by the developer. Developers might benefit from a smoother learning curve because they can ignore some settings in the beginning and tend to them later when their needs evolve.



Picking meaningful defaults isn’t as easy as it sounds. For example, when I built a [serverless demo application on GCP](#)⁴, I was happily surprised that I was able to ignore security roles and access control. With AWS, the default settings initially disabled my entire application and provided additional hurdles along the way. However, the application built on AWS was better suited to be placed into production as it defaulted security settings to “no access”.

Assumptions / Scope

In conversations with platform teams I often remind them:

⁴https://www.enterpriseintegrationpatterns.com/ramblings/loanbroker_gcp_workflows.html



A cloud provider has to build for the whole world. An in-house platform only has to build for one organization.

The somewhat philosophical statement is intended to remind them that they can make more assumptions than a cloud provider can. These assumptions can translate into other mechanisms like restricting choice or stand on their own.

Aggregation

The base layer may contain all the things you need but spread across multiple systems. For example, a data platform might provide uniform access to data elements that were previously spread across legacy systems. This mechanism also reduces complexity but from a slightly different angle. Instead of looking to just reduce the depth of the existing systems, combining things looks to reduce the breadth of systems that developers need to deal with.

Abstractions

Abstractions are an often overlooked or misunderstood mechanism. This book therefore dedicates an entire chapter to [Abstractions and Illusions](#).

Automation

Friction is the cause of many ailments. In-house platforms often look to reduce such friction, for example by automating tedious manual steps needed to dealing with infrastructure or legacy systems. Complexity can lead to friction but friction originates from many sources, such as lack of documentation, inconsistency, having to deal with many departments, so we consider it a separate motivation.

Functional Addition

Your base platform may provide many of the functions that you need but might still have gaps that don't address your specific needs. For example, a cloud platform may give you all the storage and compute options you desire, but your software delivery process might have specific needs that aren't built into the base platform. You therefore build your own platform to augment the available

baseline. If you follow this route, a key design decision will be whether to develop a complete abstraction layer or simply an add-on to the existing base layer.

Mapping Mechanisms

Although there isn't a 1:1 mapping, the following table summarizes common combinations of business and technical objectives and the associated implementation mechanisms. It is by no means intended to be comprehensive but rather just to provide examples. The implementation mechanisms are described in more detail in Part II.

Business Objective	Mechanism	Implementation
Minimize mistakes	Meaningful Defaults	Templates
Increase velocity	Automation	IaC Scripts
Improve products	Fill product gaps	New components
Enforce compliance	Restrict choice	Wrappers
Reduce lock-in	Abstraction	Service layers

Non-technical Mechanisms

Software delivery organizations are [sociotechnical systems](#)⁵. Hence, looking at technical mechanisms alone might miss some of the most impactful strategies. For example, you can shorten the learning curve for teams by reducing cognitive load of the platform interfaces, or you can provide training or hands-on assistance. Each approach achieves similar results, despite having different pros and cons (reducing cognitive load is difficult but scales well to a large number of developers, whereas training is easier but doesn't scale as well). In the extreme case you may find that you can [achieve the platform strategy's goals without actually building any platform](#).

⁵https://en.wikipedia.org/wiki/Sociotechnical_system

11. Do You Have an Opinion? A mind of your own?

Why developers love opinionated platforms but despise restrictive ones.

Productivity platforms [abstract away the complexities of the underlying technology or base platform](#). They often do so by reducing choice or making assumptions. However, reducing choice invariably leads to complaints from development teams because it is seen as (and often is) [disabling rather than enabling](#) teams. Oddly enough, those same developers devour opinionated frameworks and celebrate being opinionated as a virtue. So, where's the difference between an opinionated platform and a restrictive one?

Being Opinionated

“Opinionated” is a common word in developer circles and is typically used with positive connotations. A [StackOverflow](#)¹ user describes it as follows:



Opinionated software means that there is one right way to do things and trying to do it differently will be difficult and frustrating. On the other hand, doing things that particular way can make it very easy to develop as the number of decisions that you have to make is greatly reduced.

Both the Heroku platform-as-a-service and the [Rails](#)² framework are famously opinionated and widely revered because of it. Another example, highlighted by the [Basecamp design principles](#)³ is the Wiki. Designed by Ward Cunningham,

¹<https://stackoverflow.com/questions/802050/what-is-opinionated-software>

²<https://rubyonrails.org/>

³<https://basecamp.com/gettingreal/04.6-make-opinionated-software>

the key aspect of the Wiki was the features that it lacked, such as fancy formatting or document ownership. In return for removing those features, it emphasized document linking by making it very easy to refer to another document just by typing the name with initial capital letters. This focus on ease of use and interlinking over fancy formatting made the Wiki the ideal basis for Wikipedia.

Basecamp itself is a great example of opinionated software and the design guides state this rather clearly:



They say software should always be as flexible as possible. We think that's bullsh*t. The best software takes sides. Decide what your vision is and run with it. If they don't like your vision, there are plenty of other visions out there for people. Don't go chasing people you'll never make happy.

When it comes to framework or product design, being opinionated translates into a strong vision and focus that benefits one well-defined user group as opposed to a diluted experience for a broad market. It thus follows the common wisdom of having clarity about the target segment and building a product specifically for that segment. Or, as we were told in our entrepreneurship class at university:

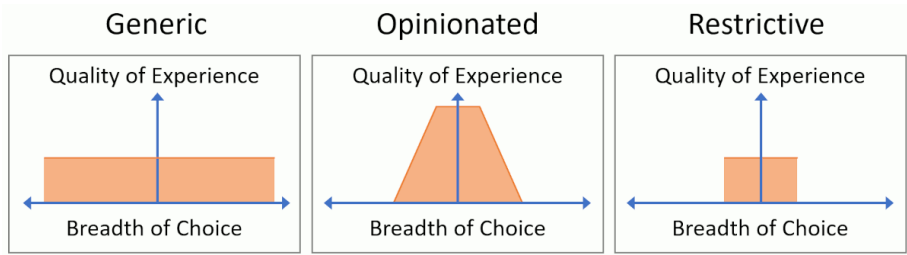


Business plans that describe how you would capture a small slice of a vast market are the first ones to go into the trash bin. Instead, we look for ideas that serve one market better than anyone else.

So, it appears that venture capitalists also like opinions, and some of them are also known to have strong ones.

Opinions have a Shape

So, if developers love opinions and opinions are healthy for product design and business plans, why do developers resist in-house platforms that have a specific and narrow focus? Why are those not considered opinionated, but seen as limiting and constraining, causing developers to engage in all sorts of maneuvers to avoid them? As with many architecture topics, a visual model, aka diagram, can provide insight:



Opinions give you something in return

The image summarizes the key differences between opinionated and restrictive frameworks:

Return on opinion

Opinionated frameworks give you a high return on following their opinion. For agreeing, you are rewarded with a simpler and seamless developer experience that makes you more productive and keeps you from making mistakes. In contrast, restrictive frameworks or platforms limit choice without giving anything in return. For example, if a developer platform restricts access to underlying cloud services, using the allowed services is no better than using the cloud platform directly, meaning the developers didn't benefit from following the opinion. That's why it's important to consider [mechanisms to reduce cognitive load](#) that can reduce cognitive load and thus provide platform users with tangible benefits.

Gentle slopes

Opinionated frameworks tend to have gentle slopes at the edge of their opinion ([the terrain](#)), meaning that the developer experience is still good even if you're looking to do something slightly outside the framework's sweet spot. Popular mechanisms are default overrides or "escape hatches" with the former sounding more comfortable than the latter. Restrictive platforms, in contrast, don't want you to go near (or beyond) the edge. If an in-house platform restricts usage to specific services, there's little you can do if you don't agree (besides perhaps circumventing the platform altogether, essentially jumping off the cliff). Restrictive platforms often use the term "happy path" or "golden path", but in reality that's the only path they allow you to take. Whether you're happy on that path or not is a different matter altogether.

Transparency

I wasn't able to show this in the diagram, but opinionated frameworks are

transparent about their opinions and the motivation behind them. They state their opinions clearly, telling users what they are getting into, and giving them an opportunity to not get into it, if they so choose. Restrictive platforms sadly are often opaque, leading to frustrating experiences when they **reject developers' work without giving clear reasons**.

Successful opinionated platforms do all three things well. They are transparent on the constraints, the benefit that this constraint provides, and provide mechanisms to deviate from the prescribed path where needed. Rails (aka Ruby on Rails) is a great example:

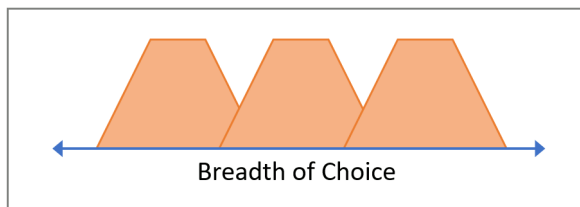


Rails put a **Convention over configuration**⁴ philosophy at the center of the framework and delivered higher developer productivity in return.

By making clear what developers will lose (e.g., explicit code or string abstraction between database and application) and what they will gain (compact code, high productivity), the framework would be sure to only attract those developers that would be happy with those trade-offs.

Choose your opinion

Opinionated frameworks have one other fundamental advantage: they are not the only choice. If you don't like one framework's opinion, you can pick another one. So, you aren't restricted at all because you can match a framework for almost any opinion:



Many opinions give you choice

⁴https://en.wikipedia.org/wiki/Convention_over_configuration

Restrictive platforms, especially in-house ones, typically offer no alternatives, making it a “my way or the highway” proposition. So, a key part of making developers agree with an opinionated framework is to present them with choice, or perhaps the illusion of choice if we borrow some philosophy from my favorite source of movie quotes, The Matrix:



“Choice is an illusion created between those with power and those without.” – Merovingian

Such platforms can only succeed if the return on opinion is high, meaning the developer experience is particularly good, and if the reasons for the chosen restrictions are made transparent so that they are not perceived as *fait accompli*.

Unnatural selection

When discussing the merits and demerits of internal platforms on social media, one reader gave the following candid input:

When a company decides to write a proprietary platform, it competes against industry alternatives. Unfortunately, in 90% of the cases the internal tooling teams are underfunded as a cost center and release buggy trash.

Opinionated developer frameworks are subject to some form of Darwinian selection by the developer community. Rails became popular because developers embraced it. Many other frameworks that aimed at doing the same likely fizzled out, and we would have never heard of them. So, in a way developers embracing popular opinionated frameworks is the inverse cause-and-effect: these frameworks are well-known because many developers liked them.

The pointed comment hints at the challenges of building in-house platforms. If it were easy, we likely wouldn’t even need this book. As we’ll [discuss later](#), building platforms requires a keen product mindset that is usually not required inside IT organizations. Failing to make this drastic shift in mindset is an equally powerful contributor to releasing “buggy trash” as overly tight budgets.



If you want developers to love your platform, make sure to build a lovable platform.

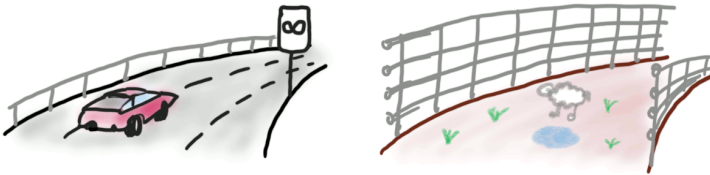
So be blunt, a lot of in-house platforms struggle with adoption because they're not really meeting the developers' needs.

Freeways are Opinionated

Never short of a car analogy, I compare opinionated frameworks to freeways:

- Driving on a freeway, following the prescribed path, is very efficient and gets you to your goal much more quickly than taking the back roads.
- Freeways also constrain: you can't just go anywhere but have to follow the prescribed path.

In comparison, restrictive in-house platforms look more like obstacle courses that have you hit sudden roadblocks for no apparent reason and without any advantage—not a lovable product. Too many in-house platforms promise a freeway-like experience but end up delivering a mud path with giant guard rails, representing governance and security:



Golden path: Expectations vs. Common Reality

This isn't called opinionated; it's called *bait-and-switch*.

Are the Streets Really Paved with Gold?

The path prescribed by opinionated platforms is often called the “golden path”, a term popularized by [Spotify](https://engineering.atspotify.com/2020/08/how-we-use-golden-paths-to-solve-fragmentation-in-our-software-ecosystem/)⁵ and apparently borrowed from the Dune sci-fi saga. Whether all platform roads are paved in gold remains to be seen because too many IT organizations don't focus on making the golden path particularly happy but rather on closing off all other paths:

⁵<https://engineering.atspotify.com/2020/08/how-we-use-golden-paths-to-solve-fragmentation-in-our-software-ecosystem/>

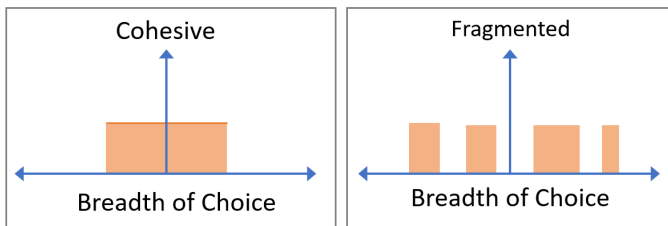


The so-called “happy path” isn’t actually that happy; it’s more like the other paths are closed off.

Although that matches the original Dune reference where all other paths lead to humanity’s extinction, it might not be the best approach to gain [widespread adoption and enthusiasm for your platform](#).

Cohesion

There’s another difference between an opinionated framework and a restrictive platform. Restrictive platforms often restrict services or features from an underlying base platform. What is removed is often driven by outside considerations, such as regulatory requirements (or rather, the interpretation thereof), or serendipity, such as which service could be certified first or was demanded most. As a result, restrictive platforms often lack cohesion but become a collection of odds and ends:



Opinionated platforms tend to be cohesive

An opinionated product, in contrast, has integrity and cohesion—that’s how it can improve the developer experience.

Employer lock-in

There’s a last consideration that can stir resistance among developers, even if your platform is beautifully cohesive and provides a great developer experience along with necessary escape hatches: lock-in. No, not locking into a third-party vendor but lock-in into your own organization. In-house built frameworks are proprietary. If developers invest large amounts of time in learning such a

framework, there's zero return on that investment in case they want to go work somewhere else.



In-house frameworks increase developers' switching cost, making for a form of "employer lock-in" that they may resist.

In comparison, working extensively with open-source platforms like Kubernetes likely increases developers' market value, so they might prefer it over in-house platforms. Although clearly a form of undesirable "resume-driven architecture" in times of skill scarcity, employer lock-in is a real concern for platform teams.

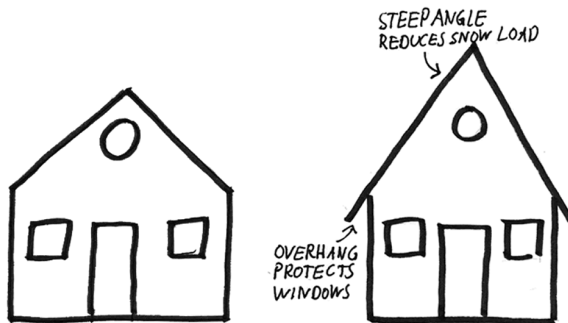
12. Making Platform Decisions

Strategy is a series of meaningful decisions

The Software Architect Elevator reminds us that architecture is best represented by a set of conscious decisions. The same applies to the design of in-house platforms. Although designing a platform is far from a paint-by-numbers exercise, a checklist of major decisions and their trade-offs can be a valuable tool to start a platform strategy.

The most important decisions might be the ones you didn't know you made

The Software Architect Elevator features a simple but powerful picture that helps you identify what makes up the essence of architecture, even if it comes in the form of a simple sketch:



Is this architecture?

The sketch of a house on the left has most of the elements that are featured in popular definitions of systems architecture, such as the main components of the system (door, windows, roof) and their interrelationships (door and windows in the wall, roof on the top). Yet, when building a house, I would not want to

pay an architect for this sketch because this “cookie-cutter” house is lacking any non-obvious decisions that an architect would have made.

The sketch on the right is equally simple and poorly drawn, and the house is almost identical except for the roof. This house has a steep roof and for a good reason: the house is designed for a cold climate where winters bring extensive snowfall. Snow is quite heavy and can easily overload the roof of the house. A steep roof allows the snow to slide off or be easily removed thanks to gravity. Additionally, an overhang prevents the sliding snow from piling up right in front of the windows.

Some people might dismiss the decisions on the right as trivial, but that’d be missing an important point.



A dangerous trap early during a platform development cycle is the team making decisions without being aware of it.

As I pointed out in a [blog post](#)¹, teams are prone to making decisions tacitly or just taking them for granted without evaluating alternatives. Those tacit decisions either result in poor design trade-offs right from the start or turn into hidden assumptions later on, which makes evolving the platform into a frustrating guessing game. A decision catalog therefore serves as a useful checklist that allows you to detect whether you made decisions without being aware of it.

Truth vs. Comfort

Some platform decisions can be controversial. For example, some users may not like a high rate of change or would want you to commit to maintain the platform forever (or their retirement, whichever comes sooner). A popular sales strategy is to gain commitment from buyers early and let them in on important details once they have committed and are unlikely to back out (“there’s actually an additional delivery charge” / “the price I quoted is valid with a 3-year commitment”). Although most of us will find this annoying, it does work. So, stating your platform decisions and the implied trade-offs early may cause you some pain. But, if you’re in it for the long run, it’ll avoid late surprises for your users and allows you to sleep at night.

¹<https://architectlevator.com/architecture/important-decisions/>

Decision Catalog

Platform teams face numerous design decisions, but trying to compile an exhaustive list would not do the complexity of the topic justice. Also, not all decisions are either-or, and virtually all of them carry trade-offs, meaning “either” isn’t necessarily better than “or” and the opposite can be true in some instances. As warned in the introduction, the goal of this book isn’t to answer all questions but to raise better ones and equip you to find better answers. That’s why this chapter lists questions, not answers.

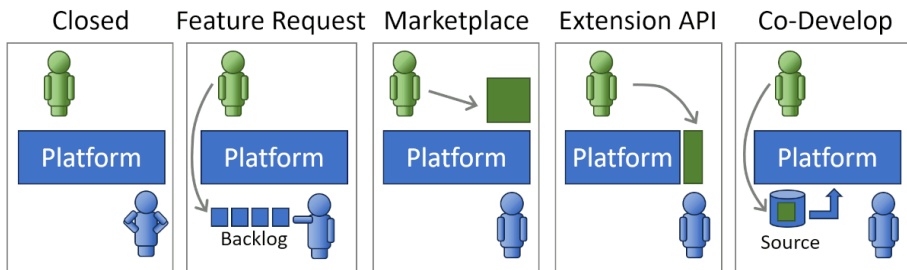
This chapter focuses on the externally visible platform choices, i.e. those that affect the platform users. Platform-internal implementation choices are the focus of [Part V - Implementing Platforms](#).

Degrees of Extensibility

Platforms are enablers for other teams and thrive because [they do not anticipate all use cases](#). In an ideal state, a platform should be flexible enough to allow platform users to do anything they desire. In reality, though, most platforms provide extension or customization mechanisms for platform users to achieve use cases that haven’t been anticipated by the platform. The following gradation shows how different platforms allow user contributions:

- **Closed:** The platform is fully under the control of the platform team with very limited input from the platform users. This model should be rare for in-house platforms and can be an antipattern as it lacks customer focus. However, there’s always an exception to the rule: for example, a platform might need to be rigid due to regulatory oversight or required certifications.
- **Feature requests:** The platform is fully under the control of the platform team, but the team encourages feature requests from users and maintains a public backlog. Taking feature requests doesn’t imply that the [team builds exactly what the users ask](#). Most cloud platforms follow this model by taking in customer feedback.
- **Marketplace:** Users can’t modify the platform directly but can offer components to be used by other platform users. Many commercial platforms actively support such marketplaces. For example, virtually all cloud platforms have marketplaces for machine or container images offered by third parties, enabling many use cases that aren’t part of the base platform.

- **Extension API:** The platform allows users to develop components that will become part of the platform, often via dedicated extension APIs (discussed in more detail in [Anatomy of a Platform](#)). A canonical example are Kubernetes Operators that extend the Kubernetes container orchestration platform to become available via the Kubernetes API. Unlike the marketplace components, these extensions sit inside the platform.
- **Co-Development:** Platform users can be actively involved in the platform development. This type of collaboration follows an open source or [inner source](#)² development model and is therefore rarely seen in commercial platforms.



Platform collaboration models

Free or Charged?

Building platforms isn't free, but building platforms pays off. Platforms deliver value to the organization indirectly, primarily by enabling platform users to deliver products faster or to innovate more freely. If upper management is convinced of the platform's value, you have the luxury of offering your platform for free. Not charging for the platform can increase adoption and thus increase the overall (indirect) value delivery, which makes for a positive business case despite not requesting any direct charges.

Many IT organizations require projects to recover their investments from their users through a direct pricing model. Although it's tempting to blame an over-eager CFO, the reasons can lie much deeper, for example in tax codes. Let's say one division of your large organization maintains a large team to build an amazing in-house platform. This team happens to be located in a high-tax country. The platform is so amazing that little work remains to be done by

²https://en.wikipedia.org/wiki/Inner_source

the development teams, who happen to be located in low-tax countries. If the platform is free to use, you are essentially shifting profit from a high-tax location (where you allocate most of the development cost) to low-tax locations (where you increase profit thanks to minimal development cost). Such a maneuver is likely to raise eyebrows in highly regulated and monitored industries like financial services who'd want to reflect that transfer of value from one country to another. This is a classic *Architect Elevator* scenario where architectural decisions are driven by business context. Architects who are well versed in the very top floors might point to clever strategies like a [Dutch Sandwich](#)³, but although I prefer architects to take a broad view on their role, I leave this topic as an exercise to the reader.

Offering a platform for free, while well-intentioned, can also lead to undesirable side effects. For example, numerous small projects may adopt the platform, leading to a significant support and maintenance burden that may slow platform evolution or make it difficult for the platform team to support major users. Platform pricing is a sufficiently complex topic that we'll dedicate [an entire chapter](#) to it.

Mandated or Voluntary?

When listing [platform characteristics](#), making platform usage voluntary was listed as a desirable property but not a required one. Voluntary adoption assures that the platform team listens to customers and evolves the platform based on customer needs. Besides, project teams are guaranteed to become creative when it comes to circumventing mandated tool usage, e.g. by using the mandated tool for an insignificant part of the project to claim compliance while using other tools for the majority of the project.

On the flip side, voluntary platforms require teams to spend valuable resources promoting their platform, especially early during the adoption cycle when word-of-mouth marketing is not yet happening due to a small user base.

Expected Lifespan

Nothing in IT lasts forever, so it's a good idea to give some indication of the intended platform lifespan, both in absolute terms and in evolution over time. In-house platforms don't exist in a vacuum but are subject to shifting developer needs and preferences, and [base platform evolution](#).

³https://en.wikipedia.org/wiki/Dutch_Sandwich

Although most platform teams would love their platform to live forever, few IT systems do (and those are often a burden for its maintainers). So it's helpful to give your users some notion of what the long-term plan for the platform will look like: will you go into maintenance mode at some point? Do you expect it to be [underwater](#) at some point and will happily sunset it? Will you open source it or pass it to the internal community? There are more choices than might initially appear, so it's good to lay out options and choose wisely.

Can the platform shrink?

The base assumption might be that a platform grows over time. However, a [floating platform](#) also retires functions. Some cloud providers have a [long history of sunsetting services](#)⁴, but most other providers have also [started to prune services](#)⁵ to focus their teams. Most IT environments have become complex because they only add but never take away, so pruning your platform is a sound idea. In any case, it's important to set these expectations with platform users right from the start.

Rate and cadence of change

Continuous evolution is a [key characteristic](#) of platforms. That chapter also tells an important anecdote about the consequences of continuous change. Too many large organizations allow applications teams to veto change. I recall when our organization looked to standardize on a version control system. Git was a somewhat natural choice, but Subversion was also allowed because exiting teams would not want to move off it. Worse yet, we routinely paid millions of dollars for extended support contracts because teams could not move off an unsupported version on an application server or similar product.



If your organization allows application teams to veto changes, your platform may not succeed.

Platforms should have a low rate of change, but it'll never be zero. Early in the platform journey you should define:

- What will change and what will not (e.g., you could keep existing APIs stable but add new ones)

⁴<https://killedbygoogle.com/>

⁵<https://honeycodecommunity.aws/c/announcements/7>

- What the expected rate and cadence of change is (e.g., each quarter or as needed but generally less than once per month)
- How you will support teams with the change (e.g., do you provide test environment for the new version, maintain multiple versions at any point in time)

These decisions will mimic traditional service management processes (hello, ITIL!), but they do apply to in-house platforms just as much.

Preconditions and assumptions

As with any product, be clear on what you assume to be in place for developers to use your platform, including skill sets, ways of working, etc. A great platform cannot deliver value if the teams lack the preconditions to use it.

13. Procuring a Platform

This chapter is still work in progress.

Not all platforms have to be built. Organizations will also buy platforms that someone else built, especially cloud platforms. Also, tools that help you build platforms are starting to emerge, e.g. from Humanitec or Syntasso. Looking at a platform from outside-in is simpler than building one yourself, but it still requires careful consideration.

Platform or No Platform?

As indicated in earlier chapters, the term “platform” has become fuzzy. So, the first question one should ask is why a platform is the right choice as opposed to a tool, a SaaS service, a framework, or perhaps [just a better approach](#). A common case for platform selection are cloud platforms, which appear to answer this question quite clearly (except for the fact that some vendors include the “p-word” in their name whereas others shun it altogether).

Buy vs. Build

You’re unlikely to want to build your own cloud platform, but for in-house platforms that’s not a foregone conclusion. Here you have several choices:

1. Buy (or “lease” in a SaaS model)
2. Build on top of a builder kit
3. Build from scratch

Selecting a Platform

I have summarized what I consider to be 9 proven poor ways to select a cloud provider in a [blog post](#)¹. Platforms have [very different dynamics from traditional IT services](#), so taking a static snapshot and running a scorecard based on the old way of working is likely to not be effective. The book *Cloud Strategy* discusses who cloud is different in much detail.

Michele has a great way of describing the common mistake of performing feature-comparisons:



I told a customer that he was selecting the platform as if they were choosing a pizzeria by looking at the menu outside, picking the one with a wider selection. They'd have no idea of thickness and texture of the crust, whether it's an electric, gas, or wood-fired oven, and whether the sauce uses fresh San Marzano tomatoes or paste from the can.

Assessing the scope of a platform is useful, but not all features translate into an actual benefit, or as *Cloud Strategy* put it:



A better knife doesn't make you a better cook.

Leveraging a Platform

Just selecting something labeled “platform” (even if it lives up to the name) doesn't guarantee that you'll be building on a platform or [standing on the shoulders of giants](#) so to speak. If you procure a powerful cloud platform, but then you manually provision virtual machines and don't use much else, I'd be hard pressed to tell you that you're a proud platform user.



Just selecting a platform doesn't guarantee that you'll be building on a platform.

¹<https://architectelevators.com/cloud/selecting-cloud-provider/>

Instead, you need to adjust your processes to take advantage of the platform. Manual approval before provisioning a server won't get you any platform benefit - it'll be like having a [person walking in front of your car waving a flag](#)²—also not a great way to speed up.

Respecting Platform Opinions

[Platforms have opinions](#), so when choosing to onboard on a platform, you have to be aware of them and likely accept them. A mismatch at the architectural or organizational levels may reduce the value you can reap from your platform.



When selecting an insurance business platform to manage claims, an organization chose a solution based on its favorable position in the respective Magic Quadrant. This platform came with a strong opinion on the entire stack, but also allowed extensions to be written in a proprietary language. Over time, the level of customization outgrew the use of out-of-the-box business processes. During the next platform upgrade the organization paid the price for adopting a platform but not its opinions.

A platform's opinions may cover not only functionality, but also the technical layers and technical processes, which may not fit the organization's established practices. Assessing the impact of a platform comprehensively can reduce the risk that the platform becomes an island in the IT sea, detached from the evolution of the IT foundation and other in-house platforms.

Not matching platform opinion against own needs and context can lead to the platform sinking under the weight of the workload. When this happens, there is little that the platform can do to increase buoyancy. Organizations are left with a classic dilemma: customize the platform even further or define a path to exit.

Not all products that are pitched as being extensible qualify as a platform. To enable development teams, the platform has to provide a coherent domain model and efficient APIs, combined with modern tooling, including version control and unit testing. Too many commercial products, often portrayed as business platforms, include extension mechanisms that resemble “escape hatches” as a last resort. Building an entire business on a set of escape hatches is as bad an idea as it sounds.

²https://en.wikipedia.org/wiki/Locomotive_Acts

Leaving a Platform

Key points:

- Consider an exit strategy, but don't fall into paralysis. Calculate the actual liability in terms of likelihood and cost.
- Aim for "accepted lock-in" where the benefit outweighs the liability. Think marriage or iPhone.
- Placing a wrapper seems tempting but [beware the Grim Wrapper](#)
- The cost of departure is not just a function of how you use the platform but also of your organization's maturity and velocity

Part IV: Designing Platforms

With the widespread success and potential of platforms, it's tempting to want to build one inside your organization, perhaps to streamline application delivery or as the foundation of your organization's business. That's generally a sound idea, but, as so often, there's a lot more behind other companies' platform success than might appear from the outside.

Designing and building a successful platform requires more consideration and engineering effort than many teams anticipate. Some organizations spent considerable time and effort to build elaborate platforms that, once launched, didn't find much adoption. This part highlights critical design decisions that guide the shape of an in-house platform. As platform design is a complex topic, the chapters suggest real-life metaphors to highlight the decision trade-offs and nuances. Those mental models can also be helpful when assessing third-party platforms.

- Architecture is often defined by quality attributes and it's [no different for platforms](#).
- A good platform is more than the sum of its part, so choose carefully whether you are designing a [Fruit Salad or Fruit Basket](#).
- Platforms are generally seen as horizontal elements, but they often [sit on vertical pillars](#).
- In-house platforms that build on top of external base platforms need to decide [whether to float or sink](#).
- Platforms can make great abstraction layers, but it's all too easy to fall victim to the [Grim Wrapper](#).
- Platforms reduce cognitive load by hiding complexity. But hiding too much can lead to [dangerous illusions](#).
- Even the most awesome abstraction crumbles when failure rears its head and reminds you that [it doesn't respect abstraction](#).

14. The 7 “C”s of Platform Quality

Platforms might not be forever, but they do have more C’s than diamonds

Conducting an architectural assessment of a platform, whether built in-house or by a third party, is no easy task. Good platforms’ [magical properties](#) mean that building one is surely not a paint-by-numbers exercise. Architects are generally less interested in specific features (which are bound to evolve anyhow) but in the critical design decisions and trade-offs that those imply.

Diamonds are famously evaluated by the “Four Cs” (carat, clarity, color, cut), so let’s see whether an equivalent architectural model can tell us how precious a platform is.

Architecture Decisions and Trade-Offs

A key goal of this book is to present models that can guide the development of your platform or the evaluation of existing ones. Given the diversity of platforms, finding a universal framework that applies to all platforms might be a tall order. However, we can copy a good idea from diamonds’ concept of the 4 C’s: instead of aiming for a single dimension that tries to line up all platforms from “good” to “bad”, we can break down platform qualities into multiple dimensions to highlight the trade-offs that you are likely to make.

As nice as it would be to have your diamonds be colorless, flawless, ideally cut, and of many carats, it’ll be guaranteed to explode the fifth dimension: cost. Instead, you might insist on an ideal cut and a near-pure color to generate that magical sparkle, but are willing to tolerate small inclusions. The framework has given you the vocabulary to easily express those trade-offs in the 4 (or 5)-dimensional space of diamonds.

We can aim for the same for platforms, and even stick with the same initial letter, defining the 7 “C”s of platforms:

Completeness

Does the platform offer a complete experience, i.e., does it provide self-service for all elements? Are helpful debugging tools and automation included? Is the documentation well-written? Is training and support available, perhaps via a user community or consulting services?

Cohesion

Does the platform present a meaningful whole, as opposed to a loose collection of pieces? In other words, is it more like a [fruit salad](#) than a fruit basket?

Closure

Does the platform contain a well-rounded set of services or are some pieces unexpectedly missing, reducing the user experience to a sadistic game of [Takeshi’s Castle’s Skipping Stones](#)¹?

Consistency

After dealing with one part of the platform, can a user apply what they learned towards other parts? For example, are shared functions like security or error handling consistent across services?

Commensurate Value

When using just a portion of the platform, do users get [proportionate value](#) from it, or will they use most of the platform before they derive measurable benefit?

Connectedness

How well is the platform connected (or connectable) to other systems? Platforms won’t live in isolation, so integration with single-sign on and monitoring systems are often essential for a platform to co-exist in an enterprise environment.

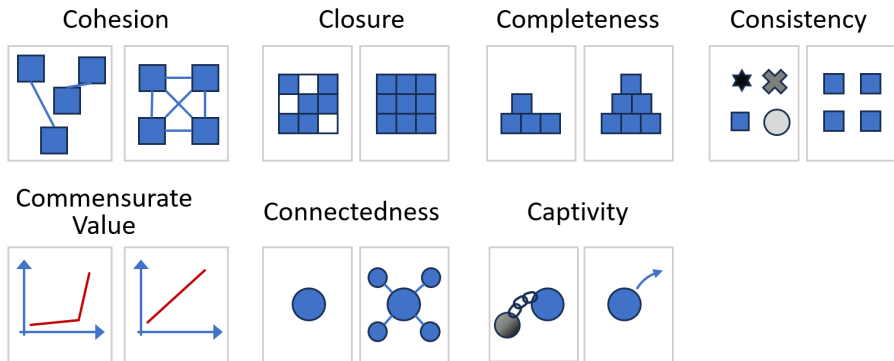
Captivity

How easy is it to move out from the platform in case you’d need to?

Delineating “C”s

Some of the C’s might appear similar but denote important nuances in platform design.

¹<https://architectelevators.com/strategy/takeshi-castle/>



The 7 “C”s of Platform Quality

For example, completeness relates to the scope of a platform, i.e. the functional surface area that it covers. A more complete platform may include monitoring or CI/CD pipelines. In contrast, closure describes how well-rounded the platform is within its scope. For example, a platform that includes a CI/CD build pipeline but doesn’t include (or integrate with) an artifact repository, would have poor closure as build pipelines typically store build artifacts in such a repository. Cohesion describes how well integrated the platform elements are. For example, a platform that includes a build pipeline and an artifact repository, but requires different user accounts for each, exhibits low cohesion.

Car Metaphors for the Win

Given that we [called on the automotive industry](#) to explain the notion of platforms, perhaps we can continue that thread to elaborate on the seven “C”s of successful platforms. Most cars will exhibit a high level of cohesion. An engine that’s not connected to the transmission or the wheels is not going to move anything. [Russ Ackoff](#)², a luminary in system design, once stated this very clearly: “the motor can’t even carry itself from one place to another”.

A car that’s lacking closure might have only 3 wheels—not a great setup unless you’re angling for a Piaggio Ape. Equally bad, a car lacking completeness might not have brakes. Consistency can play into many aspects, e.g. the pedals should be similar and close to each other. Connectedness could be compared to a dealer network where you can go and have your car serviced. Commensurate value

²<https://youtu.be/OqEeIG8aPPk?t=267>

for cars tends to be low—a car with 3 out of 4 wheels isn’t 75% complete, rather it’s 0% usable. Last, captivity for cars is generally quite low - you just need someone to buy the old one from you.

One C to Rule Them All

Luckily, the seven “C”s don’t have quite as much tension between each other as they do for diamonds. However, architects know that life is full of trade-offs, so some “C”s can imply some trade-offs. For example, in a [later chapter](#) we’ll describe how teams may have to balance cohesion against completeness.

The search for the perfect platform is also subject to [Gregor’s Law](#)³ and might therefore lead to an unwelcome “C”: Complexity. Complexity is the enemy of good platforms as their original goal is to reduce cognitive load. Also, a complex platform implementation will slow teams down as the platform needs to evolve over time. Unlike cost, there is no bank where you can take a complexity loan.

Talking in “C”s

As with most models in this book, the 7 “C”s don’t tell you what’s right or what exactly you should be doing. Instead, it lays out the *decision space*, telling you what decisions can be influenced. It also provides a simple and evocative vocabulary, so you can easily express important trade-offs like “we won’t compromise cohesion or closure for sake of completeness”.

³<https://architectlevator.com/gregors-law/>

15. Fruit Salad or Fruit Basket?

Good platforms are more than a collection of services.

Most platforms comprise a collection of individual pieces, such as components or services. However, the value of the platform doesn't just derive from its scope, i.e., how many components it contains, but also on how these pieces come together to form a meaningful whole. Given my knack for applying real-life metaphors to IT architecture, we find design guidance in a rather unexpected location: the produce section of a supermarket.

Summing up the parts

Platforms combine elements into a meaningful whole. The elements can be purpose-built components or, as is likely for in-house platforms, existing elements from a broader platform like a cloud platform or a data platform. For example, a modern application delivery platform might include a CI/CD build and deployment pipeline, a serverless or container runtime, monitoring and observability services, and a service mesh. It's therefore tempting to describe an internal platform by the list of service or projects that it comprises, in this example perhaps Bamboo, Jenkins, Nexus, Spinnaker, Kubernetes, Prometheus, Istio, and so on.

However, equating a solution to the list of its components would be missing the core purpose of architecture. I have stated before:

Architects are like chefs, not shopping for groceries. Good ingredients help, but a great meal comes from how they're put together.

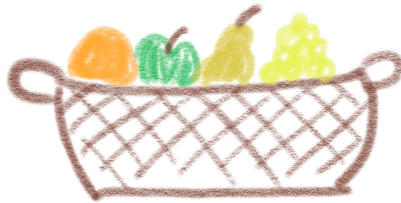
Simply listing ingredients, whether it's for dinner or for building a platform, is rather non-descript. There are many dishes you could make from flour, cheese and tomatoes, ranging from pizza to lasagna to a grilled cheese sandwich. The

same holds true for IT platforms. To make matters worse, many open source project names (and cloud provide service names) can confuse even the most well-meaning audience.

Thinking about a platform as a bill of materials also ignores the primary source of value for building a platform: if the platform is simply a collection of readily available tools, why wouldn't application teams just use their own Bamboo and Jenkins instances instead of adopting your platform? Hosted services like [Gitlab CI/CD](https://docs.gitlab.com/ee/ci/)¹ make this approach even more appealing because no installation is required and teams might even feel that using individual services gives them more freedom compared to your platform, which might be more prescriptive.

Your platform therefore must equate to more than an installation script or a catalog of already existing services. And another metaphor helps you express this design aspect vividly.

Fruit Baskets



A poorly drawn fruit basket

Platforms that are compilations of individual pieces can be compared to fruit baskets: they collate several items into a convenient collection. However, in the end all the customer gets is fruit. A fruit basket makes for a nice decoration, but it's largely a convenience. People aren't much worse off if they just buy the fruit itself, and may actually be better off because the basket may lack their favorite fruit or vice versa include fruit that they dislike. That pattern is well-known from discounted bundle sets, which invariably include items that you wouldn't buy individually (luckily the lid of the yet-to-be-used 6-liter stock pot from my cooking set fits one of my saucepans).

¹<https://docs.gitlab.com/ee/ci/>

Because purchasing bulk fruit at the supermarket is a vital alternative, fruit basket can't command much of a margin over the ingredients, aside from affording a nicer presentations for special occasions. [Japanese fruit baskets](#)² have taken that particular aspect to whole new levels of sophistication (and pricing), but it'll prove difficult to replicate with IT services.

Serving Fruit Salad



Even a poorly drawn fruit salad opens up new opportunities

Platform designers should therefore aim beyond fruit baskets. There's a much more platform-esque way of consuming fruit: the fruit salad. Just like fruit baskets, fruit salads are collections of fruit, however the fruit is cut-up and packaged into a ready-to-eat meal. What might appear like a minor convenience allows the fruit salad to deliver much more value than just the sum of its parts:

Support use cases that fruit baskets or fruit cannot

They're easy to carry and perfect for a picnic, for eating on the go, or for a lunch at your desk.

Offer an enhanced user experience

They balance texture and sweetness independent from the size of fruits. In contrast, if you buy one pear and one watermelon, you'll struggle to achieve much of a balance. You could say, fruit salads are more cohesive - they create a balanced whole.

Scale down

They're available in small portions, regardless of the fruit size. This would not work for a fruit basket with water melons. You could say, fruit baskets use a

²<https://www.buzzfeed.com/kevintang/inside-japans-most-insanely-expensive-fruit-parlor>

fine-grained, usage-based pricing model that matches the customer's needs. It scales down, something that is often more difficult to do than scaling up.

Reduce toil

Customers don't need to peel or cut the fruit and the ingredients don't crush in the bag or luggage. This can be very handy, for example when boarding an airplane where knives aren't allowed.

So, although a fruit salad might appear as not much more than cut-up fruit, it's actually a new product with a different value proposition. That's why it also fetches a significantly higher margin: many fruit salads clock in at over 10 Euros per kilogram whereas apples and oranges, the main ingredients, go for 2.49 (pre-inflation).

IT Fruit Salads

Platform builders should be inspired by fruit salads and not just define their platform by the tools and products it comprises.



When someone tells you that they build a GitLab / Spinnaker or Bitbucket / Bamboo / Xebia platform, that's a fruit basket. All the customer gets are some fruits and increasingly those fruits can also be had directly from the vendor.

To make your platform become more than just the sum of the pieces, the pieces need to be well-integrated or automated so that it becomes easier for your customers to use the whole platform as compared to individual pieces. In the simplest case sane default settings can get people going more quickly because they can avoid struggles with each individual product configuration. Or the individual products can be integrated in a meaningful way. On the other extreme, be sure to stay clear of the [Grim Wrapper](#) that tries to make the fruit salad something that it can't be.

Having Opinions

Naturally, a fruit salad-style platform will be more [opinionated](#), meaning it doesn't intend to be all things to all people. Customers will be more likely to

accept the constraints that you may impose if they benefit from a smooth and consistent integration. In comparison, opinionated fruit baskets are a tougher sell as the benefit won't outweigh the perceived lack of choice. A beautiful wrapping might get some customers' attention, but that initial attraction will wear off rather quickly.

Making Fruit Salad from Fruit Baskets

When I shared the fruit basket metaphor with a friend of mine who's running a platform tool company, he quickly concluded:



Cloud platforms are more like fruit baskets. You can refine them into in-house fruit salads.

Cloud platforms exhibit a good degree of cohesiveness, for example through unified provisioning APIs, consoles, monitoring, or security. But a cloud vendor has to build for the whole world to make the economies of scale work. That means, it's harder for the provider to offer opinionated fruit salads just for you. That's where in-house platform shine as they can make more assumptions and can be more opinionated.

Making fruit salad (in-house platforms) from fruit baskets (cloud platforms) is a common enough situation that some cloud platforms come with a knife and chopping board, so to speak. For example, AWS offers Proton. Behind the elaborate description as a "deployment workflow tool for modern applications" sits a platform aka fruit salad builder service.

Growing Salads

In the case of in-house IT fruit salads, you face the unusual situation that new fruits come along fairly regularly in the form of new services released by the base platform providers. This poses a challenge as some customers will demand those fruits to be swiftly included in your salad. However, the more balanced your current salad is, i.e. the more cohesive and well-integrated the collection of services that it comprises, the more difficult it may be to integrate new services. This is another classic architecture trade-off of optimizing for today versus optimizing for the future.

Maintaining a Margin

Being more than the sum of the parts is particularly important in a context where your platform lives on top of another platform, such as a cloud platform. Cloud service providers and other vendors aren't standing still and continuously integrate new tools into the base platform, which places pressure onto the "fruit basket"-style platforms and even those with a "fruit salad" model. It's therefore important to decide upfront whether your platform is intended to be [floating or sinking](#).

It's good to know that [Starbucks isn't the only place to learn about IT architecture and strategy](#)³, so next time you visit the supermarket, spend some extra time pondering the produce section. It isn't just healthy but also insightful!

³https://www.enterpriseintegrationpatterns.com/ramblings/18_starbucks.html

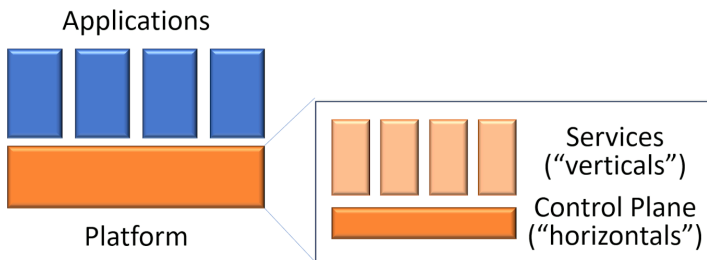
16. Cantilevered Platforms

Horizontal platforms often sit on vertical pillars

A platform is commonly portrayed as a horizontal element that sits underneath and spans across multiple applications. But zooming into the platform, it's often made up from vertical pieces, for example individual services that are [largely independent](#). Software architectures tend to be fractal, so underneath these vertical pieces lie more horizontal layers, which leads us to the question: what part of the platform should be horizontal, and what should be inside a stand-alone vertical? Time for some geometry.

Fractal Platform Architectures

The key mechanism of a platform is to harmonize across disparate systems by being the foundation underneath. That makes a platform true to its real-life counterpart, often shown in high-level architecture diagrams like the left side of the following illustration:



Horizontal platforms consist of verticals layered on more horizontals

At the same time, a platform typically offers a medley of individual pieces that come together. We already described that [platforms often sit on top of other platforms](#), giving us indication that underneath the horizontal platform we may

find other layers. Zooming into the platform itself also reveals another combination of horizontals and verticals: various “vertical” platform components (representing an individual cloud service or a third party product that’s part of an in-house platform) are tied together by means of “horizontal” services to become [more than just a fruit basket](#). Some of these platform-internal horizontals may be visible to the platform users, for example a cloud console (or an internal platform’s developer portal), whereas other horizontals like a cloud provider’s internal monitoring or resource management system would not be visible.

Do Control Planes really have Control?

The division of a system’s architecture into common services (“horizontals”) and individual components (“verticals”) that sit on top of it is about as old as the first code library that perhaps provided functionality to print results to a terminal. Many language frameworks provide unified logging, database access, identity management, exposing APIs and numerous other “common services”. So what’s different for platforms?

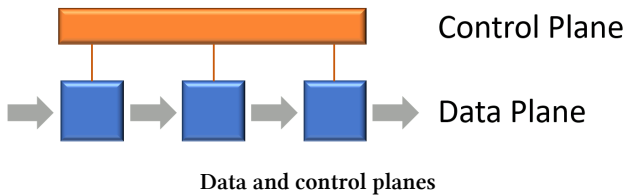
The classical architect maneuver to better understanding these nuances is to take a step back, aka “zoom out” to understand the reason behind horizontal layers:

- *Reuse*: the original motivation for extracting common elements from an application architecture is making common functionality available to many components. Most early language libraries did this for functionality like input, output, math, string operations etc. Reuse brings apparent benefits ranging from faster development, consistency, smaller code footprint, and increased quality thanks to avoiding coding errors for such common functions.
- *Governance*: As with [many in-house platforms](#), extracting common elements can also assure that all applications use the same, authorized version of a specific function or run atop the same run-time infrastructure. A common run-time layer like a cloud brings many additional benefits like uniform access management, billing, and cost management.
- *Operational Improvement*: In distributed systems, functions like logging are highly beneficial to be part of a common horizontal layer because they afford operational teams a unified view across many distributed

components. A *service mesh* gained popularity some 5 years as such a horizontal layer that manages data flow between run-time containers.

Although all these examples look the same in the abstract (a horizontal base layer with multiple verticals on top), the run-time architectures vary significantly. Common libraries are generally not visible in the deployment architecture since they become part of the application's deployment artifact during the build step. Run-time layers like a service mesh, in contrast, aim to be transparent at build time but have a pronounced run-time architecture.

Horizontal run-time layers that span distributed components are generally described as *control plane*, setting it apart from the so-called *data plane*:



The terminology derives from telecommunications and networking where data flows through a series of switches or routers. These routing elements have to be configured and managed via a separate control plane (occasionally an additional *management plane* is called out—more than that soon). In most network diagrams the control plane is shown above the data plane because it occupies a higher layer in the OSI network stack model.

Unlike libraries, which are build-time constructs that are compiled or linked into the application that uses them, platforms are generally composed of distributed services or applications. That's why most of them have a control plane, for example to provision and configure services, expose automation APIs, and aggregate billing data.



Some folks refer to the platform “verticals”, i.e. the actual services, as “data plane” to keep with the original nomenclature. This can be confusing as many of these services’ purpose isn’t to move data.

An event router or data streaming service would be accurately described as data plane, but a compute service like a virtual machine or a serverless function isn’t really analogous to a network router.

Shipping the Org Chart without Shipping the Org Chart

Control planes play an important role in abstracting away the complexities of a platform's implementation. In a [talk about Amazon S3](#)¹, distinguished engineer Andy Warfield describes:



“When you look at the highest-level block diagram of S3's technical design, [...] AWS tends to ship its org chart. This is a phrase that's often used in a disparaging way, but in this case it's absolutely fascinating.”

As a large-scale object storage service, S3 consists of a frontend fleet with a REST API, a namespace service, a storage fleet that's full of hard disks, and a fleet that handles background operations like replication. Behind these high-level building blocks sit hundreds of microservices that are developed and operated by independent teams.

“Shipping the org chart” is typically considered a negative because it exposes the user to the complexities of the organization's internal structure, requiring the user to perform the missing integration. In S3's example, which is prototypical for platforms, that integration is done in the control plane, which exposes a simple unified API. So, AWS ships its org chart to make the system distributed and scalable, but that complexity is hidden behind a simple API (S3 in fact stands for “Simple Storage Service”).

Platform Jenga: Stacking Planes

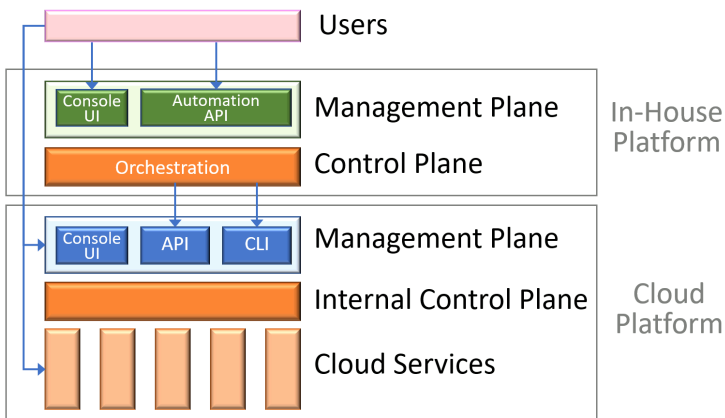
Zooming into a cloud platform's control plane, the fractal nature of our architectures becomes apparent. The control plane isn't a single element but is made up from many elements. In many cases, the control plane makes use of the same “vertical” services that it manages. For example, a serverless cloud runtime may use a NOSQL database to store configuration data, an object store to lookup deployment artifacts, and a queue service to handle event polling. That's what [AWS Lambda does](#)², leveraging AWS services like Amazon DynamoDB, S3, and SQS (Simple Queuing Service) to implement Lambda's control plane.

¹<https://www.allthingsdistributed.com/2023/07/building-and-operating-a-pretty-big-storage-system.html>

²<https://serverlessland.com/explore/qcon-2023>

As indicated before, some aspects of the control layer are visible to clients, such as automation APIs, command-line interfaces (CLIs), cloud consoles, or monitoring tools. Depending on the source, this layer is referred to as *management plane*, sitting above the control plane (other, more network-centric sources place the management plane between the data plane and the control plane).

Most in-house portals will interact with the cloud base platform via this management plane, typically through APIs or automation languages:



The Matryoshka Architecture of Platforms

But the story doesn't end there. The in-house platform also includes a control plane, which typically orchestrates task such as account setup and cloud service provisioning, for example as part a reference architecture. Not shown in the picture, but just as in the AWS Lambda example, real-life in-house platform control planes likely interact with the base platform management plane for two purposes: to provision services on behalf of the users but also to manage resources for its own usage, for example a database to store user or configuration data.

On top of all this sit the users, i.e. the developers. They access the in-house platform via a developer portal (and sometimes via an API). As long as you stay clear of the [Grim Wrapper](#), they'll also interact directly with cloud services, for example to configure them (management plane) or to store data in them (data plane).

Nesting UI Layers

One question that the previous diagram leaves intentionally unanswered is whether users interact directly with the base platform UI console in addition to the in-house platform UI, commonly referred to as a *developer portal*. In many cases, the answer is “yes”, divided along the lifecycle: users will use the in-house portal to create accounts or provision base resources and then use the cloud console to configure those resources.

This setup avoids the *Grim Wrapper* issues for the user interface, but it also makes for an inconsistent user experience, especially if users have to use separate identities for each UI layer. To unify the experience, developer portal builders like [Backstage](https://backstage.io)³ have gained popularity.

An architect pondering over the stack of layers should wonder whether we need all these layers:



Why aren't cloud consoles extensible to allow platform teams to build in-house portals directly on them?

If the cloud management plane calls public cloud APIs, that management layer could be open source and extensible to be customized as a developer portal, essentially collapsing the two layers. Thinking in platform layers raises some interesting questions.

IT Service Management (ITSM) tools, originally considered the [antonym of platforms](#), also increasingly partake in creating portals for developers to request and manage resources, creating noticeable overlap in capability. It's curious how the cloud's native ability for self-service has spawned a whole new genre of self-service portals.



Developer portals have triggered a special version of one-upmanship: each tool or vendor wants to sit on top of the others.

Most cloud automation tools support plug-ins for other services. Meanwhile, ITSM tools feel they're naturally positioned on top as they're “cloud neutral”. Developer portals like Backstage seem custom-made for this purpose and are

³<https://backstage.io>

easily extensible thanks to being open source. Folks in the Kubernetes ecosystem will consider custom resources as the common management layer.

If anything, architects have an abundance of choice. There isn't an obvious "right" answer, but visualizing the layering and asking direct questions like 'is our console an ITSM tool?' can help bring clarity.

Nesting API Layers

Using a UI to perform cloud tasks is commonly derided as "click-ops" because it's error-prone and not repeatable. We should hence have a similar look at how automation languages and APIs stack up across in-house platforms and base platforms. We'll dive into this fascinating domain, which ranges from YAML templates to technical domain models, in [Part V](#) of this book.

Developer Portals as Micro-Frontends

Many consoles and developer portals such as Backstage use a Micro-Frontends (MFE) architectural style, which also plays off verticals and horizontals. [Cam Jackson](#)⁴ defines MFEs as:



Micro-Frontends are "An architectural style where independently deliverable frontend applications are composed into a greater whole"

We can see the appeal for cloud consoles and developer portals alike. A cloud console contains configuration for often hundreds of services. Likewise, an in-house portal may be split into account creation, build pipeline setup, run-time management, and monitoring dashboards. These elements may be maintained by independent teams but should still provide a consistent user experience. MFEs decompose the user interface into individual parts, allowing platform teams to build these elements independently and combine them into a single user interface.

Interestingly, MFEs reveal another horizontal-and-vertical structure. UI elements are independent "verticals" held together by "horizontal" like shared

⁴<https://martinfowler.com/articles/micro-frontends.html>

headers and footers, common UI elements, or identity management. MFE literature⁵ distinguishes a “horizontal” and a “vertical” approach, although both include horizontal and vertical elements:

- The “vertical” style prefers largely independent teams (and perhaps run-times) that each maintain a page.
- The “horizontal” approach foresees a single page combining multiple UI elements, which can still be developed independently but need to follow a common UI framework.

Developer portal tools like Backstage follow a horizontal approach with a plug-in architecture that allows developers to build additional UI components. Many cloud consoles follow the vertical model to support numerous service teams and their respective user interfaces.

Rotating 90 Degrees: Making Horizontals

Not all orientations are set in stone. A common maneuver is to increase the amount of horizontal elements, for example to increase reuse or improve the user experience. You can observe this trend in some cloud consoles that aim to unify the user experience and provide new features like customizable home pages.

Most anyone who has tried to unify elements after the fact has faced the same challenge, though:



Everyone will agree in principle that more common elements make sense. But everyone is too busy with their project to work on it now.

Unfortunately, the “doesn’t it make sense?” argument won’t overcome this barrier as people already agree. It’s therefore a better move to not rely on good intentions, but to look for a set of valuable features that require a stronger “horizontal”. The above-mentioned custom home pages are such an example: they have a clear user benefit as users can get a custom overview of their most relevant services. They also require a “horizontal” MFE setup, so they are a useful driver for re-architecting consoles.

⁵Mezzalana, *Building Micro-Frontends: Scaling Teams and Projects Empowering Developers*, O’Reilly 2021

17. Will Your Platform Float or Sink?

Most people want to swim until they realize their cost is sunk.

Most in-house platforms are [built on top of base platforms](#), typically cloud platforms offered by major cloud providers. Those base platforms continually evolve with cloud providers adding thousands of new features each year. While cloud users benefit from this base platform evolution, it requires internal platform builders to decide—and clearly communicate—what they plan to do in case the base platform replaces a feature that was built for the in-house platform. Another metaphor will help make the options very clear: do you want your platform to float or sink (or perhaps “submerge” for less intimidating language)?

Do Rising Tides Lift all Boats?

One platform metaphor that I am not particularly fond of is the notion of a [burning platform](#)¹. Although it makes for a sticky mental image to depict the consequences of not changing, it appears to originate from a worker risking his life to jump off a burning oil platform into the ocean. The environmental impact aside, oil platforms are notoriously dangerous places to work and accidents have claimed hundreds of lives. So, conjuring the image of jumping into the raging ocean to escape being burned alive might be overdoing it a bit to convey the “jump in the cold water” that organizational transformations often require.

Luckily, the cloud platforms that we build on top of don’t burn, and they especially don’t sink. They actually grow, both in scale and also in functionality, and may now include functionality that you had previously built as part of your in-house platform.

¹<https://hbr.org/2012/12/how-to-anticipate-a-burning-platform>

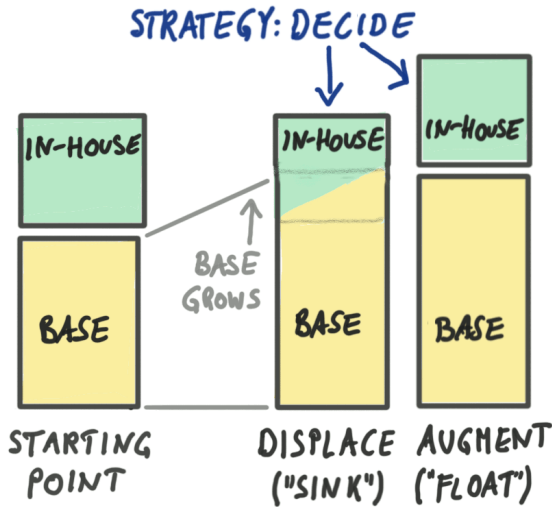


The software delivery platform we built on premises back in 2015 was largely replaced by a commercial cloud platform in 2021. This doesn't imply that our investment was poor, quite the opposite: it meant that we were on the same track as major cloud platforms, just a few years ahead of broad adoption.

There's a suitable metaphor for base platform evolution: rising ocean levels. It's also a concerning trend, but hopefully more suitable for the decisions that we need to make when building in-house platforms. And it turns out that those rising levels do not lift all platform boats.

Sinking and Floating

When water levels rise in form of base platforms adding functionality, your platform that was previously built above the water line now has two options:



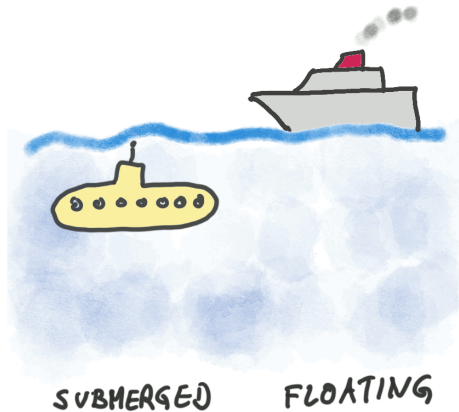
Two options for platform evolution: displace or augment

Your platform can rise to the occasion and "float" on top by augmenting its functionality. If you keep your platform as is, its functionality will be displaced by the new platform features. Augmenting your platform, which is often seen as more desirable, depends on two mechanisms:

- You retire functionality that you had built but that is now “underwater”, meaning the base platform now provides it. Doing so reduces your maintenance burden and cost as the cloud provider will invariably have better economics than your platform team.
- You use the freed up capacity to add new capabilities to your platform that aren’t yet broadly available, always staying one step ahead.

Retiring functionality will cause migration effort for your users. Keeping your platform as is will save that effort but result in a platform that’s duplicating features that are now readily available in commercial cloud platforms. Maintaining those features is not economical and may cut your users off future base platform evolution.

Returning to our metaphor, augmenting your platform allows it to float above the rising water levels. Perhaps stretching the metaphor a little, shedding the ballast of redundant functionality gives your platform the buoyancy to float up. Keeping the platform static causes it to submerge or “sink” in face of rising ocean / functionality levels.



Is your platform sinking or floating?

Giving these decision options evocative names brings clarity and allows teams to express a critical aspect of their platform strategy. For example, is your platform designed to “sink” (or “submerge”) or “float”? Sticky names help people remember the decisions that they made.

Deciding is Easy—Until You Get There

When presenting these options to project sponsors, the notion of a sinking platform rarely finds much support. The provocative language might provide an initial bias (who wants to build stuff that sinks?), but in any case sponsors will quickly realize that duplicating commercially available functionality doesn't make a ton of sense. The initial motivation for investing in an in-house platform is to add missing functionality to the existing platforms, so why wouldn't you want to keep it that way?

Ironically, though, when the time comes to scrap (or “offload”) some of that functionality that's gone underwater, quite a few decision makers get cold (or perhaps wet) feet. Internal cost accounting favors long recovery periods, which means that systems should remain running as long as possible to recover the original investment (this habit explains why IT landscapes often resemble computer history museums). Why would we want to throw away all this awesome capability we built? Add the inevitable migration cost (more on that below) into the mix and the temptation to leave the platform as is or to “address it later” grows strong.



When my on-premises platform went underwater (some years after I had left the organization), folks expected me to be sad. Instead, I was quite happy that major cloud providers followed the same trajectory as our platform.

Such organizations fall into the classic [sunk cost fallacy](#)²: decision makers have a bias to look at past investments that can't be recovered when making decisions about the future (Wikipedia's Simple English version cites watching 6 episodes of *Battlestar Galactica* as an example of sunk cost that might cause you to continue watching. This might be worse than funding an in-house platform that's gone underwater). Organizations that have a static view of the world and don't consider the [cost of delay](#) are particularly prone to making this mistake.

This is where the mental model combined with the evocative vocabulary can help increase decision discipline. It allows you to remind everyone that the platform you built is now “under water” and the only way to get it to float again is by shedding ballast. Although you can't know the future (still, assuming that cloud platforms will grow in features is a fairly safe bet), stating your evolution

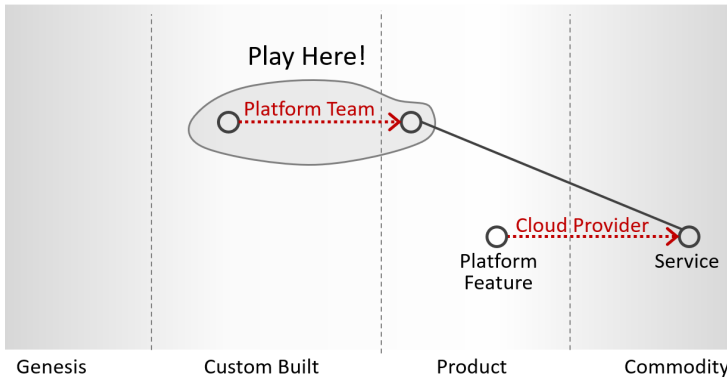
²https://en.wikipedia.org/wiki/Sunk_cost#Fallacy_effect

strategy lays out clear options how your platform will evolve in the face of changes to your environment. The fact that both options are viable and involve trade-offs is a signal that it helps you make a meaningful decision that's best for your circumstances.

Underwater Maps

[Wardley Maps](#) can provide another, perhaps more scientific-looking model to understand the implications of evolving your platform along the base platform or keeping it static. Base platforms including additional features is a classic example of the ILC maneuver of sensing innovation and commoditizing into a platform. In a way, your platform might have given the cloud provider a signal to commoditize this particular feature.

Holding on to a product that has been commoditized is guaranteed to have poor economics, resembling you building your own power station. Instead, you should be shifting your attention to components that are higher up the value chain and can benefit from new commoditization.



When your platform gets commoditized, shift the playing field!

For example, most cloud providers now offer CI/CD pipelines as a service, so it makes little sense to build such capability as an in-house product in form of a platform. Instead, you should build on top of the new commodity, focusing for example on advanced team metrics or code inspection.

In many cases, commoditization won't shift your entire service. For example, your platform's CI/CD service might include features or subcomponents that

the cloud service does not provide. The key maneuver in this case is componentization. Instead of considering a CI/CD pipeline as an atomic component, you can break it down into several subcomponents, some of which might be commoditized by the cloud provider.

Rising Levels

Cloud platforms are excellent examples of base platforms that keep growing. Large business applications like Salesforce or SAP follow a similar trajectory. The base platform growth can be roughly divided into two categories:

- 1) Enhancements to existing features
- 2) New features or the platform entering new segments

Cloud platforms enhance existing features all the time but also make noticeable forays into new segments. Build and deployment tools that provide continuous integration (CI) and continuous delivery (CD) of software used to be under-represented in cloud platforms that primarily focused on application run-times, storage and networking.



All major cloud providers plus major source code management tools like GitHub or GitLab now feature a suite of software delivery tools, which were previously often developed in house. Full-featured on-demand development environments like [AWS CodeCatalyst](https://aws.amazon.com/codecatalyst/)³, [Azure DevOps](https://azure.microsoft.com/en-us/products/devops)⁴, or [GCP Cloud Workstations](https://cloud.google.com/workstations)⁵ continue this trend.

In other cases, the platform grows by offering open-source software as fully managed services, alleviating platform teams from having to operate such tools without requiring them to change interfaces to a proprietary product. A classical example is Grafana, a popular tool to visualize operational data, which is available as a [managed service on AWS](https://aws.amazon.com/codecatalyst/)⁶ and [Azure](https://azure.microsoft.com/en-us/products/managed-grafana/)⁷ and the [GCP Marketplace](https://console.cloud.google.com/marketplace/details/grafana-public/grafana-cloud)⁸.

Many in-house platforms provide pre-configured service constellations and restrict the use of some cloud services or geographic regions due to data residency

³<https://aws.amazon.com/codecatalyst/>

⁴<https://azure.microsoft.com/en-us/products/devops>

⁵<https://cloud.google.com/workstations>

⁶<https://aws.amazon.com/grafana/>

⁷<https://azure.microsoft.com/en-us/products/managed-grafana/>

⁸<https://console.cloud.google.com/marketplace/details/grafana-public/grafana-cloud>

considerations. Such features are being incorporated directly into the cloud platforms, beginning with managed service catalogs and now evolving towards infrastructure template tools like [AWS Proton](https://aws.amazon.com/proton/)⁹.

You Pay in Opportunity Cost

When faced with a sinking platform, a platform team's initial reaction is often that it's easier to keep running the services that they have already built. After all, internal users are already familiar with those services and operating them doesn't take that many resources, meaning taking on a little water isn't so bad.

As illustrated in *Cloud Strategy*, the major mistake in this reasoning is to measure the actual cost of the internal resources as opposed to the opportunity cost:



The cost of maintaining a submerged platform is measured in opportunity cost, i.e., the value that those teams could have generated otherwise. It's often a high multiple of the actual cost.

Part of that opportunity is building new platform features, which could provide high value to the organization. After all, platforms are accelerators and multipliers. If you're busy maintaining aspects that are widely available in cloud platforms, you'll miss out on that opportunity.

Be careful of "we just maintain this on the side" fallacy. Submerged platforms, like any other running system, takes not just engineer hours, but also mindshare. Engineer context switches can often cost hours in productivity, even for a 20-minute task. Submerged platforms also need to adapt upon base platform, support customers, update documentation, etc.

Breaking Changes

When the base platform submerges your existing functionality, odds are that the replacement won't be a 100% identical copy of what you had built. Those differences imply migration effort and potential impact on your internal users. Trying to avoid this type of change is another common reason that internal

⁹<https://aws.amazon.com/proton/>

platforms sink below the water level set by the base platform. Resisting change and clinging to the status quo is so common that *The Software Architect Elevator* cites the example of someone rejecting a new car because the door doesn't make the same squeaking sound as the old one. In their view, it doesn't match the requirement.



When a platform I had built was supposed to be handed over to operations, that team demanded that the platform could not be upgraded until all users agree. This would have killed innovation and ultimately the platform. So, we operated the platform ourselves.

Replacing a platform component with an implementation of the base platform is also a good test for the platform's modularity. Platforms should exhibit a [high degree of cohesion](#) but still have a functional composition that allows replacing some functions without changes ripping through the entire platform. Balancing these needs is one of the key architectural tasks of designing successful platforms.

It's Never a Static World

Base platforms are a rare case where rising levels are a good thing because as a user you benefit from additional features. However, you have to be prepared for this continued evolution and set the appropriate expectations. Or carry a lot of oxygen.

18. Beware the Grim Wrapper!

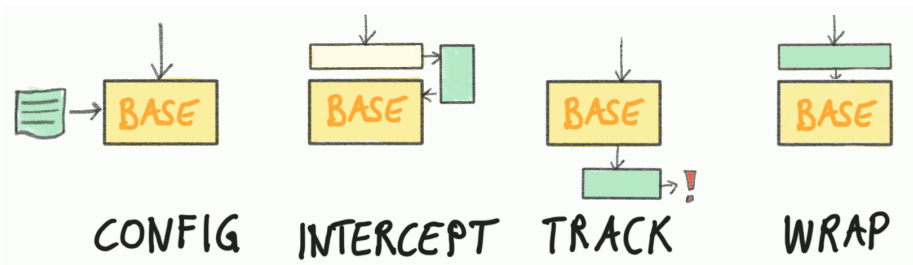
Good intentions don't guarantee success.

When building a platform over existing components, platform designers face a tremendous temptation to wrap existing APIs behind a brand-new, unified, perhaps vendor-neutral API. The perceived benefit is a simpler user experience that reduces the cognitive load for developers: instead of dealing with all the underlying bits and pieces, users “only” have to learn the organization’s purpose-made API. What could possibly go wrong?

Tailoring Existing Components

In-house platforms are generally composed of existing elements, such as open source tools or public cloud services. Just collating existing pieces doesn’t make a very valuable platform—it’ll just become a [Basket of Fruit](#)). Therefore, successful platforms tend to tailor existing services to the specific needs of the organization and often apply constraints over the usage of those services to provide governance.

When customizing an existing software component to be used in an internal platform, teams can employ several techniques, as illustrated in the following diagram:



Choices for influencing base platform behavior

Configuration

Base platform component creators often anticipated changes to be made inside organizations, so you can often achieve your customization via configuration. These configurations might set meaningful defaults or they can restrict choices, e.g. by means of service catalogs or access control that influence the availability of base platform services or compute regions. The configuration can be managed through configuration files, user interfaces (consoles), or APIs. Configuration is the simplest route for the in-house platform development team but it doesn't shield users from the underlying complexity of the base platform.

Intercepting

Another way to change how platform users can interact with underlying components is by intercepting calls made to the base platform components. In these cases, the base platform exposes an API for developers, but before the calls are executed, additional validation code can be invoked. In-house platforms can insert themselves into the “hooks” provided by the base platform. Your interceptor code can set default parameters, restrict the range of input parameters, or reject the call altogether.

Although perhaps less glorious than the Mad Max Interceptor cars, it's a very effective way to control end user access to components that make up your platform. End users use the original interface of the underlying component, albeit with modified functionality. Platform designers need to be cautious to [not cause confusion](#)¹ when some functions that appear available in the public interface description are not available or exhibit modified functionality.

Tracking

An occasionally overlooked option is tracking, which allows users to interact directly with the original components without anything intercepting the calls. However, any calls are tracked, analyzed, and can cause alerts or corrective actions. Most public cloud platforms include tracking services, such as AWS CloudTrails or GCP Cloud Audit Logs.

Security architects might be nervous about unrestricted access to the underlying platform APIs, relying merely on the fact that if users perform an undesirable

¹<https://architectelevators.com/architecture/stacktrace-abstraction/>

action, they have to detect it and throw an alert. In reality, though, tracking is a rather powerful option because it doesn't have to look at each API call in isolation but has access to a log of actions.

Wrapping

The last approach replaces the original component interface with a new interface of your choosing. Doing so affords the platform designers the most flexibility and the most abstraction from the underlying components. This abstraction would also make it easier to replace a component with a different implementation without changing the published interface.

All approaches have their place. In-house platform teams often prefer wrapping because they get to choose its own abstraction or because the other approaches don't provide sufficient flexibility. As we know, architecture is the business of trade-offs, so there are some downsides to wrapping as well, and significant ones at that.

Wrapping Considered Harmful

Whenever we want to make a concern we have sound intellectual, we postfix it with “considered harmful”, playing off Edsger Dijkstra's seminal letter in the 1968 Communications of the ACM. So, let's just say I have some concerns with the wrapping approach. Or, more precisely, I have seen numerous teams run into dead ends from what originally seemed like a viable path.

When building a wrapper, be aware of the following pitfalls:

Keeping up with the rate of change

It's OK to wrap your legacy mainframe application that hasn't changed in 30 years (even if you wanted it to). However, when building an IT platform, chances are that the components you wrap are undergoing a rapid evolution, especially if they are open source projects or cloud platforms. This means you wrapper has to be constantly adjusted to avoid falling behind and thus giving platform users one more reason to go “upstream” to use the source product directly.

Although most components minimize breaking API changes, new features or capabilities also need to be wrapped. Be careful when considering whether

you can keep up with the rate of innovation of a major cloud provider or an extensive open source community. If you can't (which is more than likely), you are cutting your platform users off of the innovation stream. Even heavily supported projects like Kubernetes struggle to keep up to provide operators for new cloud services.

Simpler doesn't mean easier

One motivation for the wrapper that I have seen many times is to “make things simpler”, by not exposing the platform user to all the complexities and nuances of the underlying components. Although a noble pursuit, the reality often looks different. To the surprise of the platform developers, users have a hard time learning the “simpler” API. A smaller API into a complex subsystem doesn't necessarily reduce the interface's [cognitive load](#)² but could present a [dangerous illusion](#).

Layering new abstractions on top of existing ones can in fact increase the cognitive load, for example when error conditions occur from a component that the user didn't even know existed (see [Failure Doesn't Respect Abstraction](#)).

Unable to leverage external knowledge

You can likely hire (or contract) people who are familiar with third party products and their APIs. However, when you define your own abstraction over existing products, you have to teach every user from scratch. This will not only burden you with more documentation effort, it will also slow down the rate at which you can scale.

You might derive the lowest common denominator

Vendors often intentionally resist common abstractions. Quite a few services offered by cloud providers are similar in nature but all but a few are compatible across providers. Trying to fold a common abstraction over competitors' services is bound to be a losing proposition and you'll end up with a lowest common denominator at best.

²https://en.wikipedia.org/wiki/Cognitive_load

You need to operate your abstraction

If you want to define a new API for your users to call, you need to operate this API. This means that you have to operate an additional run-time component. If you are able to operate in the cloud, e.g. on a serverless platform, the impact might be small but in most cases it increases your operational effort and negatively affects your service level. Of course you also have to deal with upgrades, user identity, and all the other aspects that come with building your own layer.

19. Build Abstractions Not Illusions

Sometimes less is actually less

Who doesn't like a good magic show? Too many users expect internal platforms to be nothing short of magic with the rabbit jumping out of the hat to take all the complexities, learning curves, and potential mistakes of the underlying technology away. Platforms that aim to live up to these inflated expectations are bound to find out that magic shows aren't a good match for IT platforms.

Making things simpler, but not too simple

The technology available to developers today is nothing short of amazing: we can globally deploy and scale fine-grained, loosely coupled systems with the push of a button. However, designing and running such applications can also be complex. For one, developers have to deal with the inevitable aspects of distributed systems, such as asynchrony, out-of-order delivery, partial failures, and retries. They are also faced with often hundreds of base platform services plus constructs ranging from virtual private clouds to fine-grained access control panels.

It's only natural to want our platforms to [hide that complexity](#). Reducing complexity improves productivity, avoids mistakes, and reduces cognitive load. So, one would think that the more complexity we can hide, the better our platform will perform. Actually, no. We need to be careful that we provide platform users with useful abstractions, not dangerous illusions.

Abstraction: One more layer can't hurt, can it?

Many internal platforms aim to reduce developer productivity by reducing cognitive load. The key mechanism at work is abstraction, a "magical" layer

above all that complexity and inconsistency. The idea of layering things is so common in computer science that it's been called the [Fundamental Theorem of Software Engineering](#)¹ (attributed to David J. Wheeler and named as such by Andrew Koenig):

We can solve any problem by introducing an extra level of indirection.

Not being an actual theorem, it'll likely forgive us for broad brushing over indirection vs. abstraction (see [StackExchange](#)² for a healthy discourse). For an everyday example and to satisfy my penchant for car metaphors, we can consider the naming of an automobile and its interface consisting of pedals, steering wheel, and shifter to be a useful abstraction. To highlight that this isn't to be taken for granted, I have [publicly posited](#)³ that:



If software engineers had named the car, it'd be called “piston-crankshaft-gear-wheel-assembly”

Luckily, someone else named this awesome machine after its purpose, which is to move by itself (or at least without being pulled by horses), so it's an automobile.

Not all car-related naming is doing an equally good job at abstraction. When we want to move our car, we commonly depress the “gas pedal”. That term points to an implementation detail, easily highlighted by the fact that electric cars don't actually use gasoline (and better not have gas in their batteries). A more adequate term is “accelerator”, which provides a better abstraction. It also works equally well for internal combustion engines and electric cars because it's named after the purpose not the implementation.

As we'll see [in the next chapter](#), abstractions have a tendency to leak, especially in the face of failure, and the car is not an exception. The nice pedals and steering wheels are supplemented by a gallery of warning lights that can suddenly reveal all sorts of underlying complexity, ranging from failing alternators to aging lambda sensors and lack of oil pressure. Some of these lights can be safely ignored whereas others will cause thousands of dollars in repair cost unless you react instantly.

¹https://en.wikipedia.org/wiki/Fundamental_theorem_of_software_engineering

²<https://softwareengineering.stackexchange.com/questions/111756/what-is-the-difference-between-layer-of-abstraction-and-level-of-indirection>

³<https://twitter.com/ghohpe/status/1513362076990803969>

Composition isn't abstraction

Internal platforms often compose existing base platform services, ideally in [fruit salad fashion](#). Such composition can be very useful. However, it doesn't provide an abstraction—good abstractions are actually difficult to come by. This can be easily understood by the language that they use. An easy example are the solution libraries published by many cloud providers. As I discussed [in a blog post](#)⁴, these constructs are compositions, not abstractions. An easy test that reveals this fact is the language that they use. For example, unless you are an AWS serverless developer, a construct named Lambda-SQS-Lambda will make little sense to you. It's the cloud equivalent of piston-crankshaft-gear-wheel-assembly.

So, for something to provide a meaningful abstraction, it needs to provide a higher level vocabulary like that of an accelerator and brake, not one of air-fuel mixtures, fuel pressure and fuel injectors:



An abstraction provides a higher-level vocabulary that shields the user from the underlying complexity.

This insight implies that useful abstractions are often not a single component or construct but rather a cohesive vocabulary of related concepts (once again, the fruit salad is the suitable metaphor). When writing *Enterprise Integration Patterns* 20 years ago, we experienced first-hand how easy it is for implementation details to slip through. We renamed some patterns late in the game to remove implementation details from the names:



The integration pattern *Scatter-Gather* was initially called “Broadcast Aggregate”, named after its components, not the user's intent.

So, it can be useful to take a step back and try to flush all those implementation details from your brain and see if the name you've chosen still makes sense.

⁴<https://architectevator.com/cloud/abstractions-difficult/>

Illusions Illustrated

Now that we better understand why abstractions are both useful and difficult to find, it'll be tempting to abstract away as many of the underlying details as possible. However, too much of a good thing usually turns into a bad thing, and abstraction is no exception. Abstracting too much leads to dangerous illusions. Illusions happen when the abstractions hide too much, including relevant concepts.

Illusions are dangerous because they lead to initial developer euphoria - things look so much simpler - until reality bites and the illusion bubble bursts. They are one reason for [Hockey Stick](#) profiles that hit the users hard once they leave the happy path.

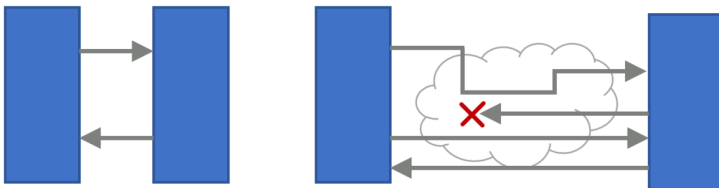


Abstractions that omit essential details become dangerous illusions.

A very common illusion in distributed system design is RPC - the Remote Procedure Call. This well-intended abstraction makes a call to a remote component look like a local method call. Technically that's entirely possible and users may enjoy their newfound simplicity—until they realize that a remote call is nothing like a local call: it's slow, doesn't have a call stack, suffers from partial failures due to lack of shared memory space, has to marshal and unmarshal data types across programming languages, just to name a few.



Remote Procedure Calls (RPC) are one of the most dangerous illusions.



The RPC illusion: model (left) and reality (right)

Domain models provide actual abstraction

Teams build platforms over existing base layers routinely struggle to define meaningful abstractions. Many of these layers are intended to provide independence from any particular cloud platform. Routinely, the platform engineers will “abstract” a provider-specific service like AWS SQS, Azure Service Bus queues, or Google Cloud PubSub into a message queue. However, there isn’t much abstraction that’s taking place, you turned one queue into another queue, perhaps [omitting some detailed settings](#).



Good technical abstractions come from really understanding your domain

I posited before that [finding good abstractions by working bottom-up is very difficult](#)⁵. Instead, good technical abstractions come from really understanding your domain. Instead of re-defining the meaning of “queue”, one customer defined abstractions in their platform that relate back to their business needs, e.g. compliance. So, instead of abstracting the cloud provider’s NOSQL database, they defined a construct called “Ledger Database”, which consists of a primary database and a second database that capture an append-only change log. This is a common concept in their industry (financial services), which is well-understood by their developers and therefore provides a meaningful abstraction.

What’s “Essential”? It depends and ChatGPT won’t tell you

The earlier definition of illusion includes a clever weasel word: “essential”. It’s in the league of “appropriate”, “adequate”, or “correct” all words that suggest that things become quite easy if you only knew the answer beforehand! Whenever I stumble on such words, I recall Joseph Bergin’s cheeky design pattern *Do the Right Thing*: all you need to do is apply this pattern and all will be well. Interestingly, one of the more common references to this (satirical) pattern is Linda Rising’s [IEEE column on The Power of Abstraction in Patterns](#)⁶.

⁵<https://architectevator.com/cloud/abstractions-difficult/>

⁶https://www.researchgate.net/publication/3249379_Understanding_the_Power_of_Abstraction_in_Patterns

There's good news and bad news here. The bad news first: there is no easy answer. And, no, ChatGPT also won't be able to tell you. The good part: that's why we have architects. Deciding what's essential is inherently a judgment call.



What's essential to avoid being an illusion can change over time: we used to largely ignore run-time efficiency and carbon footprint. Now they have become important aspects of operations management.

From the Architect Elevator we know that architects live in the first derivative, meaning that change drives many architecture decisions. This effect also applies to what's an essential part of an abstraction and what isn't: it will change over time. For example, we used to abstract away resource consumption and carbon footprint because processors kept getting faster, and the carbon footprint was considered small and difficult to measure. Now these elements are very much returning to the abstraction with serverless computing charging you by time and resource and most cloud providers including a carbon estimator tool.

Distributed system abstraction

I work in serverless integration, where we build highly scalable, asynchronous cloud services that are fully managed. On one hand they're magical because you don't need to deploy anything and just by configuring the service it'll scale to thousands of events per second. However, high-throughput, asynchronous systems must have specific run-time characteristics like throttling, backpressure, time-to-live, back-off, batching, limited retry counts etc. One frequent decision we have to make is how much of these distributed system aspect to expose to the user. If we expose too little, we create dangerous illusions. If we expose too much, we unnecessarily burden users.

I started to develop one heuristic that can help us:



If your API has two integers, one string, and 30 pages of documentation to explain what these do, you likely created an illusion.

The counter-argument is that exposing more details drives up complexity. Here I'd propose to follow the principle of essential and accidental complexity,

popularized by Fred Brook's paper [No Silver Bullet](https://en.wikipedia.org/wiki/No_Silver_Bullet)⁷. Concept like batching, backpressure, retry, and back-off are concept inherent in the domain of distributed systems. If you don't expose them in your API, they'll be exposed in your documentation or, worse yet, in your support teams. So, tackle them head on and use domain modeling techniques to minimize accidental complexity.

The happy path illusion

Another illusion that's easy to fall into is that nothing never goes wrong. Errors can make even the best abstraction go "poof". More on that in the next chapter.

⁷https://en.wikipedia.org/wiki/No_Silver_Bullet

20. Failure Doesn't Respect Abstraction

When was the last time you enjoyed a stack trace?

Abstraction is one of the most powerful tools we have for building and managing complex systems. One could hardly imagine getting by in today's world without strong abstractions. Just imagine the complexity that lurks behind the channel-flip button of a TV remote control or the start button of a car but is neatly abstracted away. Platforms rely on the same abstraction mechanism to hide implementation complexity, thereby reducing the cognitive load on the platform's users. After all, low friction of platform adoption is one of the defining features.

Alas, when something goes wrong down in the engine room, our amazing abstractions can quickly break apart, leaving the user either with a boatload of sudden complexity or a broken black box with no hope of remediation.

Flying on Empty

Air Transat Flight 236 was en route over the Atlantic from Toronto to Lisbon, Portugal, when the pilots noticed that their gauges were showing low oil temperature and high oil pressure on engine #2. Advised by the maintenance center to monitor the situation, they carried on without any issues. Half an hour later they were alerted of a fuel imbalance, which they corrected by transferring fuel between the tanks. Everything seemed normal for about 45 minutes when engine #2 flamed out due to lack of fuel. All twin-engine jets are designed to operate on a single engine, so things were OK until not long after, engine #1 also flamed out, rendering this amazing A340 aircraft into not much more than an expensive glider, 65 miles away from the closest air base. Thanks to amazing pilot skill (the pilot was said to be an avid hang glider pilot) and some luck, the

plane [landed on the Azores without any major injuries](#)¹ and ended up being nick-named “Azores Glider”.

The near-disaster struck because, unbeknownst to the crew, the indicated symptoms of low oil temperature, high oil pressure, and low fuel levels resulted from a common cause: a wrongly installed hose near engine #2 tore and leaked fuel. Modern jet engines use a fuel-oil heat exchanger (FOHE) to cool the engine oil and at the same time warm the kerosene fuel before entering the turbine. The leak increased the fuel flow through this heat exchanger, causing the engine oil to cool off more than usual, thereby increasing the oil pressure due to oil's higher viscosity at lower temperatures.

Not being aware of the root cause, the pilots pumped fuel from the “good” tank into a rapidly leaking tank and caused the plane to run out of fuel. Had the pilots known the inner workings of the FOHE and its effects, they might have correctly diagnosed the problem. However, the enormous complexity of the aircraft has largely been abstracted away from them and replaced by a set of dials, levers, and wheels. These abstractions made flying much safer and also eliminated the need for a dedicated flight engineer. However, in a failure scenario like the fuel leak, ignoring the underlying complexity can be dangerous.

Searching for Alpha Particles

Another great example of failures not wanting to respect abstractions comes from Google's early days, when the search index, Google's greatest asset, delivered wrong results. After an intense debugging session that yielded no result, two of Google's greatest minds, Jeff Dean and Sanjay Ghemawat, “went deeper”. By looking at data in binary format, they ultimately found that the bits stored in some memory chips had been corrupted, perhaps by errant alpha particles (whole story in the [New Yorker](#)² and also referenced in [Jeff Dean's presentation](#)³). The brilliant software engineers, who tried to debug before, relied on the abstractions of operating systems and programming languages and hence were unable to find the bug in the system. Only going past these abstractions allowed Jeff and Sanjay to solve the problem.

¹https://en.wikipedia.org/wiki/Air_Transat_Flight_236

²<https://www.newyorker.com/magazine/2018/12/10/the-friendship-that-made-google-huge>

³<https://static.googleusercontent.com/media/research.google.com/en/people/jeff/Stanford-DL-Nov-2010.pdf>

Stack Traces

For many developers, a stark reminder of failure causing an abstraction to leak comes in the form of the infamous *Stack trace*, a long list of source code locations that specify where and in which context an error occurred. Without having access to, and an understanding of, said source code, the stack trace does little more than just fill your screen with gibberish, though.

Abstraction is a Tool, not a Replacement

Whereas abstractions seem to work well in the happy day scenario, they can quickly break apart in case of errors. So how do you build abstractions that provide value, i.e. shield people from unnecessary complexity, but still provide some assistance in case of failure?

The first trap when designing abstractions is believing that the simpler the abstraction is, the better. To show that, as usual, architecture decisions rarely find the optimum at the extrema, consider this abstraction:

```
void doTheRightThing(int thing);
```

In terms of getting things done with little code, this method surely wins. But it scores rock bottom when it comes to flexibility, expandability, or debuggability. A suitable abstraction, therefore, isn't the one with the smallest syntactical surface but one that gives a meaningful set of constructs. "Meaningful" is admittedly a weasel word here as it assumes we have a good grasp of the meaning we want our abstraction to represent.

As so often, it helps to dig through computer science's past and see what some very bright minds had to say about abstraction:

"The purpose of abstraction is not to be vague, but to create a new semantic level in which one can be absolutely precise." —Edsger Dijkstra

If we unwrap this, we should look for abstractions that eliminate things that don't matter (the noise) to allow us to be precise about the things that do matter. That just leaves us with having to know what matters and what doesn't...

I'd posit that things that matter for a useful abstraction include:

The domain

This one might (and should) be self-evident, but it's better to state the obvious than fall victim to unstated assumptions. An abstraction defines a new vocabulary and domain, usually over an existing one. You have pretty free rein over this new domain, so the main design objective is to choose a domain that's meaningful to the users of your abstraction. With the domain comes the vocabulary and granularity of your abstraction.

The learning curve

People will use your abstraction incrementally. A beautiful abstraction that's very difficult to grasp might be suitable for math PhD candidates but isn't so great for programming interfaces. As described in [Platform Quality](#), building a gentle on-ramp for an abstraction is just as important as the abstraction itself.

Dealing with errors

Being this chapter's key point, abstractions don't just have to serve the sunny day scenario but also have to provide a lifeline to its users when things go awry.

There's a lot more to be said about abstractions and to some extent a good platform is inseparable from the abstraction that it creates.

You Can't Manage What You Don't Understand

A common management adage is that you can't manage what you can't measure. I believe that's only half the equation. Air Transat 236's cockpit crew was measuring all the right things, such as fuel level as well as oil pressure and temperature. However, translating what you measure into meaningful action requires you to have at least a high-level understanding of the system you are managing. The pilots aren't expected to know the part number and outer diameter of the fuel line (maintenance staff will do that), however, being aware of the existence and operational principles of a fuel-oil heat exchanger can be instrumental to selecting the right course of action. At 35,000 ft there isn't a "we'll deal with that later" option.

That's why I have claimed before that you [can't manage what you can't understand](#)⁴ - sometimes you have to open the hood. While not every developer needs to know the innards of the systems down to the hardware level, it's

⁴<https://www.linkedin.com/pulse/you-cant-manage-what-understand-gregor-hohpe>

extremely helpful to have a few folks on hand who can in case an alpha particle wreaks havoc deep down in your engine room. Occasionally, doing so can become a challenge with cloud platforms as many of the operational details are proprietary.

Part V: Implementing Platforms

This part is still work in progress

Building a platform includes a slew of technical decisions. Although the interface into the platform is what platform users see and what largely determines their experience, implementation details determine a platform's performance and its ability to evolve. This part dives into the design of platform internals.

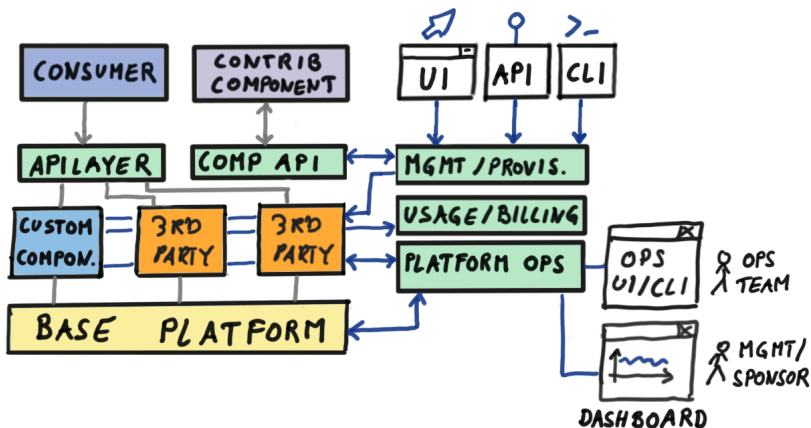
- A platform isn't a single piece but has a [distinct anatomy](#)
- Just like any other software, platforms face several [fundamental design decisions](#)

21. Platform Anatomy

Platforms are designed to be easy to use—low friction is one of their main enablers. As a result, the engineering and operational effort that’s required to build and run a platform is easily underestimated. Although platforms are far from copy-paste exercises, the anatomy of a typical in-house platform is relatively consistent. Understanding the primary user groups, the interfaces, the main components, and how they’re wired together, provides platform teams with a better starting point for teams to make effort estimates and avoid blind spots.

Main Components

Architects aren’t big fans of black boxes—they’re more likely to have taken their toys apart as children. Doing that for in-house platforms will yield the following prototypical high-level diagram (with lines!):



The Anatomy of an In-House Platform

We can divide the core elements of a platform into a few major categories:

- **Interfaces** allows various user groups to interact with the platform, e.g. to provision new resources. For a cloud platforms, the web console where you can provision new services and review your billing statements is a widely used interface.
- **Core Components** are directly used by platform consumers, usually via APIs. They provide the direct value to the users. For a cloud platform, core components would include VMs or database instances.
- **Operational components** manage the platforms behind the scenes. They are usually not visible to end users. For a cloud platform, capacity planning would be included in these components.
- **Base Layer:** most in-house platforms aren't completely built from scratch but sit on top of another base platform.

Let's look at each type of element in more detail.

Interfaces

Interfaces are any system's connection points to the outside world. Platforms support diverse user groups, for example platform users, platform operators, and stakeholders, and hence feature multiple distinct interfaces for each group. We can divide interfaces into two major categories:

- **Operational interfaces** that are used by components consuming the platform, usually via API calls.
- **Management interfaces** that are used to administer the platform, e.g. to instantiate components.

Operational Interfaces

Operational interfaces are machine-to-machine interfaces that enable the usage of the platform by consuming applications.

Platform API

This is the main interface via which consumers interact with the platform. This can be a service API (described next), for example for payment service provided by the platform, or an API provided by the platform itself. Whether to wrap existing services APIs is a major design decision for in-house platform, so this book includes a whole chapter on it: [Beware of the Grim Wrapper](#).

Service API

Many platforms allows users to interact directly with the platform services. For a cloud platform, you may provision and configure a storage unit (e.g. an S3 bucket) via the management interface and then store or retrieve data via the operational service interface.

Component API

Many platforms allow users to contribute components to the platform. Such components will need to interact with the platform in different ways. For example, they will need to advertise their services (by providing metadata), be instantiated through the platform, or report usage data for billing.

Management Interfaces

Management interfaces are primarily used by human actors, either via a UI or via automation scripts or APIs.

Console

The most visible user interface of the platform is the one used by platform customers to provision and manage services. Most cloud providers refer to this interface as “console”, so I am happy to adopt this name.

Operations Interface

The platform team will need its own interface to perform operational tasks. This interface will allow the team to monitor platform utilization, perform capacity planning, or assist users in debugging issues that are not visible through the platform interface. Generally, the more powerful the console is, the less frequently do platform teams have to assist platform users.

Dashboards

In-house platforms typically report usage and key metrics to their business sponsors. Dashboards illustrates these platform metrics in a business-friendly format.

Whereas the operational interfaces interface are virtually always APIs, management interfaces take different forms often provided in multiple forms at the same time:

Graphical User Interface (GUI)

Usually a web interface that allows users to take actions and view information visually. Cloud providers' web consoles are representative examples.

Command-Line Interface (CLI)

Although GUIs are intuitive, especially for first-time users, they don't work well for large platform users (try managing 1000 VMs by clicking checkboxes) and don't support automation, one of the maxims of platforms. Command line interfaces allow automation through scripting. Unlike web UIs, they are platform dependent and therefore require a (usually small) installation for the popular operating systems (Windows, Linux, macOS).

A combination between GUI and CLI are cloud shells that allow users to interact with a platform via command line without the need to install any local software. Instead, they are able to use the command-line interface in a browser UI. Most in-house platforms won't build this functionality from scratch but rather build it on top of existing cloud platforms, like [AWS Cloud9](https://aws.amazon.com/cloud9/)¹.

Application Programming Interface (API)

Last, automated use of the platform can also be achieved by calling platform APIs, often implemented as REST APIs using JSON payloads that are sent via HTTPS requests.

In-house platforms will often only implement a subset of these types of interfaces, most commonly a Web UI and an API. A CLI can often be emulated by making API calls from the command line with utilities like `wget` or `curl`.

Core Components

3rd-Party Components

Most in-house platforms are assembled from existing components. For example, many platforms will operate on top of a cloud provider platform and provide additional services like a Jenkins-based build pipeline for application delivery.

¹<https://aws.amazon.com/cloud9/>

Custom Components

In-house platforms will also add custom components, built by the platform team. These can be higher-level services that in turn call the base platform or 3rd party components, or completely new services.

User-Contributed Components

Many platforms grow by allowing users to contribute components to the platform to be used by other users, but managed by the platform.

Contributed services often use an additional interface, a platform component API, to interact with the platform, e.g. for provisioning, configuration, or providing monitoring and usage data.

Operational Components

Last, the operations interface typically relies on additional operations components that are not visible to platform users. Such components can include billing aggregators or various types of “glue logic” to make all the different components work together.

Base Platform

IT platforms are rarely built from scratch but commonly sit on top of a base platform. Most of the time this base platform is a cloud platform or an abstraction layer like Kubernetes.

Example: Software Delivery Platform

Example: Common Service Platform

Example: Kubernetes

22. Platform Implementation Choices

Knowing which dials are there to be turned.

This chapter is still work in progress

As we learned in *The Software Architect Elevator*¹, architecture is defined by a set of meaningful decisions, those decisions that imply conscious trade-offs. Building a platform is also guided by such decisions. Sadly, we rarely get to hear about them. Let's do some platform reverse engineering.

Design Decisions

Transparency

One of the most fundamental decisions of platform design is whether to expose the innards of your platform to your users. Should your platform be a black box, hidden behind an API layer or should it expose internal components to be accessed by users?

A black box has the following advantages:

- you have full control over the platform implementation and can change it at will because you did not commit the internals to your customers.

¹Hohpe, *The Software Architect Elevator*, 2020, O'Reilly

- You can provide a better abstraction and lower complexity for the platform users because the underpinnings aren't exposed (there are limits to how much you can abstract, though—see [Failure Doesn't Respect Abstraction](#)²)

Opening your platform innards also has advantages

- You don't run the risk of building the [Grim Wrapper](#)
- You allow others to build on top of your lower layers. This is for example useful if your platform is open source or based on open source components.

A visible example of either choice is the implementation of AWS' and GCP's serverless platforms. [AWS Lambda](#)³ is a black box—AWS doesn't expose the underlying runtime infrastructure. Rumor has it that originally it was not running on containers but now is. The [FAQ](#)⁴ simply states: “Each AWS Lambda function runs in its own isolated environment, with its own resources and file system view.” The nice thing is that we don't have to care about how that is implemented.

In contrast, Google Cloud's [Cloud Run](#)⁵ serverless solution uses Docker containers and manages the underlying infrastructure using [Knative](#)⁶, which manages a GKE Kubernetes cluster. All these are derived from open-source projects, and some folks may say that this is a “cleaner” architecture because of a well-defined separation of concerns. Additionally, one layer uses the published APIs of the next layer, so technically you could build or own Knative implementation or let Knative run on a different container orchestrator than Kubernetes. The downside is that Cloud Run comes with a lot of moving parts, including those mentioned before plus an Istio Ingress gateway, Pilot, an Activator, and lots more. Also, you have to deploy your code in a container as that's the universal unit that the underlying infrastructure can manage.

Interestingly, GCP also gives you a “black box” option, [Google Cloud Functions](#)⁷, which takes source code as a deployment artifact and doesn't bother you with all the underpinnings. It seems that GCP understands the trade-offs and hence gives you both options.

²<https://architectelevators.com/architecture/failure-doesnt-respect-abstraction/>

³<https://aws.amazon.com/lambda/>

⁴<https://aws.amazon.com/lambda/faqs/>

⁵<https://cloud.google.com/run>

⁶<https://knative.dev/>

⁷<https://cloud.google.com/functions>

Layering

A related decision is how cleanly your platform should be layered. If your platform is opaque, that's a decision you are just making for yourself. In a transparent platform, in contrast, you will likely be leaning towards layers because the internal structure is exposed to your users and you wouldn't want that to be a giant mess.

The pros and cons of layering have been fairly well explored, so here just a brief summary:

- Layers provide separation of concerns: one layer does one thing and does it well (admittedly, a component architecture that isn't layered can accomplish that also)
- Dependencies are clear
- Interaction occurs through well-defined interfaces
- Thanks to clear dependencies, it's easy to replace one layer with a different implementation
- Changes can propagate through multiple layers, slowing down development
- The layer interfaces can incur run-time overhead
- Lower layers can shield access to implementation details that could make for a more efficient or better-performing implementation

Tenancy

Most platforms support multiple tenants, giving each tenant the perception that they have an entire platform of their own. Interestingly many of the cloud's tenants are SaaS companies, which in turn serve many customers, also giving each customer the perception that they own an independent instance of the application. Such SaaS services can then be integrated into commercial products that either support multiple divisions of a large organization or multiple customers.

This layered tenancy leaves some interesting design decisions. E.g. should a SaaS provider build its own tenancy system or rely on the tenancy of the underlying platform? Likewise, in-house platforms layered on a cloud platform must decide whether to use a central set of accounts to manage services on behalf of their internal customers or whether they just hand out accounts for each team to set up their own instances.

Run-time model

Extensibility

Identity Management

Part VI: Growing Platforms

Platforms thrive on scale, but platform users don't just appear overnight. Instead, they have to be identified, recruited, and carefully nurtured. If your platform is too basic, it'll turn off early adopters and fizzle out before it ever gains traction. Vice versa, over-investing into features before you have real user feedback is risky because it could lead you down a wrong path.

This part discusses how can you grow your platform over time to expand its user base.

- Platforms live in three dimensions, so it's natural to [model platform evolution as a cube](#)
- It's easy to assume that each user is "all in" on a platform, but that's not how things start. [Plotting users' experience with your platform over time](#) can be a valuable design technique.
- Maintaining a [platform roadmap](#) is a delicate balancing act between accepting user input and sticking to the product strategy.
- You may not be able to serve a wide range of customers with a single offering. Instead, you may have to [tier and slice](#) your platform.

23. The Cube of Platform Evolution

Seeing more dimensions

Architects see more dimensions. The same holds true for in-house platform evolution, which occurs along three dimensions.

Platform Evolution

Successful platforms are rarely static. You're unlikely to be able to build a comprehensive platform in one run, so you'll have to evolve your platform over time. If you are building your in-house platform on top of an existing platform, such as a cloud platform, both platforms are **bound to evolve**. Finally, your customers' needs will evolve over time, adding to the constant movement.

The “Cube”

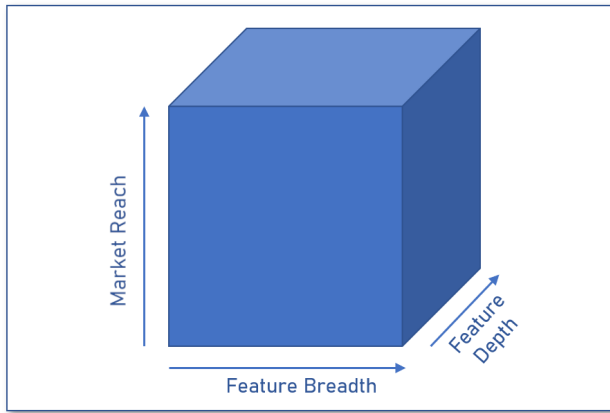
Successful platforms evolve based on user feedback. However, increasing the functional scope isn't the only dimension along which a platform grows. Along the multiple dimensions of platform evolution, there isn't a single right path for all platforms. Instead, a decision model can help teams make a better decision that balances technical aspects with platform adoption.



When **designing platforms for GovTech Singapore**¹, we used a series of simple models that helped teams prioritize their investments over time.

One of these models took the shape of a three-dimensional model, which we termed “The Cube” (in this case not inspired by the like-titled movie):

¹<https://www.linkedin.com/pulse/building-smarter-nation-simple-models-lead-better-decisions-hohpe/>



Three Dimensions of Platform Strategy

The cube includes three dimensions of platform evolution along which platform teams can plot their journey: market reach, platform breadth, and platform depth:

Market Reach

Platforms enable and speed innovation in other parts of the organization, so they need broad adoption for the platform to be successful, both economically and technically. Focusing on market reach can counter-balance the temptation to build the “perfect platform” before actively growing its user base.



Building a platform without actively growing its user base can result in a platform that’s perfect for the platform team but not its users.

A platform’s reach can be defined by its addressable market, TAM (Total Addressable Market) in sales speak. A platform increases its reach by identifying and actively promoting the platform to new user groups, new geographies, or new lines of business. So, in a way, reach itself has multiple dimensions that are sometimes referred to as market segments. For example, you may choose to push for more adoption across user groups (such as large and small projects) in a specific geography, or you may go the opposite route and try to serve a specific user group (for example, highly skilled teams) across geographies.

Cloud providers' sales teams follow a similar approach by organizing their teams (and revenue objectives) by geography (usually Americas, EMEA, and Asia-Pacific), industry segment (like Finance, Automotive, Media, Energy), and customer maturity (start-ups, small business, enterprise). An internal platform may not serve different industries, but large organizations often do consist of different product lines (e.g. retail, commercial, and wealth management for a bank) or different stages of the value chain (product development, manufacturing, distribution, retail). Plotting your current adoption and your future plans across these dimensions helps you express your platform roadmap. It can also influence your team's structure, for example, you may need experts in product development to gain users in that area, or you may need local support teams who can assist customers in their time zone.

Platform Breadth

Your platform's reach can only grow if it offers the features needed or desired by new user groups. Therefore, you need to grow your platform along with your markets. Once again, it's helpful to subdivide your platform growth into two dimensions, breadth and depth. The breadth of your platform is measured by what portion of the available problem space it addresses, a [characteristic](#) typically considered as "completeness" by its users.

If you are building an application delivery platform (see "Application-Centric Cloud" in *Cloud Strategy*²), increasing your platform's breadth could take the form of adding monitoring or communication services to existing tool chain and run-time components.

A broad platform can boost the value proposition for both users and the enterprise. At the same time, a broader platform also increases the maintenance and support burden for the platform team. Growing too fast might distract from other aspects of platform evolution.



Focusing on breadth alone can dilute a platform team and compromise other characteristics like quality or cohesion.

A very broad platform can result in a [user experience cliff](#) for new adopters and thus actually impede market reach.

²Hohpe, *Cloud Strategy*, 2020, Leanpub / Kindle Direct

Platform Depth

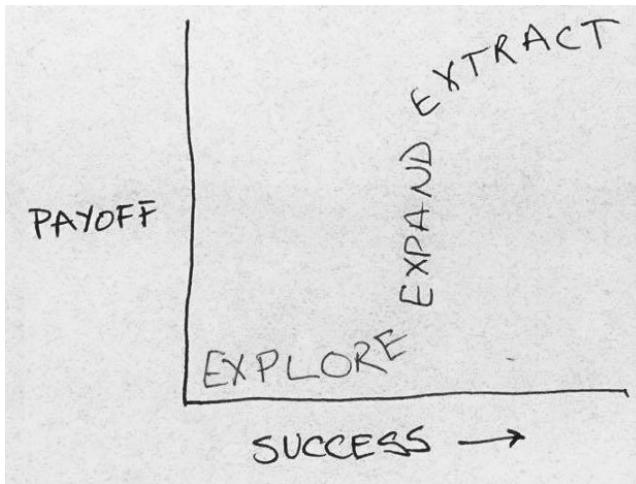
The platform depth defines the completeness, quality, and sophistication of the elements covered within the platform's breadth. Coming back to the application delivery platform example, depth inside a CI/CD pipeline can define the degree of automation, or the tightness of integration between components.

A deep platform can offer a superior user experience by offering ready-made templates for common projects types, which can be set up with the push of a button (or rather a single command line). Tight integration can take the form of webhooks setup for a git repository to achieve continuous integration, storing artifacts in a central repository, and deployment into a test environment.

A deep platform is often perceived as "well-thought-out" or "smooth" by users as they find that the features are well-rounded and tightly integrated.

The Three X of Product Lifecycle

When looking to plot your road to market, I found Kent Beck's [3X mental model](https://medium.com/@kentbeck_7670/the-product-development-triathlon-6464e2763c46)³ very useful. After all, he spent several years at Facebook, who surely knows a thing or two (or perhaps three) about platforms. Kent divides the evolution of a product (including platforms) development into three distinct phases:



Kent Becks "3X" Model

³https://medium.com/@kentbeck_7670/the-product-development-triathlon-6464e2763c46

- **Explore:** First, you need to find out what works. You could do that through clever guessing (or hypotheses), which you can test with low-cost experiments to obtain feedback from prospective users. This phase should not be confused with a technical PoC (Proof of Concept), which would assure the platform can work. Instead, it tests its viability in the hands of users.
- **Expand:** Once you find something that is valuable to users, you want to find more users to delight. The critical activity in this phase is removing obstacles to further growth, for example by making features available as self service or minimizing the needs for in-person support through ease of use and high-quality documentation.
- **Extract:** When the solution space is clear and a user base is established, you can start harvesting from your success. Even internal platforms must live up to a business case, so this phase is critical to make the economics work, for example by benefiting from economies of scale. Extracting can also take the form of replicating the now well-established platform in new geographies or markets.

Kent's model is simple but powerful because it helps us pay attention to a few critical aspects:

- Each phase is distinct and requires different tools, approaches, and sometimes people.
- The transitions need to be well-timed. Too many teams give up too early during experimentation and run with something that won't sufficiently expand. Or they succumb to (often external) pressure to extract when they could have expanded much more.

Sometimes you need different skills or different leadership for the phases:

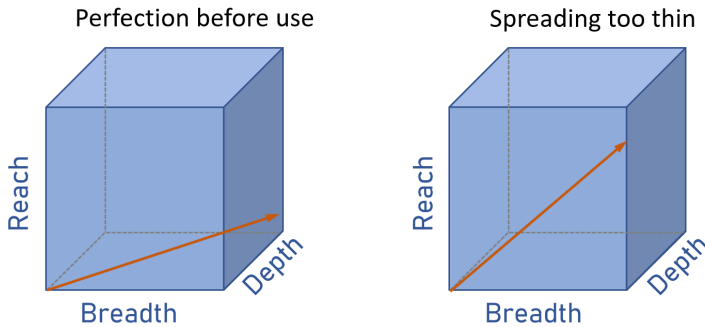


When building an internal platform to transform software delivery in a large organization, I focused on experimentation and expansion. We then handed over the effort to a different lead for the extraction phase.

If you look at the way Kent depicts the model, you will notice that I am most interested (and valuable) when the first derivative is largest. A friend of mine expresses his similar strengths in a more pointed fashion: "I am a cobbler, not the shoeshine service" (although one should be careful to not discredit the extract phase as that's where the ultimate impact lies).

Common Pitfalls

Models are useful if they help you make better decisions or keep you from repeating others' mistakes. As simple as the cube is, it enables a fruitful discussion that helps platform teams be more effective and make a bigger impact. It also allows us to illustrate common anti-patterns:



Plotting platform evolution antipatterns

Perfection vs. Actual Use

Too many teams, especially those with significant upfront funding and long planning timelines, meander along the cube's bottom plane for too long, pushing further to the right and the back but never lifting up. This approach not only slows the path to value, but it also deprives these teams of vital feedback signals from actual users.



I have seen large enterprises invest 18 months into developing a cloud platform that would “streamline cloud development” across the organization. When completed, it could hardly find any users: progressive teams were already building applications and considered the platform overly prescriptive whereas less advanced teams found the platform to be too complex.

These teams are often led by the misconception that building the platform in one shot is more efficient than growing it through discovery. Although such an approach may look promising paper, it violates a [key principle of platforms](#): you cannot anticipate every user need.

Not engaging users early leads platform developers into a dangerous trap that's aptly described by Jaroslav Tulach in his book *20 API Paradoxes*⁴. The paradox "API Authors" illustrates how API authors tend to build for themselves as opposed to their users because they talk within their development team more than with their users.

Breadth vs. Depth

No effort is ever without constraints, so your platform team needs to be conscious of the trade-offs it makes. The chapter on Application-Centric Cloud in *Cloud Strategy* suggests balancing breadth and depth by growing the platform's breadth first before making additional passes to increase depth across these components:



Growing an Application Delivery Platform

Such an approach is sometimes referred to as “upward spiral”: you work in iterations, adding a little to each portion of the platform. The alternative is “divide and conquer” where you complete one aspect (such as the build pipeline) first and tackle new areas (like observability) later.

Depth vs. Complexity

Too much depth can also be detrimental. Many cloud providers design their services based on feedback from customers. Inevitably, larger customers will get more attention because for one there's fewer of them, and they also present a larger revenue opportunity. However, large customers may have unique needs. Perhaps they need more security controls or use a service in an uncommon way. Adding features for such customers can increase the cognitive load for other users, leading to a [learning cliff](#) for those customers.

As you won't be able to [slice or tier](#) each individual service, balancing feature depth with complexity will be a tension that most platform teams have to tackle.

⁴Tulach, *20 API Paradoxes*, 2012, WalrusInk

Principles Guide

It's normal for platform teams to be pulled in many directions. For example, you'll always have a prospective customer who will adopt your platform "if only it had this one important feature". At the same time, existing customers may require significant handholding because the documentations isn't refined yet. Adopting clear principles can help platform teams chart a balanced course through the platform evolution cube.

For example, a team may adopt a [cohesion over completeness](#) principle to resist the temptation to add ever more features that aren't tightly integrated, which would lower the value proposition due to the [fruit basket effect](#). For a catchy (and less academic-sounding) way to express this principle, you could state "tasty fruit salad over large fruit basket".

Transparency buys Goodwill

Being transparent about your choices can help gain some goodwill from existing and prospective customers. Customers will understand that your team can't provide everything immediately and are more likely to be patient if you maintain and publish a credible roadmap based on your principles. Models such as the cube help you communicate your decisions and trade-offs clearly to a wide range of stakeholders.

User Perspective

Another dimension on how platforms progress is through the eyes of the user. Users don't see the whole platform right from the beginning but will explore and learn through use. We can plot such a user journey as [a set of curves](#).

24. The Shape of Platforms

Plotting a path through platform adoption

Whereas platform designers often see the platform as a whole, users experience platforms in a journey over time. That journey will be defined by an initial learning curve, followed by evolving needs, growing scale, increasing integration, or changes in other dimensions. Visualizing what that path looks like for the platform you are building can be one of the most important design decisions.

It's all about the ramp

Software pioneer Alan Kay aptly captured the complexity of designing good interfaces in a deceptively simple piece of advice:



Simple things should be simple and complex things should be possible

A platform that manages to live up to these expectations makes it easy for users to get started and grows along the users' needs.

Typical Curves

Architects make better decisions by using decision models. Those models don't tell you what to do, but they give you an expressive vocabulary of options and highlight the implied trade-offs. A decision model for platform experience profiles helps us sharpen our senses for how users perceive the platform.

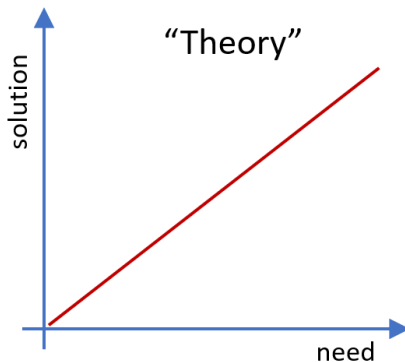
Users' experience with your platform isn't static. They might start simple, using only a portion of the platform. Over time, their demands likely grow, causing them to fully utilize and perhaps stretch your platform. This journey can occur along functional demands (the need for more features), scope of usage,

or operational needs, (e.g., expected throughput). As users' demands increase, the effort required to build on the platform will also grow. Just like the growth of needs, effort isn't a single dimension: it could be real effort in terms of developer hours, an increase in cognitive load when learning new platform features, or the complexity of the final solution built on the platform.

The correlation between these two dimensions is a critical design aspect for platforms (or most complex software systems). Plotted along two axes, indicating the degree of user needs and solution complexity provides several prototypical graphs.

Theory

In theory, the solution complexity or effort should increase linearly with your needs, in line with Alan Kay's observation:

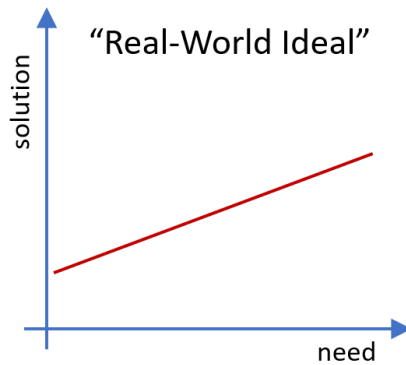


The theoretical curve. You won't see this in reality.

That's the theory. In practice, no platform, even one with low friction, has an initial effort of zero. Also, very few platforms will meet user's infinitely complex demands, so there is likely a saturation point where the users' needs will outgrow the platform's capabilities.

Ideal

The best case a platform team can hope for is a curve with a moderate base-level effort that, once that hurdle is taken, satisfies the users' needs for most of the way at a moderate increase in effort or complexity.



The best you can do in reality.

The initial complexity contains the platform vocabulary (e.g. service names for cloud providers) or fundamental concepts like fine-grained security.

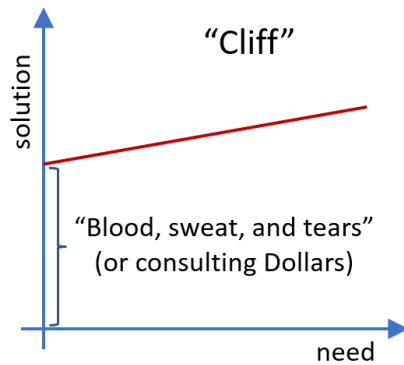
When mapping this curve to performance-related needs, most serverless runtimes would achieve this curve: there's an initial learning curve, consisting of invocation and concurrency models or restrictions like functions' runtime limitations. Once users take that hurdle, solutions can scale quite freely to heavy loads.

Related to functional needs, general-purpose programming languages coupled with generic libraries would fare similarly. Users need to tackle initial language syntax but then are able to solve a wide variety of problems using the same language.

Cliff

As the name suggests, the ideal curve is difficult to achieve. Platform providers therefore tend to find themselves having to balance the low end of the curve against the high end (gear heads will find this task somewhat similar to building an engine with a wide torque curve, before the advent of turbos and electronic engine control, that is).

Platform designers who want to make sure that the platform handles high functional, performance, and security needs, often present the user with an initial cliff:



New users may never see the beauty of your platform.

Despite the platform team's noble intentions, the cliff is going to make a very rough start for new users. They either struggle to get anything working at all or require external help.

The already steep learning curve of such platforms is exacerbated by lack of feedback that users receive: if you can't get anything working, you are not able to get into an incremental learning cycle. Instead, users feel like they're tapping in the dark. To them, the system appears chaotic (as per the [Cynefin Framework](#)¹) because they are unable to see the underlying structure and principles.

The Unix shell or editors like `vi` could be described as such a system: at the very beginning the command line seems very intimidating. You know (or are told) that even complex problems can be solved in just a few lines, but you can't even remember whether `grep`'s `-i` options ignores case or inverts the search. The initial learning curve is indeed steep because you will need to know at least a dozen or so commands before you can be proficient. But once you are, it's highly unlikely that you'll have to change tools as your needs evolve.

Folks who are proficient on the platform, which generally includes the platform provider, will suffer from the curse of knowledge:



The curse of knowledge makes it difficult for platform designers to estimate the initial learning curve.

They have long forgotten their initial struggles and might fail to empathize with

¹<https://hbr.org/2007/11/a-leaders-framework-for-decision-making>

new customers.

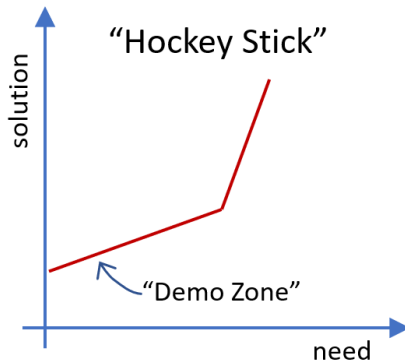
Common techniques to lower the cliff include:

- Default settings for common cases
- Templates for a defined set of base cases that allow users to get up and running quickly
- Documentation and training, although these may not scale as well as a self-service platform
- For front-end tools, tips of the day can help to gently introduce users to more advanced features

If you are looking to [increase your market reach](#), avoiding the initial cliff is going to be a critical part of your platform design. Otherwise, users may never see the great capabilities of your platform under heavy stress.

Hockey Stick

Being intent on lowering the unpopular cliff to increase adoption, many platform designers face a dangerous trade-off to compromise the upper end:



Bad things await your most loyal users

Products with a smooth on-ramp demo well as they handle the limited requirements with great ease, which often earns them the derogatory name “demo ware” because the price to pay arrives with the same surety as an overdrawn credit card. Once the users’ needs exceed the initial “honeymoon” stage, the

platform demands vastly increased effort to tackle more complex requirements or might even be unable to support the users' needs.

Such a curve that's initially quite flat, and then quickly spikes up, is well-known as the "hockey stick". Platforms that achieve easy of use by making many assumptions can fall into this category, as do some "low-code" tools: live is good inside the walled garden but sooner or later, the answer to your needs becomes: "custom function", "extension", "plug-in" or the like. Those platforms had to anticipate users' needs to make addressing those very easy. As long as the platform provider guessed correctly for your situation, live will be good. However, good guesses are hard to come by, so sooner or later you're finding that you're not in Kansas anymore.

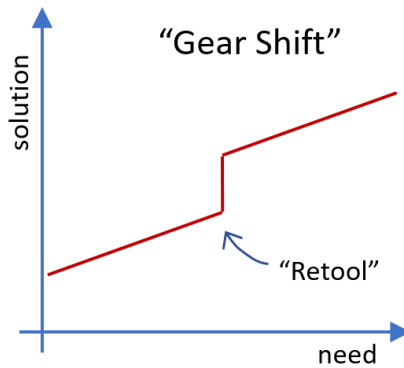
The stick can rear its head both on the platform's runtime (for example, when scalability is limited), the available flexibility (when changes are limited to configuration settings that the platform has to anticipate), or the developer experience. Visual editors (endearingly called "doodleware") are often examples for the latter. They make for a very intuitive start for users but struggle to scale to larger solutions that'd fill many screens full of boxes and lines.

Dampening the hockey stick isn't easy as it's often the result of fundamental design constraints baked into the platform. Possible approaches include:

- Providing [multiple tiers of services](#)
- Giving customers the ability to refactor their solutions to handle increased (operational or functional) demands

Gear Shift

Some platforms that do reasonably well on both ends of the spectrum often resort to having the user switch gears along the way (coincidentally that's also how cars deal with the limited torque bands of traditional combustion engines):



Shifting gears

The gearshift, although not ideal, is still much better than hitting the hockey stick. Some gear shifts can be relatively painless. For example, a platform might have operated with default settings that you now need to explicitly specify. Or perhaps you started the project based on a template that no longer serves your needs and that you now have to either customize or replace.

More significant gear shifts are those that require you to abandon part of your work so far.



Many cloud platforms provide message routing and filtering services for serverless applications. When a user outgrows the expressiveness of the filter expressions, they have to replace the logic with a custom-built serverless function.

In most cases, the work to configure the managed service (encoded in a Terraform template or CDK code) is essentially lost because the serverless function will be written in JavaScript or Python. Nevertheless, this gear shift is still preferable to a platform that won't support the increased customer needs.

Another gear shift faced by platform customers is starting work in a user interface console just to find out that they need better reproducibility and version control. In the early days of cloud consoles, this gear shift meant starting over. Now, most automation tools provide a way to retrieve the current resources and convert them into an automation template or use AI for [console to code](#)² features.

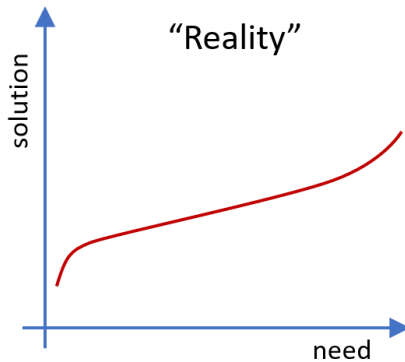
²<https://aws.amazon.com/about-aws/whats-new/2023/11/aws-console-to-code-preview-generate-console-actions/>

Common techniques to lower the gear shift jumps include:

- Automated exports from one approach into another when needed
- A wider service portfolio that provides the users with incremental solution choices

Reality

Any real platform is going to exhibit a combination of the aforementioned characteristics. The goal of this chapter, therefore, isn't to present a recipe for defining the ideal experience profile but to provide a vocabulary to describe common traits, much like design patterns do. This vocabulary enables platform designers to have conversations like “can we lower the cliff by introducing an additional gear shift?”



Real life may serve you an S-curve

In reality you can likely achieve a close-to-ideal curve around common use cases with a manageable cliff at the low and custom effort at the top end.

Growing along the platform

Teams building their own solution will invariably find them easier to use than a platform that's being provided. Part of the reason lies in the team growing a solution alongside their needs, meaning that the solution will be simple while

their needs are still simple. Once their needs become more complex, most team members are familiar with the solution and will find it intuitive, even if it isn't.

When presented with a ready-made, powerful platform such teams might be intimidated and argue that it is overly complex. Building a smooth “on-ramp” for new teams is therefore doubly important for platform teams to gain wide acceptance within an organization.

Lane Shift

In addition to a gear shift, developers who build on top of a platform may also need to change lanes once in a while, meaning they may switch from using one service to another one.



The worst assumption a platform can make is that many there is a well-defined problem, developers select the perfect services to solve that specific problem, and they live happily ever after.

Development teams may consider switching services for a variety of reasons. They may have outgrown the functional capabilities of a service; they may realize that a service was overkill and switch to a simpler service; requirements may have changes; or they may have simply picked the wrong service for their needs.

Large platforms, especially cloud platforms, can provide surprisingly little support when it comes to switching services.

25. Charting a Platform Roadmap

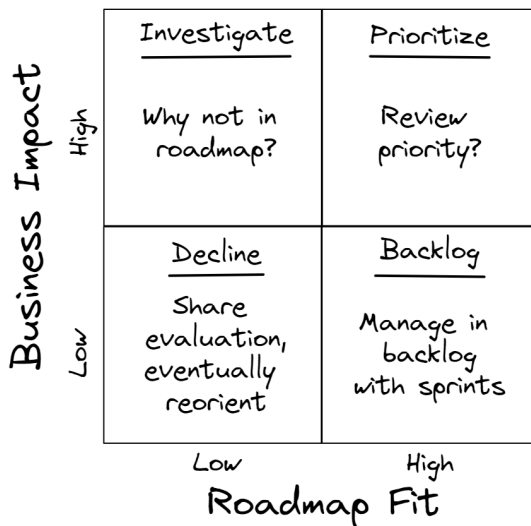
Keeping course while avoiding blind spots

User input is the most valuable asset for platform evolution. However, users rarely deliver you perfectly specified requirements that line up exactly with your planned roadmap. Hence, collecting user input is much more than taking down notes and building what users tell you they want or need. Decision models can help platform teams maintain a healthy balance between product integrity and responsiveness to user requests.

Balancing Completeness with Cohesion

Some essential [platform characteristics](#) can be pitted against one another. For example, adding features suggested by users can increase a platform's completeness, a characteristic that is likely to find immediate appreciation because it ticks an easy "can your platform do X?" box. However, haphazardly adding features is likely to hurt a platform's cohesion or closure. While equally important, these characteristics may not be as apparent to prospective users, leaving platform teams with a difficult argument to not include a requested feature.

Decision models can help teams maintain a balanced roadmap, boost decision discipline, and help communicate decisions to users and stakeholders. A simple 2x2 decision matrix across business impact and roadmap fit can be instrumental when prioritizing items on the platform roadmap:



Folding feedback into the roadmap

Assessing the fit of a request with the platform strategy helps you avoid getting sidetracked into dangerous detours while also keeping a keen eye on inputs that may lead you to pivot the platform. By mapping a feature request to a quadrant, the platform team can decide whether to prioritize, investigate, queue or decline:

- If a high-impact item is already planned (upper right), it confirms the team's chosen opinions. The team might nevertheless re-evaluate the current backlog priority to optimize value delivery.
- A user request that has a high perceived impact but is outside the current scope (upper left) challenges the current platform opinion. The team should investigate whether it's an area that has been intentionally left out of scope or whether they have a potential blind spot.
- A low-impact suggestion that doesn't fit the current direction (lower left), can represent a dangerous deviation. The team must exercise prudence, balancing the [engineering and marketing parts of its organization](#). Declining can be the best option because trying to accommodate anything into the platform will dilute the team.
- A low-impact suggestion that lines up with the platform strategy (lower right) represents the business-as-usual space. The team prioritizes requests into the backlog and delivers according to relevance and capacity.

Especially in the early stages of the platform evolution, the desire to meet an

initial customer's every demand is a slippery slope:



Building to a single customer's needs, although well-intentioned, reverts your platform team from a product team into a professional services organization.

Therefore, a product vision is important to allow the platform to gain shape and further evolve based on user feedback.

Customer Feedback is Biased

Successful platforms don't try to [anticipate every user need](#). Instead, they should grow based on user feedback. You must keep in mind, though, that your current user base is not a representative sample set. Your current users see the world through the lens of the current product, meaning they are biased.



I once challenged a team who stated that “users asked for this” whether they also talked to the 1 million non-users of their service.

This effect is particularly pronounced for platforms that comprise many services. Customers of Service A may wish for a new feature that's already available in Service B. If the team building Service A isn't aware of Service B, it may be tempted to duplicate that feature. When soliciting customer feedback, it's therefore helpful to look at the platform as a whole, not at each service in isolation. It's also good practice to approach customers with hypotheses and options as opposed to just asking “what would you like”.

Selecting Success Metrics

Expecting that a platform [fixes all problems would be naive](#). Existing organizational processes can keep even the best platform from reaching its intended potential. For example, test automation may not reduce change lead time unless upstream issues such as changes being reviewed in large batches are resolved. Vice versa, a lower friction deployment process can motivate teams to work in

smaller iterations and thus reduce debugging cycles, which would usually not be captured in platform metrics.



An IT executive evaluating investments in an in-house platform initially looked at platform-centric metrics such as how many jobs ran on it or how many teams adopted it. But did those show the real impact?

Platform teams can use or extend [DORA metrics](#)¹ to measure development team performance. These metrics value the frequency, speed, quality and stability of the software release to production. Such metrics provide insight into the build and run phases of a software product. Several products, including open source projects like [Apache Devlake](#)², provide access to such metrics and offer an open model of the underlying data. Additional indicators can measure release size, test-book size and composition (ex. manual vs automated tests), backlog trends, number and type of quality issues.

Platform runtime data can help illustrate platform usage. This second set of lower-level metrics is simpler to measure and provides additional insights into the way teams work and their usage of the platform.



Metrics should highlight the interaction between platform capabilities and other practices in the organization.

To uncover links between platform usage and practices, you can analyze teams with similar or different process performance by looking at the process flow (the process steps) and operational indicators. By identifying cohorts of groups that move across the process flow similarly (analog to customer journey funnel analysis), platform teams can collect additional evidence to analyze which factors enable or hinder higher productivity and how the platform is responding to them.

¹<https://cloud.google.com/blog/products/devops-sre/using-the-four-keys-to-measure-your-devops-performance>

²<https://devlake.apache.org/>



Collecting data from the build toolchain and planning tools helped us identify friction between the inner and outer loops. Developers postponed quality and security analysis to cope with that friction. We also discovered that application architecture limitations led development teams to deploy larger batches of changes, creating more friction in the form of more complex and longer QA cycles.

As in any IT system, platform metrics are valuable to highlight quality issues and address capacity or performance issues that will result in lower customer satisfaction and friction in its adoption.

Retrospectives

Noble intentions don't always address the users' most immediate needs. Joint retrospectives with developers and platform engineers allow the teams discuss what works well and what does not: developers may need more support or training; the platform may have undetected or unresolved gaps; architectural or organizational aspects may hinder the platform from delivering its full potential.

A retrospective helps teams understand which aspects drive higher productivity, enable the adoption of best practices, and support a compliance posture. The retrospective looks at the relationships between the platform, the organization's processes, and the output of application teams to understand how they influence each other.

The retrospective may uncover different findings leading to improvement actions for the platform team. For example:

- Application teams may miss critical features.
- Teams may be unable to exploit the platform because of knowledge gaps.
- Poor quality and security practices cause overhead or tempt teams to adopt lesser practices to match time-to-market demands.
- Governance processes required for risk and compliance management—for example, asset and release management processes—may not be well supported by the platform automation, leading to manual overhead.

The platform is not a magic wand for transforming the organization. Other aspects, often at the intersection of processes and applications, may influence

the output of application teams, for example the quality of requirements and project planning.



Running retrospectives should not be a one-and-done exercise but be an integral part of the normal mode of operations.

Sensing the Future

Data are important to run a platform, both as a technical system and as a product. Telemetry provides visibility into the stability and reliability of platform operations. Data can also be used to support the platform team in improving its customer experience.

Usage data can help the organization [sense the future of the platform](#). Relating the evolution of the landscape with the consumers' needs can identify new areas or help expand existing features. For example, an observed increase in data & analytics projects may lead the team to consider expanding the service catalog to ease data engineers' work with data orchestration or model serving.

Just like with explicit user feedback, platform teams need to balance these implicit inputs against the platform roadmap. Such additional data points feed our platform strategy, providing teams with the opportunity to measure their position relative to their users, and eventually pivot if we were to recognize a significant shift in needs and habits.

Panta rhei

Everything flows, *panta rhei*, an aphorism attributed by Plato to Heraclitus, represents the view that nature is in a continuous flow of change. The same is true about IT, whether the change is driven by technology evolution or external forces.

The previous sections show how a platform team's path may be shaped by the way existing users work and how the team chooses to respond. But bigger changes may be driven by external forces. Simon Wardley defines a set of climatic patterns, which he describes as things [which change regardless of your](#)

actions³. The use of the term “climate” to group these effects reminds of a common meteorological rule, coined by Alfred Wainwright :



There's no such thing as bad weather, only unsuitable clothing.

So, if your platform seems to face bad weather (or climate change and **rising ocean levels**), it may be time to rethink your wardrobe. Simon's list of climatic patterns help teams understand some common external forces that they'll have to react to.



Technical evolution will trigger changes in behavior, which will demand new technical capabilities.

Such changes will occur at different levels with technology being just one of them. In many cases, platform teams will witness change driven by a cycle of evolving technology, which leads to new usage patterns, which in turn trigger new demands for technical solutions. For example, cloud automation makes provisioning infrastructure repeatable and effortless. As a result, teams can make their environments disposable, destroying and re-creating them as they please. Such habits drive the need for much faster deployment cycles or more expressive automation languages.

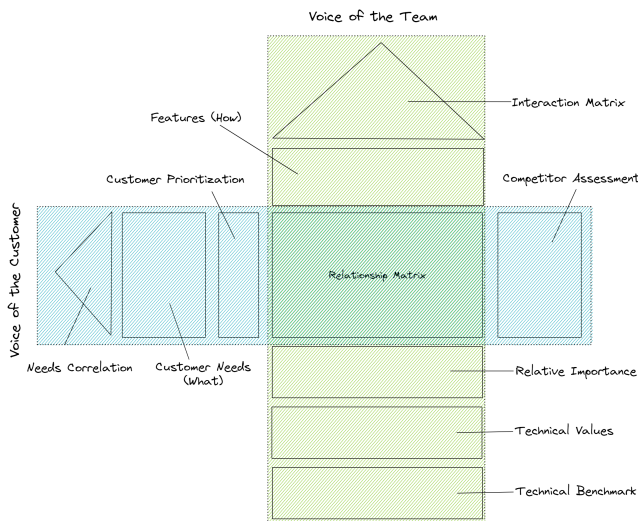
Just like user input, climatic changes do not simply translate into an action or an adjustment of the platform roadmap. Rather, a platform team needs to identify if and where to apply the new capabilities. For example, while still at an early stage, Generative AI is clearly transforming software development to be AI-assisted. This prospect raises important questions about the notion of a **golden path**: why would a platform team need to explicitly code paths when an assistant can guide application teams? Would platform teams tune the assistant to the organization's context instead of developing platforms?

The organizational ability to sense the future and take action determines the long-term viability of a platform team. Some organizations struggle to see an evolutionary path for their platforms, instead falling into a build-use-dispose cycle. For example, while they may be enamored with the power of modern AI tools, they may fail to consider it as input into their current developer platform roadmap.

³<https://medium.com/wardleymaps/exploring-the-map-ad0266fad59b>

The House of Quality

Quality Functional Deployment (QFD)⁴, another inspiration from the automotive industry, helps translate customer needs into technical requirements. In QFD, quality is a measure of customer satisfaction with a product or a service. Each identified customer need is evaluated against relevance, benefit, technical complexity, and difficulty, plus a benchmark against other in-house or external solutions. The method's primary model, the House of Quality (HOQ), provides an approach to prioritize functionalities, as popularized from books like Continuous Architecture: Sustainable Architecture in an Agile and Cloud-Centric World to plot product architecture evolution through transitory plateaus.



House of Quality

At first, prioritized customer needs are evaluated against technical characteristics based on their relevance, leading to the identification of importance. The interaction matrix (the roof) is used to identify features that may or not coexist. Technical values can be used to capture characteristics associated with features, a service level objective for example. Additionally, technical benchmarks highlight intrinsic difficulty and quality compared to competitors, informing the technical decisions of the platform team.

⁴<https://asq.org/quality-resources/qfd-quality-function-deployment>

This approach has a positive impact even when you're merely adopting a platform because clarity on features will help prioritize which features to activate and pay for. It'll also assist the team in planning enablement and creating a clear communication plan to promote the new platform.

Building Platforms Outside-In

When building an in-house platform, balancing unique user needs with providing a cohesive offering is delicate. On one hand, an in-house platform should address requirements that are unique to the organization because otherwise an existing (commercial or open source) product would be a more economical choice. On the other hand, platforms should not have to reinvent the wheel to address common needs like continuous software delivery or automated testing.

Many platform teams therefore build a platform from two separate parts, a base layer in which the platform team essentially anticipates common needs and a custom service layer that is discovered from specific customer needs.



The Singapore Government Tech Stack (SGTS⁵) consists of a base layer, which is largely pre-defined and a service layer that is incrementally built or harvested from agency applications.

⁵<https://www.developer.tech.gov.sg/singapore-government-tech-stack/>

26. Tiering and Slicing

One size doesn't fit all

By Jean-Francois Landreau

When designing platforms, like any software product, starting with a single eager customer is a good practice. Success with this initial customer prepares you for the needs of a second willing one. This second iteration helps you generalize concepts and discover patterns. With the third customers, the generalization can become good enough to address the needs of many customers. But transitioning from well-funded customers to the “long tail” may require you to chop up your platform.

Scaling down

Inside large organizations, your initial customers are likely to be “premium” ones: their business need for an internal platform derives from their current success and rapid growth. Or they need to standardize to cope with large scale and complexity. Working with well-funded and fast-moving customers is a great start for a platform, but invariably you'll encounter customers with less budget who can nonetheless benefit from your platform. However, your initial platform design may be too expensive for these new customers.



At a large financial institution with dozens of operating entities around the world, there was a saying: “you addressed the largest entity, but is it going to work for Sri Lanka?”

A single platform may not be able to serve such a wide range of customers equally well. In fact, smaller customers may challenge the design of your internal platform more than large customers:

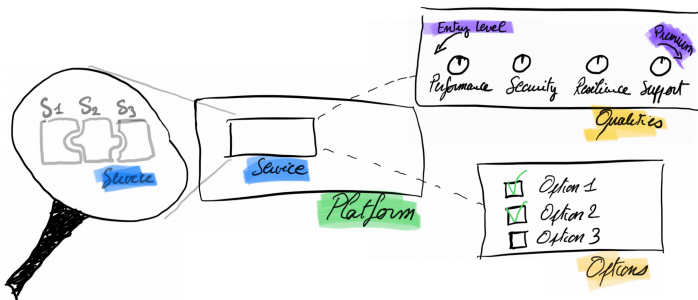


Scaling down is often harder than scaling up.

Adding more capacity or functionality is typically easier than reducing cost. For example, a customer with a single virtual machine may still require a complete [control plane](#) or services like a CI/CD pipeline, monitoring, or network security. Ironically, the more your platform is like a [fruit salad](#), which is a good thing, the harder it might become to scale down as multiple components are tightly integrated.

Two mechanisms can help a platform serve a wider range of customers:

- Providing different “vertical” tiers, e.g. for different degrees of operational demands.
- Slicing the platform’s functionality “horizontally” into modules that customers can consume independently.



Tiering and Slicing a Platform

Providing such tiers and slices not only helps your smaller customers but can provide a [smoother on-ramp](#) for large customers as well as they may start small before they grow.

Small customers come in a variety of flavors:

- *Experiments* that aren’t meant to be production-hardened, but that are intended to test functionality or new product ideas. Such experiments are important innovation drivers and should be supported by an in-house platform.

- *Development environments* will have lower security and scale needs than their production equivalents.
- So-called *Strangler Applications*, where a team replaces a small part of an existing application with a modern implementation, implement only a few components of a larger system and may have a small initial scale.
- *Low-volume, high-value* applications (for example, insurance applications) will have vastly different operational requirements than *high-volume, low-value* ones (for example, web logs). For example, the former don't need to scale but will often have strict security and transactional needs, requiring exactly-once and in-order delivery of events. In contrast, the latter may need to handle significant scale but can afford out-of-order or even dropped data.

Tiering

[Automotive manufacturers](#) have managed to employ platforms to build a wide range of vehicles from middle class to luxury cars, each of which serves a specific customer segment. That way, they manage to tier their offering while avoiding duplication.



Tiering doesn't mean offering poor products to some customers, but offering the right product for the right use case.

Tiering isn't limited to a single dimension. A single customer may require the top security tier but the lowest scalability tier whereas another customer may require just the opposite.

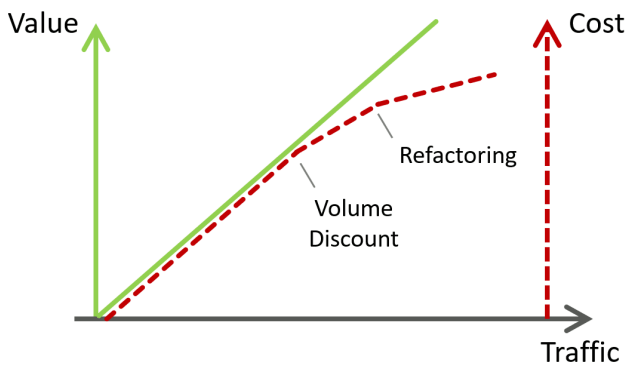
Cloud vendors have served a wide range of customers for many years by providing a growing set of options. If your internal platform is built on top of a cloud platform, you can leverage those mechanisms or gain inspiration from it. For example cloud providers offer different virtual machine types, in some cases even custom types of almost any combination of CPU and memory. Your internal platform won't be able to reproduce this many options because they require a large customer base to be financially viable. Luckily, you don't need to recreate these options but can simply base your services on these machine types, delivering the same benefits by inheritance so to speak.

Performance Tiers

Not all customers have the same performance needs or expectations, perhaps because their systems don't receive the same levels of traffic or because their end-users may have differing performance expectations. You don't need a system response time in the 10s of ms if the overall end user latency is in the high 100s of ms because end users may not even notice the extra performance. You don't need a Ferrari on dirt roads, but your users may appreciate a cheap and sturdy 4-wheel-drive car.

In-house platforms you can easily propose different tiers aligned to cloud providers' machine size. If your platform is based on serverless technologies, you can offer an entry tier without provisioned capacity and a premium tier with provisioned capacity to achieve more consistent response times.

Serverless compute illustrates how a single higher-level compute resource abstraction can address both the needs of customers with low IT budget and those of larger customers. Assuming that requests serve business value, the serverless pay-per-request model aligns your infrastructure bill to revenue or value delivered. The result is a continuous, automatic tiering system.



Serverless provides automatic tiering

Very large serverless customers may reach a volume where refactoring their application run-time to managed containers can tilt the cost curve lower than the revenue curve, harvesting economies of scale. Several cloud providers include serverless volume discounts to make even this aspect automatic.

Frequency vs. Response Time Tiers

Performance isn't a single metric. Especially for data storage, access patterns and needs may vary widely:

- Frequency of access, i.e. the number of requests over a specific time interval
- Latency, i.e. response time distribution for a single request
- Bursting, i.e. the ability to handle sudden spikes in request volume

Storage services such as Azure blob storage offer Hot, Cool, Cold, and Archive tiers with different access pattern and cost characteristics. For example, the cold storage provides low latency but is designed for infrequent access, which is reflected in a pricing model with lower storage cost and higher access cost. The hot tier has the opposite cost characteristics and is designed for frequent access.

Resilience Tiers

Consumers of IT services are well versed in specifying expected service dependability as a combination of RTO (Recovery Time Objective) and RPO (Recovery Point Objective), with the former indicating the expected length of an outage and the latter the expected data loss.



What are your customers' Service Level Objectives, Recovery Point Objectives and Recovery Time Objectives? What are yours?

Your premium customers will have higher SLOs, typically measured in the number of nines in the uptime percentage, for example "4 nines" equating to 99.99% or about 52 minutes of downtime a year.

Depending on the definition of your service, it may have to meet those SLOs itself. The cost increase from 99.9% to 99.99% is significant, often putting it out of reach for lower tier customers. A widely-used cloud service offering different SLOs are the blob storage services from the major cloud vendors. Taking Amazon Simple Storage Service (Amazon S3) as an example, the Standard Infrequent Access class is designed to offer 99.9% availability. The One Zone Infrequent Access class offers 99.5% availability at a roughly 20% lower storage

price. Azure Blob storage offers geo-redundant, locally redundant, and zone-redundant storage that ranges from 99.99% to 99% availability for read access, with Cool and Archive tiers losing one “nine” for each tier.

You can implement availability classes by varying component redundancy of your service, both by the number of replicas and their geographic distribution. A premium tier may over-provision resources to cope with failure scenarios whereas lower tiers will have just enough resources provisioned to satisfy the performance for the nominal setup. The latter will degrade performance in case of component failure.

You can address customers’ various RPO and RTO needs by providing them the tools or patterns to implement these objectives themselves. For example, a “top tier” blueprint may instruct customers to use a global load balancer across compute services and select a widely distributed storage service.

If your platform must implement these objectives, for example for your management or control plane, premium customers will receive priority during a recovery scenario and can utilize the resources already pre-provisioned for such a scenario. Lower tier customers, in contrast, will have to wait for new capacity to be built. In return, they pay a lower price.

Security Tiers

Compromising on security can sound like an indecent proposal. The reality is: there is always a limited budget whereas there is nearly no limit on the security measures you can implement.



IT Security is like insurance for your data. You only realize its value in case of disaster. Author: ChatGPT.

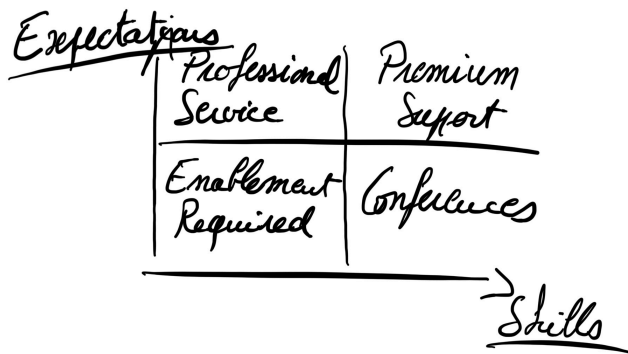
Even if the quote has been generated by a computer, it illustrates the sentiment of some security teams. The larger is the business, the higher the security concerns and the bigger the security budget. A small company being breached may not make the headlines in the newspapers and may be able to communicate with its customers in a closer manner than a large company. The small firm can accept a security risk that a larger one needs to be protected against. Hence, a successful platform offers options to meet different budgets and risk acceptance.

Some security features implemented via CI/CD pipelines shouldn’t be deactivated and wouldn’t bring much cost reductions. Containers and dependency

scanning falls into this category. However, features bringing active security at runtime like intrusion detection, detailed logs, advanced Web Application Firewall rules may be offered depending on the service tier. These features can also be managed as options, mechanism presented further down this chapter.

Support Tiers

When considering support tiers, it's helpful to **distinguish between enablement (training) support and production support**. For enablement support, cloud vendors' experience shows that smaller customers tend to have the necessary skills to adopt services faster than enterprise customers. They may be happy to learn from on-line documentation or by attending conferences. Large customers typically also have highly skilled teams, but they also have higher expectations for "high-touch" support.



Support depending on skills and customers' expectations

You might be tempted to distribute support costs via the price of your services across the entire range of customers. However, it can move the price tag too high for lower-tier customers (or be uneconomical if you charge a fixed percentage). Likewise, some higher tier customers may not want to pay for a service they don't need. Offering enablement and production support as an option can increase customer satisfaction and simplify capacity management for the support teams.

Other qualities of your platform may be tiered to offer the right price tag for the right customer, maximizing the market size for your service. A premium service may offer more durability or longer storage time for the data than an

entry-level service. Other qualities like accessibility, or extensibility can also be adapted to different tiers.

Slicing

Despite all your efforts to offer different tiers for the different customer budgets, smaller potential customers may still not be able to afford your entire platform. Deliver value in one single package provides a great ROI (Return on Investment) for customers utilizing the entire package but not for customers just tipping their toe into the water.

It's time to slice your service portfolio.

Options

The easiest way to slice an existing service is to implement feature flags, allowing customers to activate features and the corresponding price tag. Let's say that you deliver a CI/CD pipeline as a service including security checks. This pipeline has been made extremely easy to use and only requires the presence of a file in a repository to trigger it. In the version of the service supporting multiple customer tiers, the file can specify which options should be enabled or not.

Smaller services

Another way to address the CI/CD pipeline example is creating smaller services from the large services. For example, instead of offering a fully packaged CI/CD service with options, you can offer each service like the build, artifacts repository, code scanner or deployment services individually. This solution offers more flexibility for the customer that can implement her/his own pipeline composed with the individual services. This flexibility may be appreciated but can make creation of the pipeline more complicated.

Stack of services

It's possible to offer the best of both worlds by offering the pipeline services individually to be composed by the customer and also offering preconfigured pipelines for a higher price tag than the set of individual services composing it.

Most cloud vendors follow this pattern with higher-level services, for example database services that are built on top of virtual machine services and are offered at a premium.

When growing your platform, you may realize that your platform doesn't fit all of your customers' respective budgets. In such a difficult moment you may feel like you need a major redesign, but such a drastic move might not be necessary. As demonstrated in this chapter, you can use techniques to enlarge the customer base without redesigning the whole platform. When starting a new platform, it's good to have these elements in mind which can make the adoption of multiple tier or the slicing of services easier.

Platform Product Lines

Slicing and tiering a platform resembles the approach of *software product lines*, defined by [Wikipedia](#)¹ as:

Software engineering methods, tools, and techniques for creating a collection of similar software systems from a shared set of software assets using a common means of production

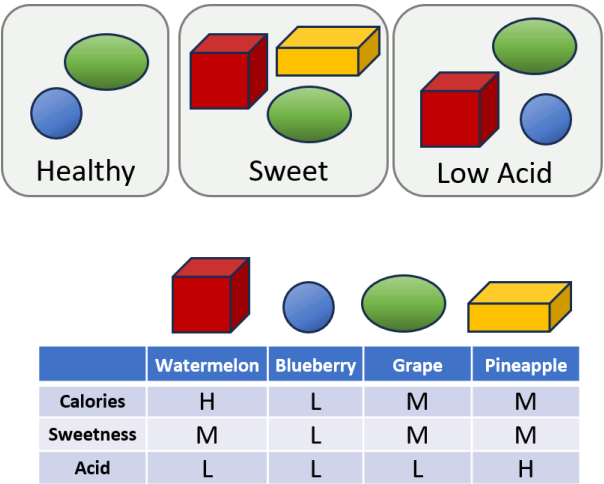
Your in-house platform can bundle existing base platform capabilities into a collection of offerings that address different customer tiers, similar to Hot/cool/cold storage tiers that combine different combinations of access frequency, latency, and availability. The precondition for a successful product line is a clean domain model that describes the platform's elements and its useful combinations.



Using a household domain like fruit salads to illustrate a concept keeps participants from raising technically detailed but irrelevant discussion points.

A fictitious [fruit salad](#) product line (this is a book about platform strategy, not nutrition) may offer a healthy fruit salad, a sweet one, or one that isn't acidic. Once you understand your fruits' characteristics (like calories, sweetness, or acidity), you can assemble a meaningful portfolio of salads:

¹https://en.wikipedia.org/wiki/Software_product_line



A fruit salad product architecture based on a clean domain model

Interestingly, cloud providers’ service portfolios have often grown over time based on customer feedback and may not exhibit a clean product line architecture. Reams of documentation and training classes for service selection are typical symptoms of this underlying issue. Internal platforms can make more assumptions, but will find it challenging to “correct” a cloud platforms underlying product architecture.

Part VII: Organizing for Platforms

Platform technology can be complex, and building one requires significant technical expertise. However, addressing sociotechnical aspects, handling the dynamics between people and untangling the organizational knot, are equally relevant. Whether you are setting up a [platform team without a platform](#) or [building one](#), leaving these aspects unaddressed spells doom for even the most technically brilliant platform.

Rather than work off service tickets, as would be the case in [traditional IT organizations](#), platform teams enable development teams without injecting themselves into the inner loop of feature delivery. But that doesn't mean that platform teams work in isolation, rather the opposite: they must be connected to parts of the organization as diverse as product engineering, operations, IT and business leadership, HR, and finance. That's why staffing and structuring a platform team isn't easy.

This part provides guidance on how to structure platform teams, the skills they need, and how they interact with other teams:

- Running a platform team can be [like running a small company](#)
- A platform needs well-defined interfaces [and align along two axes](#)
- Platform teams are well-advised to place the [customer into the center](#)
- Some platform teams can achieve benefits [without actually building a platform](#).

27. Platform, Inc.

Running a platform team is a bit like running a company

In-house platforms are built and maintained by IT teams, which either evolve out of existing IT infrastructure teams or are newly formed as cross-functional teams. The former requires present to pivot away from a typical IT project mindset whereas the latter requires a new team constellation to gel into a performing team.

A useful mental model for platforms teams resembles that of a product team. The platform team owns a product and serves customers, even if those customers are internal development teams. Platform teams struggling with this setup can benefit from behaving as if they were a small independent company. Think of yourselves as “Platform, Inc.”

A Product Organization

Although [platforms aren't regular products](#)¹, platform teams look similar to products teams, because platforms evolve just like a product evolves to stay competitive in a market. In contrast, IT is used to working in a project-based model with defined beginning and end dates, plus the assumption (or illusion) that scope is fixed. This means that when looking to set up a platform team, it's likely more helpful to look for inspiration outside traditional IT than inside.

Luckily there are plenty of examples of product organizations. Bookshelf are also full of advice on how to build a successful one, so this chapter will be a much simplified summary. In any case, Mik Kersten's excellent book *Project to Product*² should likely be required reading for anyone building a platform.

A prototypical platform team inspired by a product company would include the following roles and functions. For very large platforms, like those built by cloud providers, each service team can follow this structure.

¹<https://hbr.org/2016/04/products-to-platforms-making-the-leap>

²Mik Kersten: *Project to Product: How to Survive and Thrive in the Age of Digital Disruption with the Flow Framework*, 2018, IT Revolution

CEO

An independent team benefits from an accountable leader who helps maintain the team's "balance sheet" between headcount, timeline, and value delivery. Models like [The Cube](#) can be helpful planning and communication tools for this task.

If the platform is chargeable, the CEO will likely also have a hand in service pricing to maintain the team's profitability and long-term viability.

The CEO will also manage the communication with stakeholders. For a start-up these are the investors and the board. In corporate IT, the "investors" range from a CIO, executive sponsors, to steering committees that want to be updated on progress made, money spent, users attracted, and benefits generated.

An often overlooked function is recruiting. Platform teams aren't looking to build an entirely new company from scratch, but they still must attract talent to their cause. Attracting talent internally requires you to be well-connected in the organization and well-respected as a leader. Not every talented developer will jump to the next shiny object, so internal recruiting requires much more than drumming up excitement.



When I built the team that ended up delivering the first major in-house platform, I often spent 30% of my time on recruiting. Many candidates only responded because I contacted them directly instead of an HR department.

For most traditional organizations, external recruitment hinges very much on employer branding. At the same time, too many companies feel that they have little to offer when they are competing for talent with tech giants. I found that not to be true. "Traditional" businesses typically have rich domains, a diverse product range, international operations, and great leadership. All these can be valuable recruiting assets.



When recruiting, play to your strengths not other companies strengths!

In many team setups, the CEO would translate into the tribe lead, the Tech Lead Manager (TLM) or a single-threaded leader, depending on local parlance. In

some cases, team members report into the CEO. For example, a TLM is usually also the team's HR manager in the traditional sense. Some cloud platform providers will map this role to a service's General Manager (GM) who owns the service P&L and works closely across product evolution, pricing, and headcount.

CTO

A successful platform journey requires a clear technology strategy—after all you are building a technical product for a technical customer base. Many of the decisions presented in Parts IV and V of this book play into this strategy: What base technologies should the platform rely on? Does it need to be portable across base platforms? How can the platform evolve without causing large migration efforts? Should the platform be exclusively built from open source components? How much can be built in house vs. assembled from existing components or outsourced?

Numerous critical technical decisions need to be made. A CTO (Chief Technology Officer) should not be the person making these decisions, but they should assure that these decisions are [consciously made](#)³ and documented.

A CTO can also be a great recruiting asset, for example by being externally visible at events. Good developers want to work with other good developers, so make sure your work and the quality of your team are visible.

VP Product

The Product head (in small companies usually a vice president) owns the product roadmap, i.e. what gets built and roughly in which order. This roadmap should be based on [customer input](#), the overall product vision, and coordination with the delivery function via the VP Engineering.

In larger organizations, this role will map to Product Management (PM). An interesting question is whether platform product heads should be technical. Having worked for several large tech companies, I have seen either model work well. One company exclusively hired PM's with computer science degrees whereas in the other company that's a rare exception. Interestingly, the choice reflected the respective organization's *modus operandi*: the former was mainly technology-driven whereas the latter was customer-driven.

A PM essentially sits between users and engineering, so the decision how technical a PM should be depends on where on that spectrum you want to

³<https://architectlevator.com/architecture/important-decisions/>

position the PM. A technical PM will be “closer” to engineering and require fewer translation steps. On the flip side non-technical PMs might be more nuanced at soliciting input from users.

VP Engineering

The engineering VP is responsible for product delivery. They head the engineering teams and set timelines based on the scope and team velocity. They coordinate with the CTO on product selection and key technical decisions.

In small platform teams, the VP Engineering role translates into the development team lead.

VP Marketing

“If you build it, they will come” rarely holds true, even in corporate environments. Some enterprises [mandate platform usage](#), but acquiring customers by decree rarely works well—perhaps outside of tax authorities. So, you’d be better off making marketing a first class citizen in the team.



When my team built a platform, we posted regular updates, moderated discussion groups, established a visual identity, and held community events.

Marketing would own outbound communications, articulating platform value and vision to the user community. This translates into holding internal events, community meetings, and hackathons. They’d also create brochures, establish branding, and maintain the team’s internal on-line presence.



Building an on-line identity inside an organization is often harder than externally due to the lack of search engines, open source repositories, and easy ways to build web pages.

If you think such efforts aren’t needed inside an organization, think again. You may have thousands of internal customers, spread around the globe and sliced into divisions. If you want your platform to gain traction, you need to build mind share, which can be difficult, for example because of company policies limit spending hours on non-project activities.



Platform branding matters. You want people to call your platform by its name.

When rolling out internal platforms, we found that traditional marketing elements that are usually ignored in corporate environments can be very helpful. Giving the internal product a catchy name or printing stickers, hats, and flyers helped build an identity for the team and the platform.



Perhaps the most creative marketing ploy was to deploy a newspaper vending box (a very common sight in Germany) to distribute architecture booklets.

Support and Professional Services

Engineering teams often forget that, no matter how awesome your product is, customers need help. Self-service being a defining feature of platforms doesn't mean that the entire interface between the user and the platform team is an API call.

Platforms are often part of a corporate transformation or change agenda, meaning your users may not yet be familiar with the kind of platform and programming model that you are rolling out. They will ask questions and need support. They may also be concerned about their current role and resist the new platform.

Some customers will need more help and want to engage with what companies would call “professional services”. Virtually all successful platform companies offer (paid) professional services support, either in house or via consulting partners. For small in-house platform teams, conducting such engagements might stretch existing resources. Also, internal charge-back models might complicate charging customers for time instead of platform usage.



Even small platform teams can build a “partner ecosystem” of architects and ambassadors throughout the organization.

In this case, platform teams can follow a model inspired by cloud providers' partner ecosystem and build a network of architects or ambassadors that are part of delivery teams but also support platform adoption.

Roles aren't people

Do you need this many folks to build and manage a platform? Not all roles correspond to a single individual. Also, roles may shift as the platform grows or shifts focus. Still, the need for each role is all but guaranteed to arise, so you're better off recognizing them.



If your team doesn't have a support function, most likely your engineers will do support.

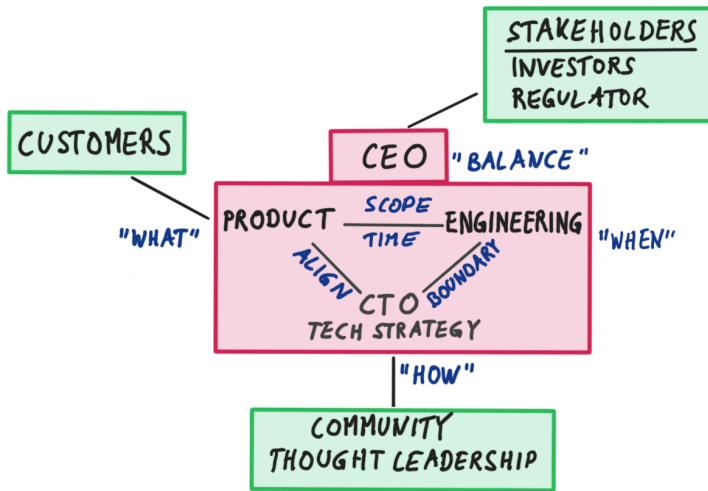
Team members juggling multiple roles can work if supporting mechanisms are in place. For example, engineers performing support can function if:

- 1) it consumes a small percentage of their time
- 2) the effort is recognized
- 3) Support requests can be managed efficiently, e.g. via a knowledge base.

In the beginning, one person might wear multiple hats, and when the time comes you can split them out to an individual. For example, your engineering VP may also be the CTO just like product and marketing may be handled by the same person in the beginning. When your team grows, and these roles likely will be split, no one will be surprised.

Fitting it Together

As with any good architecture, the functioning of a team isn't defined by the list of roles but by how they interact. The following diagram illustrates the relationship between the key team functions:



Typical interactions in a platform team structure

As expected, the product and marketing functions will interact most closely with customers, negotiating for scope and prioritization with the engineering leads. The CTO guides both sides with an overarching technology strategy and boundaries for the development teams.



A mild tension between product, who's envisioning new features, and engineering, which has to manage staffing and timelines, can be healthy.

Cloud Service Teams

Major cloud providers use similar models for teams building parts of their platform. Due to their size, each role typically exist for one individual cloud service or a small grouping of services. For example, AWS service teams are structured along the following roles:

General Manager (GM) = CEO

AWS GMs are mini-CEOs of their autonomous teams. They report service statistics and financials on a frequent basis and look for opportunities to grow their business. Some GMs have a deep technical background whereas others rely on their principal engineers.

Principal (PE) or Sr. Principal Engineer (SPE) = CTO

(Senior) Principal engineers are less involved in day-to-day coding but help shape the technical service vision or tackle particularly difficult problems.

Software Delivery Manager (SDM) = VP Eng

Reporting to the GM, the SDM manages the development teams. Often, the PM will also report into the CEO, sitting alongside the SDM.

Product Manager (PM) = VP Product

PMs act as the conduit between customer needs and engineering teams. AWS PMs rely on mechanisms like written [PRFAQ](#)⁴ to lay out the product vision.

Product Marketing Manager (PMM) = VP Marketing

PMMs take a mainly outbound role to carry the key product message to the audience.

Developer Advocate (DA) = Community management and advocacy

Not explicitly called out in our model above, DAs frequently speak at events or author blog posts. In small platform teams, this role may rest with the CTO or be spread across team members.

Keeping a platform team

Building and running platform team is only half the story. You'll have a team of folks who can drive change in large organizations, overcome existing constraints, work with modern technology, and enable large organizations with platforms. If that sounds like the dream profile of any hiring organization, then you are spot on.

No hiring strategy should go without a retention strategy:



Your best hire is the person who doesn't leave.

Team management and staff retention isn't unique to platform teams. However, with the current interest in platforms, a team of successful platform engineers is an asset that many other organizations will want to take a piece from. So, I'll share some personal thoughts on keeping high-performing teams inside a large IT organization that isn't the latest unicorn or Silicon Valley darling.

⁴<https://coda.io/@colin-bryar/working-backwards-how-write-an-amazon-pr-faq>

People wanting to steal your engineers is a high-class problem

An often-quoted conversation, sometimes attributed to Henry Ford, highlights that other organizations trying to poach your team members can actually be a good problem to have:



What if we train our people, and they leave? Well, what if you don't train them, and they stay?

If they do leave, you had a good deal

Some of the folks I hired left for significant opportunities, becoming start-up CTOs or very senior consulting positions. Some of these folks almost doubled their salary. I was happy for them and felt that I got a good deal while they were part of the team. We paid them fairly, but their value was apparently much higher.

Play your assets

Engineers can always make more money somewhere. Still, changing jobs can be risky and means leaving relationships behind. Pay your staff so that compensation isn't a concern and make them want to stay by having them be part of a great team. Don't forget that even in a traditional organization you have many more ways to reward team members than just compensation: Invite them to co-present with you at a conference; set up a lunch with a senior exec; encourage them to write a blog; invite them to lunch with other thought leaders; invite them to contribute a book chapter. These are all things that I have done, and none of them really cost anything.

Personal relationships outlast employment relationships

It's interesting to see how few people keep in touch after you leave an organization (asking you to help them find a job doesn't really count). I have always been a firm believer in building lasting relationships that don't depend on us being on the same payroll. I have become good friends with candidates who we ultimately didn't hire (or who declined). Michele and Jean-Francois used to work for me and still contribute to the books many years later.

28. The Soft-Side of Platform Teams

Platform teams also need APIs and facades

By Michele Danieli

[IT platforms](#) can only generate value when they are rolled out across an organization. However, the path to widespread adoption carries a variety of risks and challenges. A platform that's been scoped to overly specific needs will have difficulty attracting a broad audience. Other platforms might be too expensive for the perceived or real value, too rigid to evolve along users' needs, or too complex to adopt for casual users.

Internal platforms are products that have to prove themselves in their respective markets. And as with any other product, that path isn't simple.

Platform Teams need to architect themselves to cope with the challenges of building an internal platform.

Shifting gears: Platform teams are the clutch

Sitting at the crossroads between developers and operations, building platforms can feel like navigating between a rock and a hard place: developers push for higher levels of flexibility and independence whereas operations primary concerns are operational stability and security. It is often felt like a struggle between control and speed: development wanting to move faster, misinterpreting freedom for speed, but risking operational and security concerns. Meanwhile, operations aim to slow down to keep control.



A Head of Infrastructure and Operations once explained how he could sympathize with developers' desire to adopt cutting-edge technologies, but that he was more concerned about the operational and security SLA that the Board considered him accountable for.

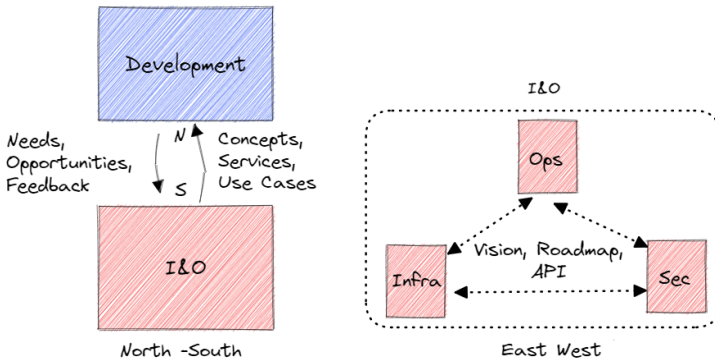
Failing to balance these forces leads to immature and incomplete platform solutions. We had **compared platforms with a transmission** that allows higher speeds. But before everything is moving smoothly, sitting between two parts that move at different speeds can be more **like the clutch on a car**¹. When shifting gears, e.g. to speed up the organization, having a clutch is exactly the right mechanism. However, if the speeds never align, the clutch will absorb the misalignment and burn out quickly.

Aligning the axes

Platform teams aren't just one more team within the organization—they challenge existing relationships along two axes:

- *East-West* inside Infrastructure and Operations (I&O), where previously isolated teams now need to collaborate more closely but may push back against challenging their status quo.
- *North-South* between operations and developers, those who build and those who consume platform services. This relationship isn't new, but the nature of it changes dramatically.

¹https://architectelevators.com/transformation/shifting_gears_clutch/



Content and Relationships are shaped by dialog

For a platform initiative to succeed, healthy relationships must avoid the friction originating from *us vs them* attitudes. Rather, teams must establish bidirectional and transparent conversations that focus on outcomes and value for the end customer.

East-West: Breaking down technology silos

Different competencies need to be coordinated to build and sustain a platform. Teams within I&O are used to splitting responsibility along technical boundaries. While this setup allows separation of concerns and specialization, it also leads to platform components being delivered with different lifecycle management practices, pushing complexity to the users. Likewise, different levels of automation make integration unnecessarily complicated and cause friction for developers adopting the platform. In the worst case, developers are presented with different interfaces or the interface falls back to a much-feared ticketing system.

A venerable piece of advice very much applies to platforms:



Don't ship your org chart!

If your team remains silo'd, you either need to defeat Conway's Law (this isn't impossible—no one went to jail for violating the law—but it's a lot of work) or

you need to regroup into a cohesive platform team with aligned goals and fluid communication.



Adding new features to a developer platform while fighting against organizational silos will lead to components that aren't well integrated.

Can an extra layer abstract the org chart?

Organizations may be tempted to adopt virtual team models, leaving the existing organizational hierarchy in place. Such setups are hard to implement because they require internal coordination and may run against egos and existing incentive structures.

The alternative is a single large platform team. But such a team can't exist in a vacuum and must relate to the other existing teams. A platform (and the team behind it) depends on cross-cutting aspects like security, network or access to base services. If the team is perceived as too far detached from the organization, access to services can become a source of friction and a bottleneck.



Infrastructure teams often hold hidden power in large IT organizations. Without the necessary network configuration, a team won't be able to launch anything.

Operating in a commercial cloud setup will soften the dependency somewhat, but the setup of virtual private clouds, IP address ranges, and NAT gateways will still depend on infrastructure teams.

Interestingly, major cloud providers manage to roll out complex systems despite being organized as small and independently cooperating teams. They achieve this for example by [hiding the complexity of the distributed system behind a relatively simple API](#).

While a platform's internal architecture may be distributed and supported by different autonomous systems, the quality of interaction between its parts is fundamental. East-West collaboration aims to enhance this collaboration, not necessarily increasing the amount of communication, which would overload teams, but its quality.

Focus on outcomes to improve collaboration quality



Before I ever heard the term *platform engineering*, my team at a large financial group set out to achieve a seamless experience for developers by automating moving code to production. Each sub-team building their automation independently - or doing things manually - caused friction in the end-user experience. We quickly learned that sociological aspects were more difficult than the architectural and technology problems we were addressing.

East-West relationships can challenge platform teams as interdependencies between units require coordination, often including negotiation. When initiating these conversations, agreeing on outcomes before trying to agree on a solution opens up space for more solutions, leaving more room for autonomy across teams and making it easier to defuse any egos.

When team dynamics are a concern, instead of pushing a master design that may raise the risk of increased confrontation, focus on expected behaviors and dependencies. Shifting the conversation can ease tensions and create psychological safety for collaborating teams: nobody is forcing a solution, and conversations center on objective improvements to make *IT faster, better and cheaper*, thereby empowering teams.



Setting the expectations for platform benefits can avoid some early fighting and move the platform forward.

It is not always sufficient and there may be situations when silos mindset and confrontational attitudes will make the conversation impossible. In such cases, it's best to minimize dependencies and isolate with an anti-corruption layer. Time will tell and clunky solutions will prove them on the ground, giving slack for splitting apart. Behind the curtains, you may still be able to put some R&D budget to explore better ways.

Organizations rolling out IT platforms should evaluate their ability to work across teams and support this effort at the organization level. Culture will be fundamental as well as the ability to adopt and maintain discipline with [mechanisms](#).

North-South: Adoption is a two-way street

Aligning the platform team for smooth collaboration is an important foundation for a successful platform rollout. Think of this aspect as the team's internal architecture. Now you have to design your team's external interface, its API and facades, so to speak.

Internal marketing or the art of dialog

Because a platform needs to enable its users, defining the product scope, collecting marketing intelligence, and testing hypotheses are critical steps in building a platform. You won't be able to (and shouldn't) anticipate all needs but instead follow a continuous process of listening and responding to the market. Timing matters: too early you will encounter a lack of understanding and urgency whereas too late you won't have time to act. Even the brightest idea if ill-timed will sink into oblivion.



Platform teams require product management skills to identify the opportunity and drive adoption.

While successful product companies are good at this, enterprise IT teams generally aren't. Constraining budget policies, project-centric planning approaches, and lack of product management skills represent just a few of the typical hurdles. IT organizations embarking on a platform journey should therefore sharpen their internal marketing skills and their willingness to engage in effective dialog, leaving behind any inner and outer group dynamics.



I have experienced many conversations that started on the wrong foot, with each party making assumptions about the other and taking strongly biased views. For example, assuming that your users are lacking technical skills may not be true. Rather, approaches may simply differ. Vice versa, the fear of being constrained or a lack of understanding of ops concerns is equally detrimental.

A platform can only succeed if organizations understand and address true needs instead of copy-pasting tools from other companies or vendor case studies. For example, a self-service developer portal will have much less impact if an

organization lacks trust between teams or requires manual hand-overs. In such cases, identifying the root cause and making necessary improvements is the only path to a better result.

Listen to your users but don't expect print-ready requirements

As with any software product, building what users tell you is well-intentioned but unlikely to succeed. Platform teams should therefore heed [Kent Beck's advice](#)²:



Build a system that the users wish they had asked for!

Instead of writing down user input as immediate requirements, examine process flows to understand impediments and why they happen. Otherwise, you might be curing symptoms instead of addressing the underlying needs.



In implementing our developer platform, we found teams deploying large units with low functional cohesion. The root cause was friction in onboarding a new software component, so we automated deployment pipeline provisioning and made that process trackable.

Methods like the “5 Why’s” and value stream mapping can help teams address the underlying problems and build to real customer needs.



At a large financial organization, I was tasked to support the adoption of a monitoring platform. Without a clear mapping of features to relevance and technical difficulty of adoption, the owner was betting on a blanket (aka “shotgun”) approach. Evaluating enablement effort and impact allowed a paced approach and better return on investment.

Unlike project activities driven by business demand or technology roll-outs, platforms depend on a strong vision and the ability to execute along a nonlinear and ambiguous path.

²https://www.enterpriseintegrationpatterns.com/ramblings/37_oopsla.html

Avoid user churn by listening to feedback

Mandating platform use can drive initial adoption, but **voluntary adoption** assures much deeper engagement, driving retention based on customer satisfaction. Unlike in the past when users had limited options outside the centrally provided solutions, today's cloud platforms and open source projects provide viable alternatives.

Traditional IT organizations are not well-versed in engaging with their users, after all, they are not operating in an open market. Changing this approach will be difficult because of the different perceptions, concerns, and established ways of doing things, but internal teams have little choice.



Failing to respond to feedback can erode your customer base. But failing to resist bad ideas may send your platform sideways.

Cloud service providers maintain community relationships and professional service organizations, which play a dual role in enabling adoption and collecting valuable feedback for constant product evolution. A cloud vendor frequently starts its announcements with *we listened to our customers* before introducing new or enhanced features, emphasizing the dynamic nature of their platforms. Marketing an internal platform should follow their example.

IT teams need to change their mindset, becoming more extroverted and able to communicate and interact with their customers. Platform teams need to establish a dialog with their users and use feedback constructively, but firmly stand when needed. Feedback on quality issues should always be evaluated and prioritized.



When our team started to build an internal development platform, we used tech talks and company events to show what we were working on and why.

Maintaining a balanced roadmap

A team might decline a feature even though it recognizes it as valuable because it is not related to the scope of the platform. Avoiding a mechanical approach to feedback evaluation (e.g. simply rejecting an issue) and preferring to be engaged

in the conversation openly but firmly, can help the organization evaluate its capabilities and the platform team avoid being side-lined. For example, a valuable feature that's a poor roadmap fit may take an alternative path, like spinning off an independent sub-project to extend the platform itself. Kubernetes as a platform has evolved and thrived also thanks to an ecosystem of projects that have enhanced its capabilities by running on top and in parallel and not overloading its core. For example, autoscaling capabilities have been enhanced by projects like KEDA or KPA in KNative.



Platform teams must embrace complete ownership of the quality and usability of their platform. They must also acquire soft skills like communication to interact more broadly within the organization.

Defining what a platform means to you

Given the buzz around buzzwords, organizations can be tempted to dive into the technical aspects of internal platforms, focusing on cool features over business impact. These teams ignore the [characteristics](#) that separate platforms from other IT services. Because each organization has unique goals and constraints, an adoption framework is useful to clarify the benefit that a platform will bring to your organization. From these goalposts, teams can lay out a path forward that includes required organizational changes, new operational procedures, and skills development.

29. The Customer-Centric Platform Team

Self-service doesn't have to be anonymous

Platforms scale by enabling users to perform tasks as self-service through portals (aka consoles), APIs, automation languages, or command-line interfaces (CLI). That doesn't mean, though that platform teams don't interact with their customers—quite the opposite. Some of the most successful platform businesses introduce new features with the statement “customer have told us”, so it behooves any platform team to be centered not just on its amazing technology but first and foremost on its customers.

Customer Engagement Models

As architects, we use models to help teams make better decisions. When building an internal developer platform for the government of Singapore, we used the following model to discuss and decide how we should interact with our internal customers:



Customer Engagement Models

Self-service

Self-service is the key mechanism that gives platforms low friction of onboarding and the ability to scale. Self-service is more than just an API to deploy a platform resource. Users will also want to monitor the status of their resources, structure their resources into projects and accounts, or obtain billing statements (if the platform charges).



An API is a user interface for developers.

The better designed the self-service APIs are, the easier it is for users to become self-sufficient with their usage. One could write an entire book about API design (one is indeed planned for this series), but just like any good user interface APIs must be consistent and set right expectations with developers. For example, if one API behaves in a certain way, developers expect to use what they learned for the next API, a characteristic that's called "the rule of least surprise".

It's easy for platform teams to underestimate how intimidating a large set of APIs can be. I have teased that the slogan "the broadest and deepest set of services" can also translate into "we overwhelm users with choice".

Setup

Traditional IT teams would often assist their internal customers in installing or configuring the software or hardware that they require. Single-tenant systems require installation and setup for each customer, working against the low-friction principle of successful platforms.

Even if the platform provides technical self-service, teams may appreciate help with setup activities that can also include training and onboarding. But if your platform team is constantly engaged in setup activities, you should likely think about ways to make the platform more automated or more intuitive.

Consulting

[Consulting or professional service](#) are a frequent element of product teams, and platform teams are no exception. Depending on the charge-back model, consulting can be a welcome vehicle to generate short-term revenue. It can also

provide a helpful feedback channel by having the platform team engaged in real-world problem solving.

Too much of a good thing can be a bad thing, also with consulting. If much of your team is engaged with customers, you are no longer a product team but a professional services organization. That [can be alright](#), but has to be a conscious choice.

Community

Your team doesn't have to be the only one supporting customers. As your user base grows, early or more advanced users can be great amplifiers for a small platform team. The platform team may facilitate or moderate such communities.

Co-Creation

The closest interaction with your customers can be to [build something together](#), for example a new platform feature. Co-creation can be a useful recruiting tool inside large organizations as you can get to know developers and their grasp of the platform.

Like any good model, this overview of engagement modes doesn't favor one option over another. Rather, it provides a list of named choices and allows teams to have intelligent discussions about their desired staff allocation across these models. It also enables discourse on how the split should shift over time.

Centers of Excellence

Many large-scale platform roll-outs foresee the creation of a “Center of Excellence” or CoE to the point where it's seen as a critical part of any transformation playbook. As discussed in *Cloud Strategy*, I have observed CoE's act as a useful vehicle to start a major internal initiative, but rarely as a sustainable model. The fact that the [Wikipedia page](#)¹ exists only in 3 languages, might give us a hint that a CoE is an equally fuzzy and overused concept.



Simply renaming centers of excellence into platform teams is a major failure mode of platform engineering.

¹https://en.wikipedia.org/wiki/Center_of_excellence

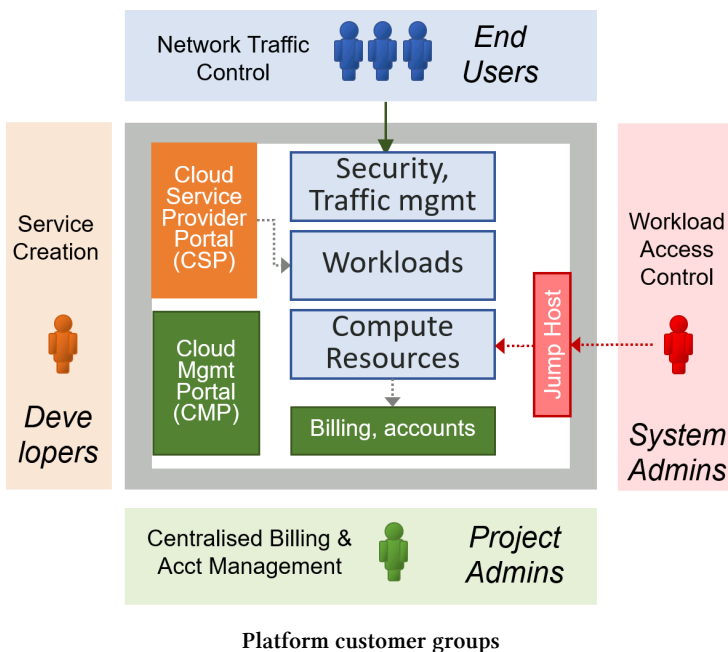
We have also observed instances where Cloud Centers of Excellence (CCoE's) were renamed as “Platform teams” in the everlasting belief that renaming teams will lead to actual changes. CCoE's can provide training and consulting and therefore resemble some interaction models of platform teams. However, they will lack most the of **functions inherent to a successful platform team**.

Which Customers?

Platform teams may serve more sets of customers than they initially realize.



When discussing platform strategy for Singapore GovTech, we used a central picture to remind us of the different customer groups we would serve:



Most internal platforms will serve at least four groups of customers with diverse needs:

Developers

The most obvious user group of users for internal platforms are developers. Developers may use the cloud console and the platform console to provision and manage services.

Project Administrators

In large organizations, project administrators manage budgets and forecasting. They often look at billing statements or desire aggregated billing and account reports.

System Administrators

To perform operational tasks, technical administrators may need direct access to compute resources like virtual machines or databases. Most organizations require special provisions to restrict the use of long-lived credentials (e.g., passwords) or direct access to customer data.

End Users

Platform teams should not forget that applications built on top of the platform serve internal or external end users. Most platforms provide network security services such as DDoS attack prevention, Content Delivery Networks, or Web Application Firewalls to manage external user traffic.

Dividing customers into these personas allows platform teams to better understand support needs. For example, developers will often need enablement support in form of training whereas system administrators will look for on-call production support in case of operational issues.

The Power of Community

As hinted in [Platform, Inc.](#), internal marketing is essential for platforms to succeed. Such marketing shouldn't just be one-way communication, but should harvest the power of community.



Your happy customers are your most powerful sales people.

Especially within an organization, prospective users will want to know how well your platform works by talking to current users. You should harness the power of word-of-mouth by creating an open forum for experience exchanges.



When launching our Agile Delivery Platform, we used the internal architect community to share progress, solicit input, and also give users a forum to share their experiences.

Although the vehicles for internal community building may seem obvious (“pizza night”), compiling a list of possible activities can be a valuable exercise for platform teams. For example, the following activities can help foster communities:

- Lunch-and-learn sessions or pizza nights where current platform users share tips and tricks
- Focused offsite events of internal conferences where representatives from different business units can meet and deliver input
- Posts on private social media platforms or private knowledge sharing instances, like Stack Overflow for teams
- Newsletters and posters

30. Platform Teams Without Platform

Standing on the giant's actual shoulders

By Jean-Francois Landreau

Managing skills in an engineering team can follow two approaches. The pyramid model assumes that you can have only a few principal engineers, a larger number of senior engineers, and an even larger base of junior engineers. With this model, the principal engineers design frameworks. The senior engineers implement them. The junior ones use those frameworks to develop product features. With slow-moving monoliths, such an approach can work, but when web-scale architecture and distributed systems turn frameworks into platforms the pyramid may be out-of-date.

A different approach, the diamond shape skills distribution, foresees a majority of senior engineers, skilled enough to use low-level frameworks (or platforms) and navigate the full code set implementing the product or service. This model gives every team member the ability to change anything, anytime, providing higher levels of agility.



When I was working in the periphery of Paris for a large software editor, one of our peers, a very successful software company on industrial design, decided to move. Some of their employees decided to leave. This side effect of companies moving around Paris is called the centrifuge. While we were excited to pick up several good candidates, they were used to the pyramid model whereas we had opted for the diamond. The candidates had been working on features for multiple years but didn't pass our tests that assumed a deep understanding of programming languages and base frameworks. Getting these candidates to transition from one model to another was something we were not prepared to do.

Transitioning from one model to another challenges most teams. A team that elevates other teams towards a diamond shape of skills can be considered a platform team. However, that team doesn't need to deliver a software platform—it's a "platform team without platform".

Human giants

A platform team without platform raises the base of the pyramid to build a diamond shape of skills distribution. The team won't achieve this by building a one-shot framework or a platform. New technologies and methodologies that invariably arrive nudge the diamond team back to the pyramid form. Whereas principal engineers stay on top of the pyramid thanks to having lived through several cycles of novelties, some senior engineers will be destabilized or resist change dropping them down to the base of the pyramid. They may have successfully adopted and gained expertise in technology only once and aren't yet used to starting over. When starting a new cycle of knowledge acquisition, you need to recognize which knowledge you can re-use and which knowledge you need to let go of. For engineers who have gone through only one cycle, it may be difficult to let go of this part of their expertise and start to learn again. This behavior is what brings them back to the bottom of the pyramid for this new iteration.

The platform team without platform concentrates the most skilled persons into one team. But this team exhibits different qualities than a traditional platform team. It must recognize a paradigm shift and the benefits the new paradigm brings. Its second quality is its ability to quickly demonstrate the new paradigm in action to recognize potential hindrances and how to overcome them. The third quality is the ability to communicate with the non-pioneers of the team and raise their understanding and expertise of the new paradigm.

The mechanisms to be used by such teams are numerous and need to be tuned to the situation. Spikes can help validate assumptions about a new technology or methodology. Pair-programming, pair designing, pair- is a good mechanism to not only arrive at a conclusion more quickly or to train junior team members, but also to improve the understanding of senior team members. Junior members of the pair throwing challenges at a senior member provides an effective form of reverse mentoring. Traditional training is an option if team members have only the challenge of finding time to learn about innovations. Architecture references, blueprints, and templates are an intermediate step toward a platform

that gives junior colleagues a starting point with less effort than building a full-fledged platform.



About 20 years ago, I had the opportunity to reboot a project that was stuck due to past technology choices and methodologies. A colleague and I started to rewrite the software with a better technology stack and better practices using eXtreme Programming. Although we could have built a framework for other team members to work on top of, we onboarded the members of the team through pair programming, aided by the extensive unit tests we had developed as a safety net.

The “human giants”, in reference to these platforms many others sit on top, are not static. We’ve seen that one of their qualities is their ability to adapt to what’s come next. Having them stuck with a platform that they will have to maintain may not be the best usage of their skills. Having them raise the larger team without platform keep them ready for the next challenge.



In the previous example, we had not created a platform but instead raised the team to use the new technology and methodology. This setup allowed us to quickly move to other teams and take on more responsibilities there. Not having to build a platform gave us this opportunity.

Cloud Adoption

When adopting the cloud, most companies suffered from an initial lack of skills. While top management often had a good understanding of the benefits, they were less concerned with implementation details. Being unable to send their entire IT team into a three-month training course, they started with a project and some pioneers to build a foundation. Once the teams gained traction, they opted for the pyramid shape and the development of an internal platform. The objective was for rest of the company to easily jump onboard.

This natural evolution runs the risk of missing the mindset change across the population. Interestingly, such change is more readily achieved when the teams feel some pain. Following the path of least resistance can mean that teams

simply map their on-premises knowledge to the cloud. As a result, the company won't harvest the value that was promised with the cloud and colleagues at the top of the pyramid losing patience might quit in the worst-case scenario. A platform team without platform supporting mindset changes and helping junior adopters get to the next level faster can dramatically improve this situation.

DevOps Transition

The transition to DevOps is another example where platforms are built. There are many documented anti-patterns in the Team topologies book. The main risk with the platform is to believe that the platform solves everything. It creates a clearer responsibility model with the API and UI of the platform as the border between applications and the platform (ops?) team. But it runs the risk of creating rigidity and having a part of the team maintaining their old mindset. The platform team without platform lives in the area of the "DevOps team with an Expiry Date". DevOps is not only about the Software Delivery Life Cycle made easy by a developer platform. It's about practices like detections of bottlenecks on the software production chain. It's about the understanding of concepts outside of pure software development by the application teams.

New Technologies

Generative AI is one example of new technology that moves many lines at the same time. Most companies realize the transformative aspects of the technology and have a desire to gain a competitive advantage with it. But most of them as well had no teams working on this technology close enough to get efficient with it on day one. The desire to concentrate on the skills and make it easy for anyone in the company to try is high. The idea to build a platform to lower the angle of the ramp certainly comes. The generative AI vendors work to make the technology easier. But there is a risk again for the adopter. If they don't get confronted with the difficulties of the technology, they'll soon not be able to recognize what's the best way for them to use it. They run the risk of being at the mercy of the platform team or the vendors to make the decisions. The member of the platform teams may also soon realize that they can do better in companies recognizing their unique skills.

Keeping the diamond shape

The diamond shape of skills has a natural tendency to revert to a pyramid one. The pyramid model is efficient, known and feels innate. But it reduces agility and the ability to move quickly. Knowing this fact is already one step toward avoiding the trap. Surveying your team on their levels of expertise in technologies and methodologies can help detect if the diamond shape is collapsing. If it does, it's time to deploy the free electron: the platform team without platform.



The platform team without platform helps people become better at fishing the new fish. It's not making the fish easier to catch.

It's by developing, showing and training the new techniques to catch the new fish. You should always have such a team in waiting mode, detecting the new trends and monitoring the skills shape, ready to intervene to maintain the diamond shape.